ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ

ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΒΙΟΪΑΤΡΙΚΗΣ

Επεξεργασία και Ανάλυση Σημάτων φορητού Ηλεκτροεγκεφαλογράφου (EEG) σε Raspberry Pi

ΔΕΝΤΣΑΣ ΠΑΝΑΓΙΩΤΗΣ
Αριθμός Μητρώου: 21388015

Επιβλέπων Καθηγητής
ΙΩΑΝΝΗΣ ΚΑΚΚΟΣ, Επίκουρος Καθηγητής

Αθήνα 2/10/2026

Η Τριμελής Εξεταστική Επιτροπή

Ο Επιβλέπων Καθηγητής

Ιωάννης Κάκκος	Αικατερίνη Σκουρολιάκου	Παντελεήμων Ασβεστάς
Επίκουρος Καθηγητής	Καθηγήτρια	Καθηγητής
IOANNIS KAKKOS Digitally signed by IOANNIS KAKKOS Date: 2026.10.19 10:21:32 +03'00'	AIKATERINI SKOUROLIAKOU Digitally signed by AIKATERINI SKOUROLIAKOU Date: 2026.10.19 13:37:56 +03'00'	Digitally signed by PANTELEIMON ASVESTAS Date: 2026.10.20 10:54:01 +03'00'

ΔΗΛΩΣΗ ΣΥΓΓΡΑΦΕΑ ΔΙΠΛΩΜΑΤΙΚΗΣ ΕΡΓΑΣΙΑΣ

Ο υπογράφων Δέντσας Παναγιώτης του Κωνσταντίνου, με αριθμό μητρώου 21388015 φοιτητής του Τμήματος Μηχανικών Βιοϊατρικής της Σχολής Μηχανικών του Πανεπιστημίου Δυτικής Αττικής, δηλώνω υπεύθυνα ότι:

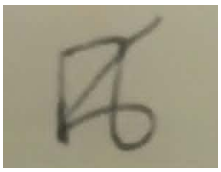
«Είμαι συγγραφέας αυτής της διπλωματικής εργασίας και κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, οι όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε ακριβώς είτε παραφρασμένες, αναφέρονται στο σύνολό τους, με πλήρη αναφορά στους συγγραφείς, τον εκδοτικό οίκο ή το περιοδικό, συμπεριλαμβανομένων και των πηγών που ενδεχομένως χρησιμοποιήθηκαν από το διαδίκτυο. Επίσης, βεβαιώνω ότι αυτή η εργασία έχει συγγραφεί από μένα αποκλειστικά και αποτελεί προϊόν πνευματικής ιδιοκτησίας τόσο δικής μου, όσο και του Ιδρύματος.

Παράβαση της ανωτέρω ακαδημαϊκής μου ευθύνης αποτελεί ουσιώδη λόγο για την ανάκληση του διπλώματός μου».

Ημερομηνία

2/10/2026

Ο/Η Δηλών/ούσα



Ευχαριστίες:
Αρχικά, θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή της διπλωματικής μου εργασίας, κ. Ιωάννη Κάκκο, για την υλικοτεχνική του υποστήριξη, καθώς και για την πολύτιμη καθοδήγηση και βοήθειά του κατά τον σχεδιασμό και την εκπόνηση της παρούσας εργασίας.

Επίσης, θα ήθελα να ευχαριστήσω τον κ. Παντελή Ασβεστά για την παραχώρηση εξοπλισμού από τα εργαστήρια, καθώς και τον κ. Σταύρο Μηλούλη για την παροχή του λογισμικού INSPiRE και τη συνδρομή του κατά τα πρώτα στάδια της έρευνας.

Επιπλέον, οφείλω ένα μεγάλο ευχαριστώ σε όλα τα μέλη του διδακτικού προσωπικού του Τμήματος Μηχανικών Βιοϊατρικής. Η αφοσίωσή τους στο έργο τους και η αγάπη τους για τη μετάδοση της γνώσης μάς δίδαξαν και μάς βοήθησαν να εξελιχθούμε, αποκτώντας τόσο εξειδικευμένες τεχνικές δεξιότητες όσο και ευρύτερη ακαδημαϊκή παιδεία.

Τέλος, θα ήθελα να εκφράσω τη βαθιά μου ευγνωμοσύνη στην οικογένειά μου και στους ανθρώπους μου, οι οποίοι στάθηκαν δίπλα μου σε όλη τη διάρκεια των πανεπιστημιακών μου σπουδών, προσφέροντάς μου την αμέριστη ηθική και υλική τους στήριξη.

ΠΕΡΙΛΗΨΗ

Στόχος της παρούσας διπλωματικής εργασίας είναι η ανάπτυξη ενός συστήματος που επεξεργάζεται και αναλύει δεδομένα από ένα φορητό ηλεκτροεγκεφαλογράφο (EEG) χρησιμοποιώντας ένα Raspberry Pi 4 Model B ως κεντρική υπολογιστική μονάδα. Το EEG μεταφέρει δεδομένα μέσω ενός RF Bluetooth dongle στο Raspberry Pi, όπου μπορούν να εφαρμοστούν φίλτρα μέσω επιλογής που γίνεται στην διεπαφή χρήστη (interface). Αυτά τα φίλτρα αφαιρούν τυχόν ψευδενδείξεις - όπως συσπάσεις των μυών της γνάθου και παρεμβολές 50Hz από την παροχή ρεύματος - αφήνοντας μόνο τις χρήσιμες συχνότητες εγκεφαλικών κυμάτων. Το σύστημα διαθέτει δύο ανεξάρτητα φίλτρα: το ένα εφαρμόζεται στα αποθηκευμένα δεδομένα, ενώ το άλλο είναι υπολογιστικά ελαφρύτερο και χρησιμοποιείται σε πραγματικό χρόνο. Στη συνέχεια, τα χαρακτηριστικά εξάγονται σε πραγματικό χρόνο κάθε 0,5 δευτερόλεπτα από ένα μικρό τμήμα δεδομένων που αποτελείται από 130 δείγματα. Αυτά τα χαρακτηριστικά εμφανίζονται γραφικά στη διεπαφή χρήστη. Με βάση αυτά τα χαρακτηριστικά, το σύστημα λαμβάνει μια απόφαση, ορίζοντας την κατάσταση είτε σε ενεργή είτε σε ανενεργή. Όταν η κατάσταση είναι ενεργή, παρέχεται τάση σε μια συγκεκριμένη ακίδα στο Raspberry Pi, ανάβοντας ένα κόκκινο LED συνδεδεμένο σε ένα εξωτερικό κύκλωμα.

Τα αποτελέσματα αυτής της διπλωματικής εργασίας παρέχουν ισχυρά θεμέλια για τη συνεχή ανάπτυξη του έργου, προσφέροντας πολύτιμα εργαλεία δοκιμών που θα διευκολύνουν τη μελλοντική έρευνα και εφαρμογή. Συνολικά, οι στόχοι της μελέτης επιτεύχθηκαν με επιτυχία. Το σύστημα είναι πλήρως ικανό να λαμβάνει, να επεξεργάζεται και να αποθηκεύει δεδομένα του EEG, ενώ ταυτόχρονα εκτελεί αποφάσεις σε πραγματικό χρόνο. Ωστόσο, πρέπει να σημειωθεί ότι οι υπολογιστικοί πόροι του Raspberry Pi 4 Model B λειτουργούν κοντά στη μέγιστη χωρητικότητά τους. Επομένως, για οποιαδήποτε μελλοντική ανάπτυξη ή επέκταση του συστήματος, πιθανότατα θα χρειαστεί αναβάθμιση υλικού.

Λέξεις-Κλειδιά: Raspberry Pi, BCI, EEG, OpenBCI, σύστημα χαμηλού κόστους, πλακέτα Cyton +Daisy

ABSTRACT

The aim of this thesis is to develop a system that processes and analyzes data from a portable electroencephalograph (EEG) using a Raspberry Pi 4 Model B as the central computing unit. The EEG transfers data via an RF Bluetooth dongle to the Raspberry Pi, where filters can be applied through the user interface. These filters remove artifacts—such as jaw muscle contractions and 50Hz power line interference—leaving only the useful brainwave frequencies. The system features two independent filters: one is applied to saved data, while the other is computationally lighter and utilized in real time. Subsequently, features are extracted in real time every 0.5 seconds from small data segments consisting of 130 samples. These features are displayed graphically using bars on the user interface. Based on these features, the system makes a decision, setting the state to either active or inactive. When the state is active, voltage is supplied to a specific pin on the Raspberry Pi, illuminating a red LED connected to an external circuit.

The outcomes of this thesis provide a robust foundation for the continued development of the project, offering valuable testing tools that will streamline future research and implementation. Overall, the aims of the study were successfully achieved. The system is fully capable of receiving, processing, and storing the EEG data, while simultaneously executing real-time decisions. However, it must be noted that the computational resources of the Raspberry Pi 4 Model B are operating near their maximum capacity. Therefore, for any future development or expansion of the system, a hardware upgrade will likely be necessary.

Keywords: Raspberry Pi, BCI, EEG, OpenBCI, low-cost system, Cyton +Daisy board

Πίνακας περιεχομένων

ΠΕΡΙΛΗΨΗ 5
ABSTRACT 6
Πίνακας περιεχομένων 7
Πίνακας Εικόνων: 8
1. ΕΙΣΑΓΩΓΗ 9
1.1 ΥΠΟΒΑΘΡΟ ΚΑΙ ΕΠΙΛΟΓΗ ΘΕΜΑΤΟΣ 9
1.2 ΣΚΟΠΟΣ ΚΑΙ ΑΝΤΙΚΕΙΜΕΝΟ ΕΡΓΑΣΙΑΣ 10
1.3 ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ 12
1.3.1 Εγκέφαλος και εγκεφαλικά κύματα 12
1.3.2 Συστήματα EEG 13
1.3.3 Φιλτράρισμα 14
1.3.4 Ζώνες ισχύος 15
1.4 ΔΟΜΗ ΤΗΣ ΕΡΓΑΣΙΑΣ 16
2. ΥΛΙΚΑ 17
2.1 Raspberry Pi 4 model b 18
2.2 Το Σύστημα EEG Ultracortex Mark IV 19
2.2.1 Αξιολόγηση Αξιοπιστίας του συστήματος της OpenBCI 23
2.3 Λογισμικό που χρησιμοποιήθηκε 24
3. Μεθοδολογία 29
3.1 Βήματα υλοποίησης: 29
3.2 ΠΡΑΚΤΙΚΗ ΥΛΟΠΟΙΗΣΗ: 30
3.2.1 Φόρτωση firmware 30
3.2.2 Επικοινωνία Raspberry Pi με τον υπολογιστή 31
3.2.3 Περιβάλλον διεργασίας 33
3.2.4 Λειτουργικές τροποποιήσεις 33
3.2.5 Λογισμικό φιλτραρίσματος 34
3.2.6 Εξαγωγή χαρακτηριστικών 37
3.2.7 Ταξινόμηση (Classification) 37
3.2.8 Διεπαφή Ελέγχου 37
3.2.9 Έλεγχος κατάστασης συστήματος 38
3.2.10 Απεικονίσεις 38
3.3 Προγράμματα πειραματισμού-ελέγχου 39
3.4 Κύκλωμα led 41
3.5 Προβλήματα και Προκλήσεις που Αντιμετωπίστηκαν 43
3.6 Ανάλυση κίνδυνων και περιορισμών 44
4. Αποτελέσματα 46
5. ΣΥΖΗΤΗΣΗ & ΣΥΜΠΕΡΑΣΜΑΤΑ 55
5.1 ΣΥΖΗΤΗΣΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ 55
5.1.1 Ερμηνεία βασικών λειτουργικών ευρημάτων 55
Επικοινωνία Raspberry Pi-υπολογιστή 55
Επικοινωνία Raspberry Pi-dongle-EEG 55
Raspberry Pi και εσωτερικο λογισμικο 55
Φιλτράρισμα και απεικόνιση 55
Εξαγωγή χαρακτηριστικών και αποφάσεις 56
Έξοδος Led 56
5.1.2 Σύγκριση με τον αρχικό στόχο της εργασίας 56
5.2 ΠΡΟΤΑΣΕΙΣ ΓΙΑ ΜΕΛΛΟΝΤΙΚΗ ΕΞΕΛΙΞΗ 57
5.3 ΓΕΝΙΚΑ ΣΥΜΠΕΡΑΣΜΑΤΑ 58
Αναφορές - Πηγές 60
Παράρτημα 1: Κώδικας 63

Πίνακας Εικόνων:

Εικόνα 1.1:Διάγραμμα αρχιτεκτονικής συστήματος 11
Εικόνα 1.2: Αναπαράσταση συχνοτήτων εγκεφαλικών κυμάτων. 13
Εικόνα 2.1: Φωτογραφίες του EEG που χρησιμοποιήθηκε (back view).....20
Εικόνα 2.2: Φωτογραφίες του EEG που χρησιμοποιήθηκε (top view).....20
Εικόνα 2.3: Δομή πακέτου πληροφοριών του EEG 22
Εικόνα 2.4: USB Dongle της Open BCI..... 22
Εικόνα 2.5: USB Dongle της Open BCI πάνω στο Raspberry Pi εν λειτουργία 23
Εικόνα 2.6: Στιγμιότυπο οθόνης του κώδικα python στο inspire_main. 25
Εικόνα 2.7 Αποτυχία επικοινωνίας με το USB dongle..... 27
Εικόνα 2.8 Αποτυχία επικοινωνίας με το σύστημα EEG. 27
Εικόνα 2.9 Επιτυχημένη διασύνδεση και επικοινωνία του εξοπλισμού.....28
Εικόνα 3.1: Στιγμιότυπο οθόνης του Arduino IDE (1.8.19) κατά την εγκατάσταση του λογισμικού στο firmware. 30
Εικόνα 3.2: Στιγμιότυπο οθόνης της επιφάνειας εργασίας του Raspberry Pi 33
Εικόνα 3.3: Στιγμιότυπο οθόνης απο το παράθυρο εντολών κατά την καταγραφή δεδομένων, του μηνύματος του inspire_utils 38
Εικόνα 3.4: Στιγμιότυπο οθόνης της απεικόνισης των χαρακτηριστικών στην διεπαφή (interface) χρήστη 39
Εικόνα 3.5: Στιγμιότυπο οθόνης με την λειτουργία του inspire_offlinefilter.py στο παράθυρο εντολών 40
Εικόνα 3.6: Το κύκλωμα led ελεγχόμενο από το Raspberry Pi κλειστό. 42
Εικόνα 3.7: Το κύκλωμα led ελεγχόμενο από το Raspberry Pi ανοικτο. 42
Εικόνα 3.8: Διάταξη ακροδεκτών (GPIO) του Raspberry Pi και αντιστοίχιση λειτουργιών.. 43
Εικόνα 4.1: Στιγμιότυπο οθόνης που παρουσιάζεται η διεπαφή χρήστη (interface) του προγράμματος καθώς εκτελείται καταγραφή.....47
Εικόνα 4.2: Παράθυρο εντολών κατά την καταγραφή δεδομένων 47
Εικόνα 4.3: Διάγραμμα που παρήγαγε το inspiret_singlefiltersvisual.py 48
Εικόνα 4.4: διάγραμμα που παρήγαγε το inspiret_multyvisual.py 49
Εικόνα 4.5: Διάγραμμα που παρήγαγε το inspiret_frequency.py.....49
Εικόνα 4.6: Διάγραμμα που παρήγαγε το inspiret_testrealtime.py 50
Εικόνα 4.7: Διάγραμμα που παρήγαγε το inspiret_testrealtime.py του αποτελέσματος του φίλτρου πραγματικού χρόνου χωρίς history buffer 51
Εικόνα 4.8: Διάγραμμα που παρήγαγε το inspiret_realtimefft_visualizer.py 51
Εικόνα 4.9: Διάγραμμα που παρήγαγε το inspiret_realtimefft_visualizermulty.py…….....52
Εικόνα 4.10: Διάγραμμα που παρήγαγε το inspiret_present.py 53
Εικόνα 4.11: Διάγραμμα που παρήγαγε το inspiret_present.py 53
Εικόνα 4.12: Διάγραμμα που παρήγαγε το inspiret_presentrealtime.py.....54
Εικόνα 4.13: Διάγραμμα που παρήγαγε το inspiret_presentrealtime.py.....54

1. ΕΙΣΑΓΩΓΗ

1.1 ΥΠΟΒΑΘΡΟ ΚΑΙ ΕΠΙΛΟΓΗ ΘΕΜΑΤΟΣ

Η φορεσιμότητα (wearability) των διεπαφών εγκεφάλου υπολογιστεί (BCI) θεωρείται πλέον ζωτικής σημασίας για την αποτελεσματική καθημερινή τους χρήση. Οι φορητοί, φθηνοί και εύκολοι στη χρήση ήλεκτροεγκεφαλογράφοι (EEG) επιτρέπουν την χρήση τους σε ένα ευρύ κοινό. Τα συστήματα αυτά έχουν αρχίσει να ξεφεύγουν από την περιορισμένη χρήση τους ερευνητικά και εργαστηριακά, με τη δυνατότητα να βοηθήσουν πολλούς ανθρώπους με κινητικές αναπηρίες. Ενώ στο παρελθόν η χρήση των EEG ήταν σχεδόν αποκλειστικά για διαγνωστικούς σκοπούς, καθώς η τεχνολογία ωριμάζει, μπορεί να εισέλθει σαν ένα «χειριστήριο» για τις ηλεκτρονικές συσκευές της καθημερινότητας. Ένα εργαλείο για τη γρηγορότερη ωρίμανση και πρόοδο της τεχνολογίας αυτής είναι η αύξηση της πρόσβασημοτητας και της δημοφιλίας της, κάτι που επιτρέπει ο σχετικά φθηνός και τύπου ανοιχτού κώδικα (open source) OpenBCI.

Τα παραδοσιακά συστήματα EEG αντιμετωπίζουν αρκετά εμπόδια, ώστε να μπορούν να χρησιμοποιηθούν και να ενταχθούν στην καθημερινή ζωή. Οι μεγάλοι, ογκώδεις EEG με υγρά ηλεκτρόδια προκαλούν δυσφορία και κάνουν τις μακροχρόνιες καταγραφές δύσκολες. Το ογκώδες μέγεθος τους και των ενισχυτών κουράζει τον ασθενή, το τζελ των αισθητήρων στεγνώνει αλλάζοντας την αγωγιμότητα και τα πολλά, χοντρά καλώδια προκαλούν κίνδυνο. Αυτά τα προβλήματα καθιστούν αδύνατον τέτοιου είδους συστήματα να χρησιμοποιηθούν στην καθημερινότητα.

Έπειτα, συγκρίνοντάς τα με τα εμφυτεύσιμα (invasive) συστήματα BCI, πρέπει να ομολογηθεί ότι υπερέχει πλεονέκτημα σε πολύ βασικό επίπεδο. Το σήμα που λαμβάνουν είναι πιο ισχυρό, διότι αποφεύγουν την εξασθένιση, καθώς είτε τοποθετούνται πάνω στο κρανίο είτε κατευθείαν πάνω στον εγκέφαλο. Αυτό δίνει καλύτερο λόγο σήματος προς θόρυβο (SNR), αποφεύγει αρκετές πηγές θορύβου του σώματος και προσφέρει υψηλότερη χωρική ανάλυση. Ακόμη, τα ηλεκτρόδια βρίσκονται σε σταθερές θέσεις. Όλα αυτά τα χαρακτηριστικά κάνουν την επιλογή τους ιδιαίτερα δελεαστική ως προς την έρευνα, αλλά ο κίνδυνος και η καταπόνηση των εγχειρήσεων, καθώς και οι επιπλοκές τους, κάνουν την ευρεία διάδοση τέτοιας τεχνολογίας, προς το παρόν, φαντασία.

Για την πρακτική χρήση ενός συστήματος BCI είναι σίγουρα απαραίτητη η μικρή καθυστέρηση, η λειτουργία σε πραγματικές συνθήκες και η αξιοπιστία στη σωστή λειτουργία. Σε αυτήν την εργασία, ο σκοπός ήταν τα πρώτα βήματα για την ανάπτυξη ενός τέτοιου συστήματος. Το σύστημα χρησιμοποιεί έναν φορητό, φθηνό EEG, ξηρά ηλεκτρόδια, τοπική επεξεργασία και λήψη αποφάσεων με ένα Raspberry Pi. Η χρήση ξηρών ηλεκτροδίων προσφέρει ευχρηστία και η τοπική επεξεργασία συμβάλλει στην ανεξαρτησία από εξωγενή προβλήματα επικοινωνίας.

Αρχικά, το σύστημα εγκαταστάθηκε εκ νέου και προσαρμόστηκε, ώστε να λειτουργεί σε ένα Raspberry Pi 4 model B, προσαρμοσμένο με τις πιο πρόσφατες εκδόσεις του λογισμικού. Έπειτα, προστέθηκαν επιπλέον στοιχεία ώστε να διαχειρίζεται η πληροφορία του EEG. Το σύστημα φιλτράρει στο πεδίο των συχνοτήτων τα αρχικά δεδομένα, αποθηκεύει τα φιλτραρισμένα δεδομένα, εξάγει τα χαρακτηριστικά των δεδομένων και με τα χαρακτηριστικά αυτά εξάγει μια απόφαση. Περαιτέρω, αρκετά εργαλεία έχουν δημιουργηθεί για την υποστήριξη και τον έλεγχο αυτής της διαδικασίας του φιλτραρίσματος. Πρέπει επίσης να αναφερθεί ότι η παρούσα διαδικασία αποτελεί

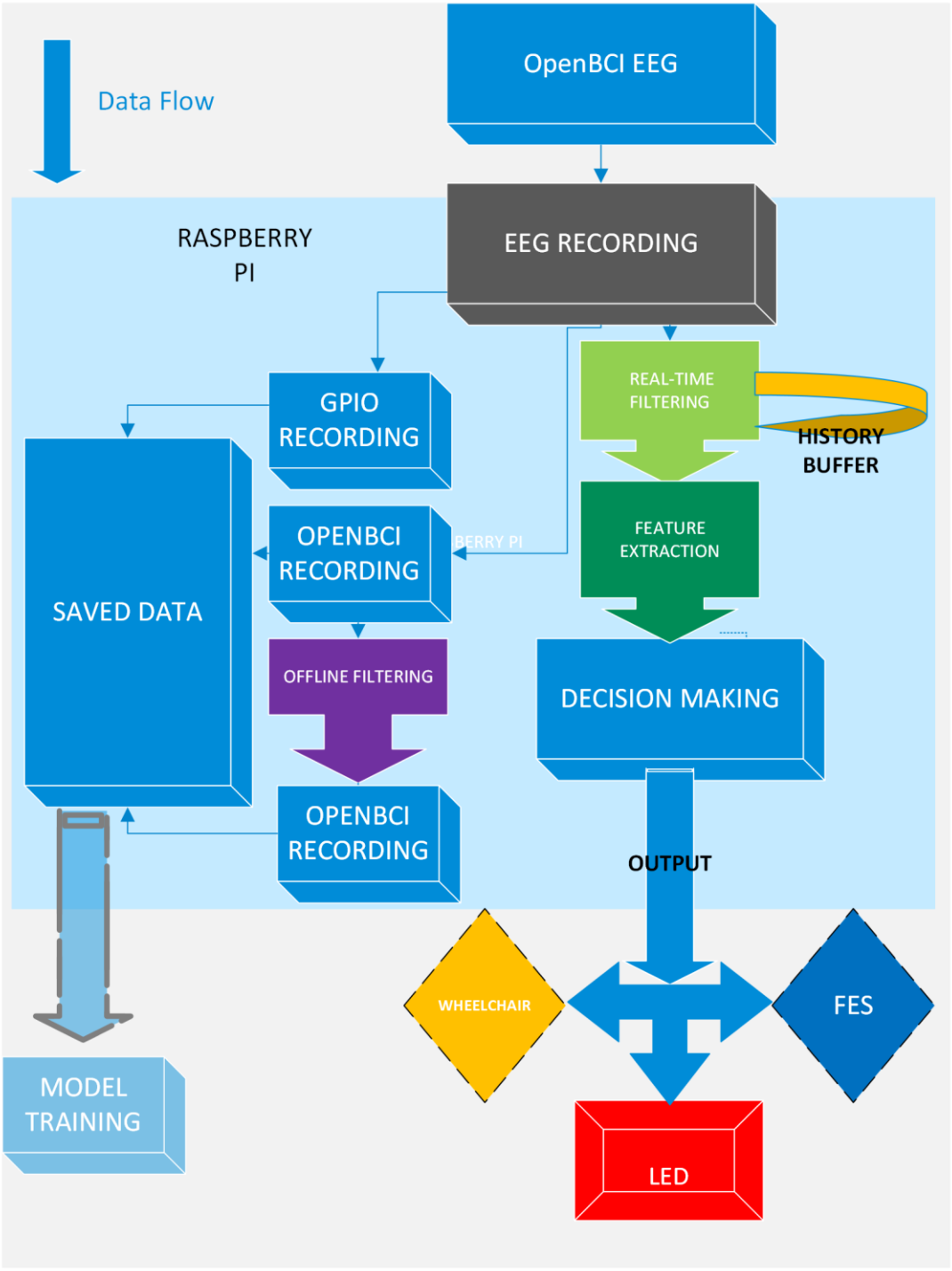
το θεμέλιο μιας πραγματικής διαδικασίας και όχι ένα αυτοτελές μέρος. Αποτελεί ιδιαίτερο ενδιαφέρον η δυνατότητα προσαρμογής πολυτροπικής λειτουργίας, χρησιμοποιώντας δεδομένα όχι μόνο από τον εν λόγω EEGαλλά και από άλλα συστήματα.

1.2 ΣΚΟΠΟΣ ΚΑΙ ΑΝΤΙΚΕΙΜΕΝΟ ΕΡΓΑΣΙΑΣ

Βασικός σκοπός της παρούσας διπλωματικής εργασίας είναι η διερεύνηση και η επιβεβαίωση της βιωσιμότητας ενός φορητού συστήματος Διεπαφής Εγκεφάλου-Υπολογιστή (Brain-Computer Interface της OpenBCI) χαμηλού κόστους, εκτελώντας της διεργασίες με ένα φθηνό υπολογιστικό σύστημα Raspberry Pi. Η εργασία στοχεύει να δείξει ότι είναι δυνατό, με τεχνολογίες σχετικά μικρού κόστους, να υλοποιηθεί ένα BCI το οποίο μπορεί να φιλτράρει τα αρχικά σήματα του EEG, να τα αποθηκεύει και να εξάγει από αυτά χαρακτηριστικά και αποφάσεις σε πραγματικό χρόνο, με ικανοποιητική ακρίβεια.

Για την επίτευξη του παραπάνω σκοπού, το αντικείμενο της εργασίας επικεντρώνεται στην ανάπτυξη, διασύνδεση και προγραμματιστική διαχείριση μιας ολοκληρωμένης πλατφόρμας υλικού και λογισμικού. Πιο συγκεκριμένα, το τεχνικό αντικείμενο αναλύεται στους εξής άξονες:

- Διασύνδεση Υλικού (Hardware Integration): Η εγκατάσταση του κατάλληλου λογισμικού της 3D-εκτυπώσιμης κάσκας OpenBCI Ultracortex Mark IV, εξοπλισμένης με ξηρά ηλεκτρόδια (dry electrodes). Παράλληλα, περιλαμβάνει την ασύρματη σύζευξη της πλακέτας καταγραφής 16 καναλιών (Cyton + Daisy Module) με τον μικροϋπολογιστή Raspberry Pi, μέσω του εξειδικευμένου πρωτοκόλλου ραδιοσυχνοτήτων του USB Dongle.
- Λήψη Δεδομένων (Data Acquisition): Το λογισμικό, με κώδικα σε γλώσσα Python στο περιβάλλον του Raspberry Pi, πραγματοποιεί την ανάγνωση, αποκωδικοποίηση (πακέτα 33 bytes) και καταγραφή της ροής των raw δεδομένων EEG με ρυθμό δειγματοληψίας 125 Hz, έχοντας εγκαταστήσει το κατάλληλο λογισμικό γι' αυτό.
- Ψηφιακή Επεξεργασία και Αποθορυβοποίηση (Signal Processing): Ο σχεδιασμός και η εφαρμογή ψηφιακών φίλτρων τοπικά στο Raspberry Pi, καθώς και η αποθήκευση των δεδομένων πριν και μετά την επεξεργασία. Αυτό το στάδιο είναι κρίσιμο για την αντιμετώπιση του υψηλού θορύβου (artifacts) και της αυξημένης εμπέδησης (impedance) που εισάγει η χρήση των ξηρών ηλεκτροδίων.
- Εξαγωγή χαρακτηριστικών και λήψη απόφασης: Το σύστημα, σε πραγματικό χρόνο, λαμβάνει τα τελευταία δεδομένα που εξήγαγε το EEG και εξάγει από αυτά κάποια χαρακτηριστικά, τα οποία επίσης απεικονίζονται με γράφημα. Στη συνέχεια, τα χαρακτηριστικά αυτά χρησιμοποιούνται για τη λήψη μιας απόφασης. Για την ένδειξη αυτής της απόφασης, στέλνεται τάση στα pins του Raspberry Pi, κάνοντας ένα κόκκινο LED να ανάψει.



Εικόνα 1.1: Διάγραμμα αρχιτεκτονικής συστήματος

Παράλληλα με τα υπόλοιπα βήματα, έχουν επίσης αναπτυχθεί βοηθητικά εργαλεία. Πρόκειται για ανεξάρτητα προγράμματα, τα οποία βοηθούν τη λειτουργία του προγράμματος απεικονίζοντας τα δεδομένα των καταγραφών για ανάλυση, είτε στο πεδίο του χρόνου είτε στο πεδίο των συχνοτήτων. Ακόμη, κατά τη λειτουργία του κύριου προγράμματος, ένα υποπρόγραμμα ελέγχει και απεικονίζει την κατάσταση του επεξεργαστή.

Μέσα από αυτά τα βήματα, η εργασία φιλοδοξεί να παραδώσει ένα λειτουργικό «πρωτότυπο» φορητού EEG, αναδεικνύοντας τις τεχνικές προκλήσεις, τους περιορισμούς, αλλά και τις τεράστιες δυνατότητες των ανοιχτών τεχνολογιών στην επιστήμη της νευρομηχανικής.

1.3 ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ

1.3.1 Εγκέφαλος και εγκεφαλικά κύματα

Σε κυτταρικό επίπεδο, είναι σημαντικό να διευκρινιστεί ότι το σήμα του ΗΕΓ δεν αντικατοπτρίζει τα δυναμικά ενέργειας (action potentials) των μεμονωμένων νευρώνων. Αντιθέτως, πηγάζει από τα εξωκυττάρια ρεύματα που δημιουργούνται από τα μετασυναπτικά δυναμικά. Οι πρωταρχικές γεννήτριες αυτών των σημάτων είναι οι πυραμιδικοί νευρώνες του εγκεφαλικού φλοιού. Λόγω της ανατομικής τους δομής, οι άξονες αυτών των κυττάρων διατάσσονται παράλληλα μεταξύ τους και κάθετα προς την επιφάνεια του φλοιού, λειτουργώντας ηλεκτροφυσιολογικά ως μικροσκοπικά ηλεκτρικά δίπολα (Kandel et al., 2012).

Η ηλεκτρική δραστηριότητα του εγκεφάλου, η οποία καταγράφεται μέσω του συστήματος EEG, κατατάσσεται σε πέντε κύριες ζώνες συχνοτήτων. Κάθε μία από αυτές τις ζώνες αντιστοιχεί σε μια συγκεκριμένη κατάσταση συνείδησης, γνωστικής λειτουργίας ή βαθμού εγρήγορσης (Bear et al., 2020).

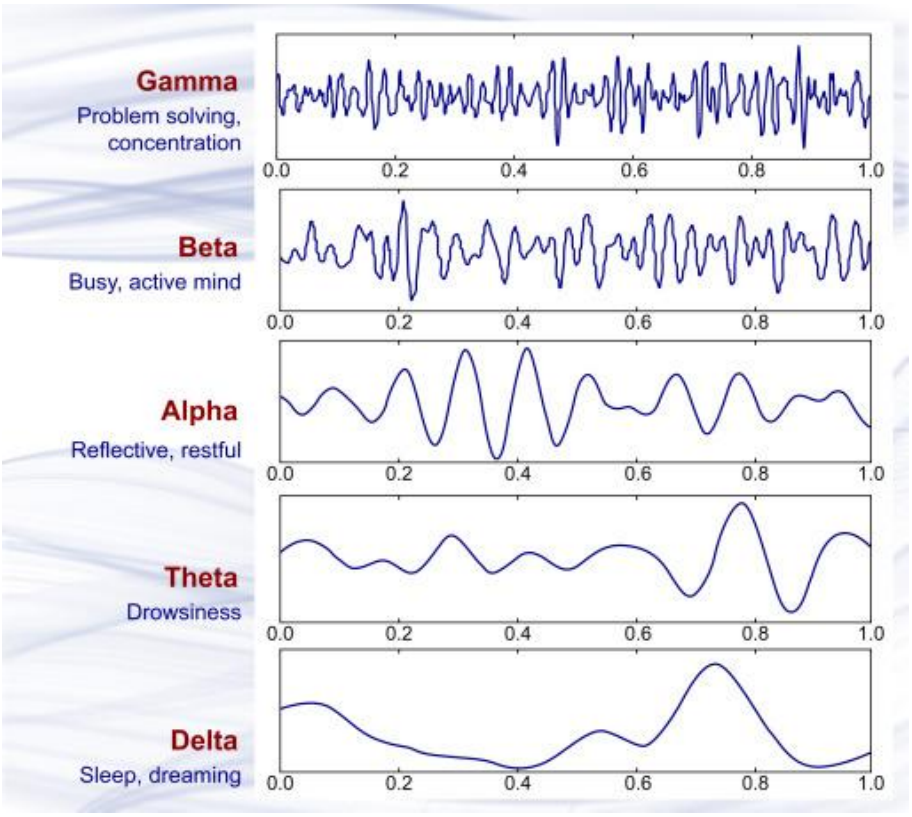
Στο κατώτερο άκρο του φάσματος των συχνοτήτων συναντώνται τα κύματα Δέλτα (0.5-4 Hz), τα οποία εντοπίζονται εντονότερα κατά τη διάρκεια του βαθύ ύπνου. Ακολουθούν τα κύματα Θήτα (4-8 Hz), τα οποία θεωρούνται ως γέφυρα μεταξύ ύπνου και εγρήγορσης, ενώ σχετίζονται με την εδραίωση της μνήμης και τη βαθιά χαλάρωση. Στη συνέχεια, τα κύματα Άλφα (8-13 Hz) σηματοδοτούν συνήθως μια κατάσταση ηρεμίας με κλειστά τα μάτια, ενώ τα κύματα Βήτα (13-30 Hz) σχετίζονται με πιο ενεργές καταστάσεις, όπως η συνειδητή σκέψη και η αλληλεπίδραση με το περιβάλλον. Τέλος, όλες οι συχνότητες άνω των 30 Hz ταξινομούνται ως κύματα Γάμμα, τα οποία συνδέονται με την ανώτερη γνωστική επεξεργασία, την υψηλή συγκέντρωση και τις πολύπλοκες νοητικές διεργασίες (Buzsáki, 2006).

Θα πρέπει, βέβαια, να σημειωθεί ότι οι παραπάνω αποτελούν γενικές συσχετίσεις και ενδέχεται να διαφέρουν ανάλογα με την περιοχή του εγκεφάλου και την εκάστοτε κατάσταση του υποκειμένου. Στη συνέχεια, παρατίθενται ορισμένες επιπλέον πληροφορίες σχετικά με τις ζώνες Βήτα και Θήτα, οι οποίες αξιοποιήθηκαν για την εξαγωγή χαρακτηριστικών στο παρόν σύστημα.

Η δραστηριότητα στη ζώνη Βήτα υποδηλώνει συνήθως ότι ο εγκέφαλος διατηρεί ενεργά την εστίαση της προσοχής του σε ένα εξωτερικό ερέθισμα. Επιπλέον, ακόμη και εντός της συγκεκριμένης ζώνης, οι χαμηλότερες συχνότητες συνδέονται με διαφορετικές νοητικές καταστάσεις συγκριτικά με τις υψηλότερες. Τα κύματα στο χαμηλό και μεσαίο εύρος της ζώνης Βήτα φαίνεται να σχετίζονται με την αυξημένη συγκέντρωση. Αντιθέτως, τα κύματα στο υψηλότερο εύρος εμφανίζονται κυρίως σε καταστάσεις έντασης, στρες και άγχους. Οι κύριες πηγές προέλευσης αυτών των κυμάτων είναι ο μετωπιαίος και ο βρεγματικός λοβός, καθώς και οι κινητικές περιοχές του φλοιού.

Η δραστηριότητα στη ζώνη Θήτα είναι εντονότερη κυρίως κατά τη διάρκεια της χαλάρωσης και στα αρχικά στάδια του ύπνου. Παράλληλα, ωστόσο, φαίνεται να

διαδραματίζει σημαντικό ρόλο και σε ανώτερες γνωστικές διεργασίες του εγκεφάλου, όπως η μάθηση. Απαντάται έντονα στην περιοχή του ιπποκάμπου και θεωρείται θεμελιώδης για την εδραίωση της μνήμης και τη νευροπλαστικότητα (Engel & Fries, 2010) (Bear et al., 2020) (Klimesch, 1999) .



Εικόνα 1.2. Αναπαράσταση συχνοτήτων εγκεφαλικών κυμάτων Αναδημοσίευση από Brain waves, από ScienceDirect, Αναδημοσίευση από Introduction to EEG- and speech-based emotion recognition, από P. A. Abhang, B. W. Gawali και S. C. Mehrotra, 2016, Academic Press.

1.3.2 Συστήματα EEG

Το κύριο μέρος αυτής της εργασίας αφορά ένος συτηματος EEG. Η καταγραφή και η ανάλυση της εγκεφαλικής δραστηριότητας αποτελούν έναν από τους πιο κρίσιμους τομείς της βιοϊατρικής μηχανικής και των νευροεπιστημών. Το EEG είναι η ιατρική και ερευνητική διάταξη που χρησιμοποιείται για τη μη επεμβατική μέτρηση και καταγραφή των ηλεκτρικών δυναμικών που παράγονται από τον ανθρώπινο εγκέφαλο.

Προκειμένου το παραγόμενο ηλεκτρικό πεδίο να είναι αρκετά ισχυρό ώστε να ανιχνευθεί μη επεμβατικά, απαιτείται η χωρική και χρονική άθροιση (spatial and temporal summation) της ηλεκτρικής δραστηριότητας. Συγκεκριμένα, εκατοντάδες χιλιάδες νευρώνες πρέπει να εκπυρσοκροτήσουν ταυτόχρονα (σύγχρονα). Το άθροισμα των διπόλων τους δημιουργεί ένα μακροσκοπικό ηλεκτρικό πεδίο, το οποίο διαδίδεται μέσω του φαινομένου της αγωγιμότητας όγκου (volume conduction) διαπερνώντας τους εγκεφαλικούς ιστούς, το εγκεφαλονωτιαίο υγρό, τις μήνιγγες, το κρανίο και το δέρμα, μέχρι να φτάσει στους αισθητήρες καταγραφής (Niedermeyer & da Silva, 2005).

Λόγω της αντίστασης που προβάλλουν τα παραπάνω βιολογικά στρώματα (κυρίως το κρανίο), τα εγκεφαλικά σήματα υφίστανται σημαντική εξασθένηση. Φτάνοντας στην επιφάνεια του τριχωτού της κεφαλής, το πλάτος τους χαρακτηρίζεται ως εξαιρετικά μικρό, συνήθως από 10 έως 100 μικροβόλτ (μV). Η χαμηλή αυτή τάση καθιστά τα βιοσήματα ευάλωτα σε ηλεκτρομαγνητικές παρεμβολές και σε ψευδενδείξεις (artifacts), όπως η μυϊκή δραστηριότητα (ΗΜΓ). Για την αντιμετώπιση αυτών των προκλήσεων, το hardware του ΗΕΓ περιλαμβάνει ενισχυτές οργανολογίας υψηλού λόγου απόρριψης κοινού σήματος (CMRR) και αναλογικά φίλτρα. Στη συνέχεια, το σήμα οδηγείται σε έναν Μετατροπέα Αναλογικού σε Ψηφιακό (ADC) υψηλής ανάλυσης για να ψηφιοποιηθεί (Bronzino, 2000).

1.3.3 Φιλτράρισμα

Μετά την ψηφιοποίησή του, τον κεντρικό ρόλο αναλαμβάνει η Ψηφιακή Επεξεργασία Σήματος (Digital Signal Processing - DSP) και ειδικότερα η Θεωρία Φίλτρων. Η εφαρμογή ψηφιακών φίλτρων αποτελεί το κρισιμότερο στάδιο για την απομόνωση της εγκεφαλικής δραστηριότητας από τον περιβαλλοντικό και φυσιολογικό θόρυβο. Αρχικά, εφαρμόζονται υψιπερατά φίλτρα (High-pass filters), τα οποία επιτρέπουν τη διέλευση συχνοτήτων πάνω από μια συγκεκριμένη συχνότητα αποκοπής (συνήθως 0.1 - 0.5 Hz). Σκοπός τους είναι η εξάλειψη της συνεχούς συνιστώσας (DC offset) και των παρασιτικών χαμηλών συχνοτήτων που προκαλούνται από μικροκινήσεις των ηλεκτροδίων ή τον ιδρώτα (baseline drift). Στον αντίποδα, τα βαθυπερατά φίλτρα (Low-pass filters) αποκόπτουν τις υψηλές συχνότητες (συνήθως άνω των 50 ή 100 Hz), καταστέλλοντας δραστικά τον θερμικό θόρυβο των κυκλωμάτων και τις παρεμβολές από τη γειτονική μυϊκή δραστηριότητα (ΗΜΓ).

Θεμελιώδους σημασίας σε κάθε διάταξη ΗΕΓ είναι το φίλτρο εγκοπής ή αποκοπής ζώνης (Notch filter). Πρόκειται για ένα εξειδικευμένο φίλτρο με εξαιρετικά στενή ζώνη αποκοπής, το οποίο εφαρμόζεται αποκλειστικά για την αφαίρεση της ηλεκτρομαγνητικής παρεμβολής του δικτύου ηλεκτροδότησης (Power Line Interference), το οποίο στην Ευρώπη εντοπίζεται στα 50 Hz, αφήνοντας ανέπαφο το υπόλοιπο φάσμα (Sanei & Chambers, 2013).

Ακόμη στην συγκεκριμένη εργασία χρησιμοποιήθηκαν κάποιες πιο εξειδικευμένες τεχνικές:

- Φίλτρα Άπειρης Κρουστικής Απόκρισης, IIR
- Φιλτράρισμα μηδενικής φάσης filtfit
- Κυλιόμενης μνήμης (Rolling buffer)

Τα φίλτρα άπειρης κρουστικής απόκρισης (IIR) είναι μια κατηγορία φίλτρων που χρησιμοποιούν ανατροφοδότηση. Για να εξάγουν τα αποτελέσματά τους χρησιμοποιούν τις προηγούμενες τιμές εξόδου και εισόδου τους. Το πλεονέκτημά τους είναι ότι μπορούν να κόψουν απότομα τα ανεπιθύμητα σήματα με μικρής τάξης φίλτρα. Αυτός είναι ο λόγος που επιλέχθηκαν καθώς αυτό τα κάνει υπολογιστικά οικονομικά και γρήγορα, καθιστώντας τα ιδανικά για συστήματα με λίγους πόρους και πραγματικού χρόνου σαν αυτό της εργασίας. Το μειονέκτημά τους είναι ότι στην έξοδό τους δεν υπάρχει γραμμικότητα φάσης που μπορεί σε ένα EEG να προκαλέσει πρόβλημα στη χρονική ακρίβεια (Proakis & Manolakis, 2006)(Widmann et al., 2015).

Στην τεχνική filtfilt εφαρμόζεται το IIR φίλτρο κανονικά (προς τα εμπρός) και στη συνέχεια ανάποδα (προς τα πίσω). Αυτή η διπλή διεργασία αλληλοαναιρεί την καθυστέρηση φάσης που προκαλείται μετά το πρώτο πέρασμα. Με αυτό τον τρόπο το σήμα που εξάγεται δεν φέρει καμία χρονική αλλοίωση. Αυτή όμως η τεχνική καθώς είναι μη-αιτιατή (θέλει και μελλοντικά δεδομένα) δεν μπορεί να χρησιμοποιηθεί σε εφαρμογές πραγματικού χρόνου (Widmann et al., 2015).

Όταν το φιλτράρισμα των συχνοτήτων πρέπει να γίνει σε πραγματικό χρόνο η τεχνική rolling buffer είναι κατάλληλη. Σε αυτήν την τεχνική υπάρχει μια προσωρινή μνήμη μέσα στο RAM σταθερού μεγέθους. Η λογική του συστήματος είναι FIFO (first in first out). Κάθε κύκλο τα νέα σήματα μπαίνουν και τα πιο παλιά μέσα στην μνήμη φεύγουν (Oppenheim & Schafer, 2010).

1.3.4 Ζώνες ισχύος

Ακόλουθως, θα παρουσιαστούν τα χαρακτηριστικά που εξήχθησαν στην παρούσα εργασία, δηλαδή η ισχύς των ζωνών Βήτα και Θήτα.

Αρχικά, το σήμα πρέπει να μεταβεί στο πεδίο των συχνοτήτων μέσω του μετασχηματισμού Fourier (Fourier Transform). Η φασματική ισχύς υπολογίζεται ως το τετράγωνο του πλάτους των κυμάτων και μετράται σε τετραγωνικά μικροβόλτ (μV^2). Έτσι, η ισχύς σε κάθε ζώνη (π.χ. 13-30 Hz) αντιπροσωπεύει την ενέργεια που εκπέμπει ο εγκέφαλος στο εκάστοτε εύρος συχνοτήτων.

Δεδομένου ότι το συστημα EEG δεν καταγράφει το σήμα μεμονωμένων νευρώνων, αλλά τη συνολική δραστηριότητα μεγάλων νευρωνικών πληθυσμών στον φλοιό, τα σήματα αυτά υποδεικνύουν τον νευρωνικό συγχρονισμό (neural synchrony) ως αποτέλεσμα μιας συγκεκριμένης νοητικής κατάστασης και δεν είναι τυχαία (Schomer & Lopes da Silva, 2017) (Cohen, 2014) (Buzsáki, 2006)

Έχοντας εξεταστεί η γενική εικόνα των δράσεων των ζωνών βήτα και θήτα και τις ζώνες ισχύος, ας παρουσιαστούν τα συμπεράσματα που έχουν εξαχθεί από τις καταγραφές των ζωνών ισχύος αυτών.

Ενώ η αύξηση της ισχύος της ζώνης θήτα στις μετωπιαίες περιοχές υποδηλώνει γνωστική δοκιμή και κόπωση, όταν αυτή η αύξηση γίνεται σε όλο τον φλοιό υποδηλώνει αυξομείωση της εγρήγορσης και της νοητικής κόπωσης.

Από την άλλη, η ισχύς στη ζώνη βήτα, όπως και με τη συσχέτιση των κυμάτων, υποδεικνύει την συνέχεια μια διαδικασία. Επίσης, είναι ενδιαφέρον το φαινόμενο λίγο πριν την εκούσια κίνηση (στον φλοιό). Λίγο πριν γίνει η κίνηση, η ισχύς πέφτει απότομα (αποσυγχρονισμός) και στη συνέχεια ξανά ανεβαίνει απότομα **(**Engel & Fries, 2010).

Ο λόγος μεταξύ τους ισχύος κυμάτων θήτα προς βήτα (Theta/Beta Ratio - TBR) στη σύγχρονη νευροεπιστήμη χρησιμοποιείται για την εύρεση της ικανότητας εστίασης. Αν ο λόγος είναι ισορροπημένος και χαμηλός αυτό υποδεικνύει εστίαση σε εκτελεστικό έργο ενώ η παθολογική αύξηση αυτού του λόγου υποδεικνύει εύρημα σε διαταραχές της προσοχής όπως η ΔΕΠΥ (Arns et al., 2013).

1.4 ΔΟΜΗ ΤΗΣ ΕΡΓΑΣΙΑΣ

Η παρούσα διπλωματική εργασία οργανώνεται σε πέντε κεφάλαια, τα οποία παρουσιάζουν με λογική ακολουθία το θεωρητικό υπόβαθρο, τη μεθοδολογία, την υλοποίηση, τα αποτελέσματα και τα τελικά συμπεράσματα της έρευνας.

- **Στο Κεφάλαιο 1** παρουσιάζεται η εισαγωγή της εργασίας, το υπόβαθρο και η επιλογή του θέματος, ο σκοπός και οι στόχοι της μελέτης, καθώς και οι βασικές θεωρητικές έννοιες που σχετίζονται με την υλοποίηση του προτεινόμενου συστήματος.
- **Στο Κεφάλαιο 2** παρουσιάζονται τα υλικά και το λογισμικό που χρησιμοποιήθηκε στην εργασία. Γίνονται αναφορές σε έρευνες που εξέταζαν την ποιότητα του EEG και παρουσιάστηκε η δομική λειτουργία του προγράμματος inspire.
- **Στο Κεφάλαιο 3** περιγράφεται αναλυτικά η μεθοδολογία που ακολουθήθηκε για την ανάπτυξη της πειραματικής διάταξης. Παρουσιάζονται τα υλικά και ο εξοπλισμός που χρησιμοποιήθηκαν, η επαναδημιουργία του συστήματος Inspire και οι απαραίτητες προσαρμογές του, καθώς και η συνολική διαδικασία υλοποίησης. Επίσης, αναλύεται η αρχιτεκτονική του συστήματος, η δομή λειτουργίας του προγράμματος καταγραφής, τα βοηθητικά προγράμματα και το κύκλωμα LED εξόδου. Τέλος, γίνεται ανάλυση ορισμένων προβλημάτων που αντιμετωπίστηκαν και παρουσιάζονται οι σχετικοί κίνδυνοι και περιορισμοί.
- **Στο Κεφάλαιο 4** παρατίθενται τα αποτελέσματα της πειραματικής διαδικασίας. Παρουσιάζονται και αξιολογούνται οι επιδόσεις του συστήματος, όπως η απεικόνιση των χαρακτηριστικών σε πραγματικό χρόνο, τα μηνύματα στο παράθυρο εντολών (command window), η επίδραση των φίλτρων και οι καταγραφές μέσω των απεικονιστικών προγραμμάτων που υλοποιήθηκαν.
- Τέλος, **στο Κεφάλαιο 5** αναπτύσσεται η συζήτηση των αποτελεσμάτων και διατυπώνονται τα συμπεράσματα της εργασίας. Παρουσιάζονται οι περιορισμοί της προτεινόμενης προσέγγισης, αξιολογείται ο βαθμός επίτευξης των στόχων και προτείνονται πιθανές μελλοντικές επεκτάσεις και βελτιώσεις του συστήματος.

2. ΥΛΙΚΑ

Στο παρόν υποκεφάλαιο θα παρουσιαστούν τα υλικά και ο εξοπλισμός που χρησιμοποιήθηκαν για την υλοποίηση του συστήματος προσομοίωσης. Τα υλικά που χρησιμοποιήθηκαν επιλέχθηκαν με βάση τη διαθεσιμότητα τους από εξοπλισμό που ήταν ηδη διαθέσιμος και από το εργαστήριο του κυρίου Ασβεστά, τη συμβατότητα μεταξύ τους, καθώς και τις απαιτήσεις της πειραματικής διαδικασίας. Ιδιαίτερη έμφαση δόθηκε στη χρήση εξαρτημάτων χαμηλού κόστους και εύκολης ενσωμάτωσης, ώστε το σύστημα να είναι απλό και πρακτικά υλοποιήσιμο. Στον Πίνακα 3.1 παρουσιάζονται συνοπτικά τα βασικά στοιχεία του εξοπλισμού, ενώ στη συνέχεια δίνονται επιπλέον πληροφορίες για κρίσιμα υποσυστήματα.

***Πίνακας 2.1**: Εξαρτήματα πειραματικής διαδικασίας*

Α/Α	ΥΛΙΚΟ	ΠΕΡΙΓΡΑΦΗ
1	Συστημα eeg(electroencephalogram)	OpenBCI "Mark IV" Cap
1.1	Πλακέτα Cyton	Παρέχει τη δυνατότητα καταγραφής 8 καναλιών
1.2	Πλακέτα Daisy	Η πλακέτα Daisy συνδέεται με τη «μητρική» πλακέτα Cyton, λειτουργώντας ως επέκταση που προσθέτει άλλα 8 κανάλια καταγραφής, φτάνοντας τη δυνατότητα παροχής 16 καναλιών από τη συνδυαστική χρήση των 2 πλακετών
1.3	Κράνος EEG	Ο σκελετός της κάσκας είναι εξ' ολοκλήρου κατασκευασμένος απο 3D εκτυπωτή, σε αυτόν προσαρμόζονται οι αισθητήρες, τα καλώδια και οι πλακέτες
1.4	Αισθητήρες & Καλώδια	Ηλεκτρόδια Καταγραφής OpenBCI
1.5	USB Dongle	Το usb dongle υλοποιεί τη σύνδεση μεταξύ της πλακέτας Cyton με υπολογιστική μονάδα
2	Υπολογιστής	Συμβατικό λάπτοπ
3	Μπαταρίες	4 Μπαταρίες ΑΑ (οι οποίες αντικαταστάθηκαν 2 φορές κατά τη διαδικασία) για το EEG.
4	Κάρτα microSD	Κάρτα μνήμης που περιέχει λογισμικό και αρχεία για την εκκίνηση και έλεγχο του Raspberry
5	Raspberry Pi	Raspberry Pi 4 model B
6	Τροφοδοτικό για Raspberry Pi	Τύπου micro usb
7	Breadboard	Βάση για την δημιουργία των κυκλωμάτων
8	Αντίσταση	120ΚΩ και ανοχή 5%
9	USB to ethernet converter	Για τη διασύνδεση του φορητού υπολογιστή με το Raspberry Pi
10	Ethernet cable	Για την διασύνδεση του USB to ethernet converter με το raspberry Pi

11	Jumper καλώδια	2 Θηλυκό σε αρσενικό καλώδια
12	SD adapter for microSD	Μέσα περνάτε η microSD τα στοιχεία από τον υπολογιστή μας.
13	LED (Light Emitting Diode)	1 Δίοδος εκπομπής φωτός που εκπέμπει οπτική ακτινοβολία στο κόκκινο
14	Πολύμετρο	Φορητό ηλεκτρονικό όργανο μέτρησης

Τα βασικά εξαρτήματα της διαδικασίας είναι τα εξής:

2.1 Raspberry Pi 4 model b

Η υλοποίηση της παρούσας διπλωματικής εργασίας στηρίχθηκε στη σύνθεση τεχνολογιών από τους τομείς των ενσωματωμένων συστημάτων (embedded systems), του ηλεκτρονικού σχεδιασμού, της παραγωγής και επεξεργασίας σημάτων, καθώς και του προγραμματισμού αυτοματισμών. Για την πλήρη κατανόηση της λειτουργίας της προτεινόμενης πειραματικής διάταξης κρίνεται σκόπιμη η παράθεση των βασικών θεωρητικών αρχών που τη διέπουν.

Ως ενσωματωμένα συστήματα ορίζονται οι υπολογιστικές μονάδες που έχουν σχεδιαστεί αποκλειστικά για την εκτέλεση στοχευμένων λειτουργιών ελέγχου και αυτοματισμού, ικανοποιώντας συχνά αυστηρούς χρονικούς περιορισμούς (λειτουργία σε πραγματικό χρόνο). Διαφέροντας από τους παραδοσιακούς υπολογιστές γενικής χρήσης, συνδυάζουν την επαρκή υπολογιστική ισχύ με τη χαμηλή ενεργειακή κατανάλωση, τις συμπαγείς διαστάσεις και την άμεση ικανότητα διασύνδεσης με περιφερειακούς αισθητήρες και ενεργοποιητές. Το Raspberry Pi που χρησιμοποιήθηκε, βέβαια, μπορεί να λειτουργήσει και ως υπολογιστής, αλλά μελλοντικά μπορεί να αντικατασταθεί με κάποιο άλλο σύστημα, εξειδικευμένο και σχεδιασμένο για τη διαδικασία.

Στην καρδιά της προτεινόμενης διάταξης επιλέχθηκε να ενσωματωθεί το Raspberry Pi 4 Model B**,** ένας ισχυρός υπολογιστής μονής πλακέτας (Single Board Computer – SBC). Σε σχέση με προγενέστερα μοντέλα, το Raspberry Pi 4 προσφέρει σημαντικά αναβαθμισμένες επιδόσεις, καθώς ενσωματώνει τετραπύρηνο επεξεργαστή Broadcom BCM2711 (Quad-core Cortex-A72 64-bit) χρονισμένο στο 1.5GHz. Η αρχιτεκτονική του, σε συνδυασμό με την ταχύτερη μνήμη RAM (LPDDR4), του επιτρέπει να ανταπεξέρχεται με ευκολία σε απαιτητικές διεργασίες, όπως η ταχύτατη επεξεργασία σημάτων. Το σύστημα εκτελεί λειτουργικό σύστημα βασισμένο στο Linux (π.χ. Raspberry Pi OS) και παρέχει εγγενή υποστήριξη σε σύγχρονες γλώσσες προγραμματισμού όπως η Python. Η επιλογή του συγκεκριμένου μοντέλου βασίστηκε στην εξαιρετική σχέση κόστους-απόδοσης, την υψηλή υπολογιστική του ευελιξία και την αξιοπιστία του σε βιομηχανικές και ερευνητικές εφαρμογές.

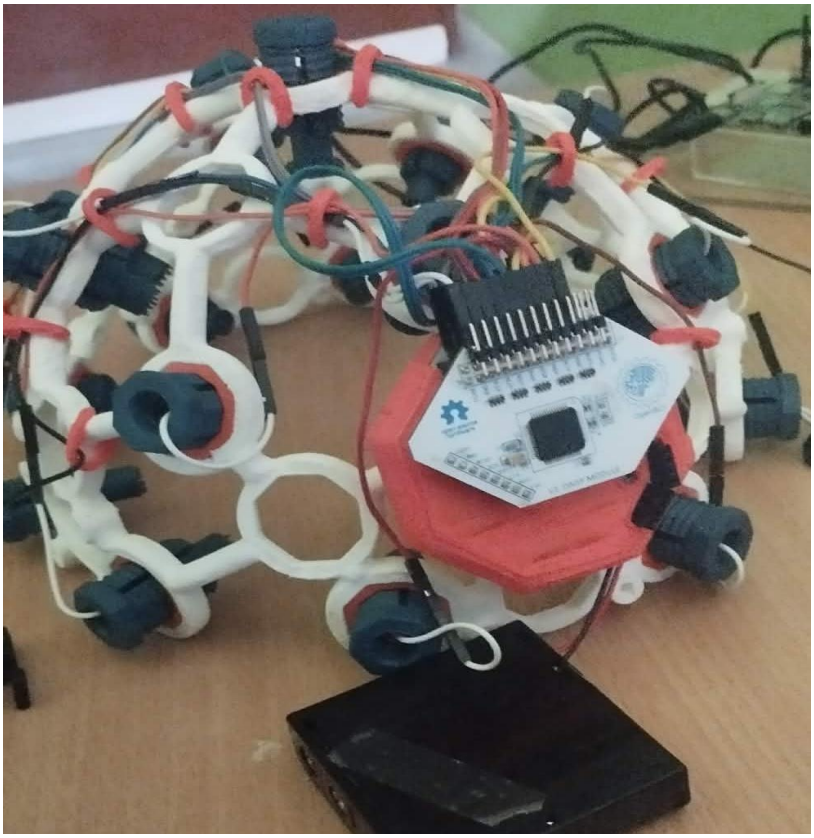
Καθοριστικό ρόλο στη διασύνδεση του συστήματος παίζει η κεφαλή 40 ακροδεκτών γενικής χρήσης (GPIO - General Purpose Input/Output) που διαθέτει η πλακέτα του Raspberry Pi 4. Αυτοί οι ακροδέκτες προγραμματίζονται ψηφιακά ανάλογα με τις ανάγκες της εφαρμογής. Όταν ορίζονται ως είσοδοι, είναι ικανοί να ανιχνεύουν λογικές στάθμες, διαχωρίζοντας το λογικό χαμηλό (0V) από το λογικό υψηλό (3.3V). Αντίστοιχα, όταν λειτουργούν ως έξοδοι, παρέχουν αυτές τις τάσεις για τη διέγερση εξωτερικών κυκλωμάτων. Είναι κρίσιμο να σημειωθεί ότι η λογική τάση λειτουργίας των GPIO είναι αυστηρά στα 3.3V, συνεπώς απαιτείται προσεκτικός σχεδιασμός

συμβατότητας (level shifting) όπου χρειάζεται, για την αποφυγή ανεπανόρθωτων βλαβών από υπέρταση. Στα πλαίσια του πειράματος, οι ακροδέκτες GPIO αξιοποιήθηκαν διττά: αφενός ως είσοδοι για τη σύλληψη και ανίχνευση τετραγωνικών παλμών, και αφετέρου ως έξοδοι για τη χορήγηση σημάτων ελέγχου στο κύκλωμα οδήγησης του χειριστηρίου.

Για την ευκολότερη ανάπτυξη και διαχείριση της διάταξης, εφαρμόστηκαν τεχνικές απομακρυσμένου ελέγχου (headless setup). Η απομακρυσμένη διαχείριση επιτρέπει τον πλήρη έλεγχο του Raspberry Pi 4 από έναν κεντρικό υπολογιστή, εξαλείφοντας την ανάγκη για περιφερειακά (οθόνη, ποντίκι, πληκτρολόγιο) πάνω στην ίδια τη συσκευή. Για την ασφαλή επικοινωνία σε επίπεδο γραμμής εντολών μέσω δικτύου χρησιμοποιήθηκε το πρωτόκολλο SSH (Secure Shell). Παράλληλα, για την πρόσβαση στο γραφικό περιβάλλον (GUI) του λειτουργικού συστήματος αξιοποιήθηκε η τεχνολογία VNC (Virtual Network Computing). Οι τεχνολογίες αυτές διασφάλισαν την απρόσκοπτη συγγραφή, δοκιμή και συνεχή παρακολούθηση της εκτέλεσης του κώδικα απευθείας στον χώρο του πειράματος.

2.2 Το Σύστημα EEG Ultracortex Mark IV

Το EEG σύστημα Ultracortex Mark IV είναι μια ανοιχτού κώδικα (open-source), 3D-εκτυπώσιμη κάσκα του συτηματος EEG που σχεδιάστηκε από την εταιρεία OpenBCI. Είναι σχεδιασμένος να παίρνει καταγραφές ερευνητικού επιπέδου και αποτελεί ένα από τα πιο διαδεδομένα εργαλεία στην έρευνα BCI, καθώς συνδυάζει χαμηλό κόστος, ευελιξία και αξιόπιστη λήψη σήματος. Έχει 32 θέσεις όπου μπορούν να τοποθετηθούν τα ηλεκτρόδια και ακολουθεί το διεθνές σύστημα 10-20 για την τοποθέτηση των ηλεκτροδίων. Ο σκελετός είναι εξ ολοκλήρου 3D εκτυπωμένος, επιτρέποντας διαφορετικά μεγέθη ανάλογα με τον χρήστη. Οι πολλές δυνατές θέσεις επιτρέπουν την κατάλληλη διαμόρφωση.



Εικόνα 2.1*: Φωτογραφίες του EEG που χρησιμοποιήθηκε (back view)*



Εικόνα 2.2*: Φωτογραφίες του EEG που χρησιμοποιήθηκε (top view).*

Χρησιμοποιώντας 16 ξηρά αγκαθωτά ηλεκτρόδια για την ελαχιστοποίηση του θορύβου χαμηλών συχνοτήτων, δεν είναι απαραίτητη η χρήση τζελ, εξοικονομώντας

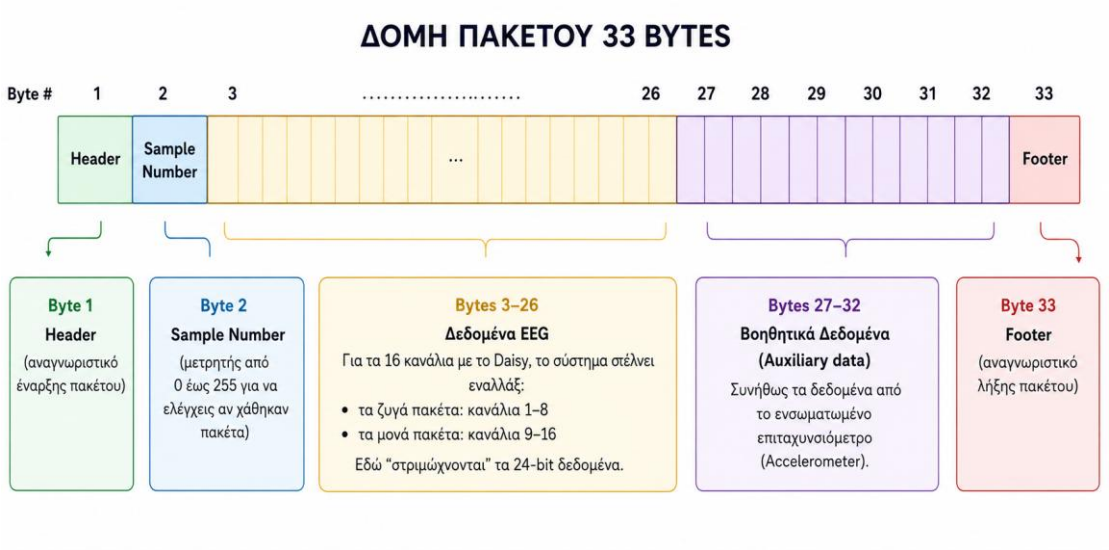
χρόνο. Ενώ, χρησιμοποιούνται 2 κλιπ αυτιών, όπου το ένα λειτουργεί ως γείωση (BIAS) και το άλλο ως σημείο αναφοράς (SRB).

Χρησιμοποιήθηκε η πλακέτα Cyton, η οποία διαθέτει 8 κανάλια, ως μητρική, σε συνδυασμό με την πλακέτα Daisy που κουμπώνει από πάνω, αυξάνοντας τα κανάλια του συστήματος σε 16. Όταν όμως το σύστημα χρησιμοποιεί 16 αντί για 8 κανάλια, η δειγματοληψία του πέφτει στο μισό, δηλαδή στα 125 Hz από τα 250 Hz. Αυτός είναι ένας τεχνικός συμβιβασμός, ο οποίος πρέπει να γίνει λόγω του εύρους ζώνης του ασύρματου πρωτοκόλλου. Το USB dongle πρέπει να μεταδώσει διπλάσιο όγκο δεδομένων στο ίδιο χρονικό παράθυρο, οπότε το υλικολογισμικό (firmware) ρίχνει τον ρυθμό δειγματοληψίας στο μισό για να αποφευχθεί η απώλεια πακέτων (packet loss).

Ο ρυθμός δειγματοληψίας των 125 Hz είναι σχετικά μικρός σε σχέση με αυτόν των κλινικών συστημάτων, τα οποία συνήθως δουλεύουν στα 500, 1000 ή και παραπάνω Hz. Σύμφωνα με το θεώρημα Nyquist, η μέγιστη συχνότητα που μπορεί να αναγνωριστεί θα είναι τα 62,5 Hz. Έτσι, κόβεται ένα μέρος της ζώνης γ (gamma), αλλά αυτό δεν αποτελεί μεγάλο πρόβλημα, καθώς σπάνια χρησιμοποιείται σε τέτοια συστήματα, επειδή εισάγει πολύ θόρυβο. Από την άλλη μεριά, αυτό δρα ως πλεονέκτημα για το Raspberry Pi, μειώνοντας στο μισό τον όγκο δεδομένων ανά δευτερόλεπτο. Αυτό μειώνει δραστικά τον υπολογιστικό φόρτο (CPU load), την απαιτούμενη μνήμη RAM και τον χρόνο εκτέλεσης των αλγορίθμων (π.χ. υπολογισμός FFT, εξαγωγή χαρακτηριστικών, μηχανική μάθηση).

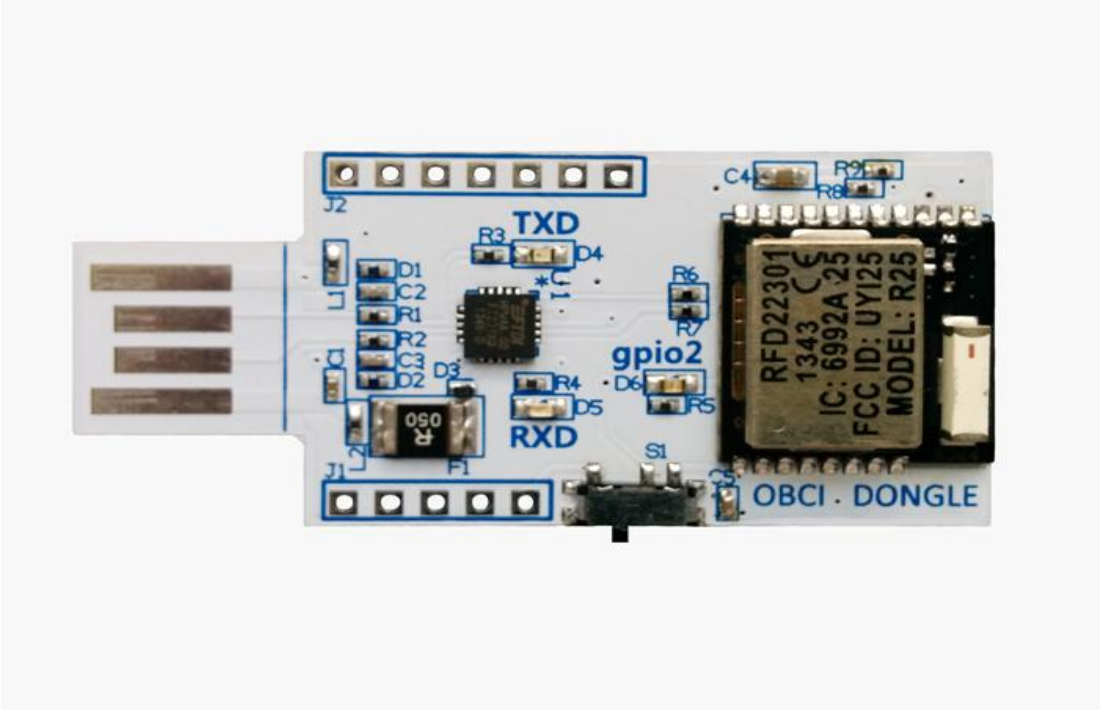
Διαθέτει ανάλυση 24-bit, έχοντας πολύ καλή πραγματική Διακριτική Ικανότητα (Scale Factor / LSB): περίπου 0.022 μV και Λόγο Απόρριψης Κοινού Σήματος (CMRR) -110 dB. Παρόλα αυτά τα πολύ καλά χαρακτηριστικά, λόγω των ξηρών ηλεκτροδίων το σήμα υποβαθμίζεται σε ποιότητα εξαιτίας των αντιστάσεων, σε σχέση με τα ιατρικά που χρησιμοποιούν τζελ. Το τελικό σήμα που τροφοδοτείται στο Raspberry Pi θα περιέχει περισσότερα παράσιτα (artifacts) και περιβαλλοντικό θόρυβο από αυτό των ιατρικών συστημάτων EEG.

Η πλακέτα Cyton (πομπός) και το USB Dongle (δέκτης) είναι εξοπλισμένα το καθένα με έναν μικροελεγκτή RFduino (RFD22301). Χρησιμοποιούν ένα custom firmware βασισμένο στο πρωτόκολλο Gazell. Το πρωτόκολλο Gazell λειτουργεί στην ίδια συχνότητα με το Bluetooth, αλλά είναι ένα ιδιόκτητο (proprietary) πρωτόκολλο σύνδεσης "Device-to-Host" (συσκευή προς κεντρικό υπολογιστή). Ο λόγος που επιλέχθηκε είναι ότι έχει κάποια πλεονεκτήματα έναντι του συμβατικού Bluetooth, καθώς υπάρχει μικρότερη καθυστέρηση, αξιοπιστία ροής δεδομένων και Εικονική Σειριακή Θύρα (Virtual COM Port). Κάθε πακέτο που λαμβάνει το dongle απο την πλακετας cyton έχει σταθερό μέγεθος 33 bytes και έχει τη δομή που φαίνεται στον πίνακα.



Εικόνα 2.3*:Δομή πακέτου πληροφοριών του EEG.*

Το USB Dongle έχει πάνω του και ένα τσιπ FTDI (FT231X), που ο ρόλος του είναι να παίρνει τα ασύρματα δεδομένα από το RFduino και να τα μετατρέπει σε USB δεδομένα που καταλαβαίνει το Raspberry Pi. Το σήμα που μεταφέρεται δεν επεξεργάζεται με κανέναν τρόπο, αποτελεί απλά έναν μεταφορέα δεδομένων. Όταν συνδέεις το Dongle στη USB θύρα του Raspberry Pi, το λειτουργικό σύστημα (π.χ. Raspberry Pi OS / Linux) δεν το αναγνωρίζει ως συσκευή Bluetooth, αλλά ως Εικονική Σειριακή Θύρα (Virtual COM Port-VCP). Αυτο είναι δυνατόν να συμβεί λόγο του ότι το τσιπ **FTDI** (FT231XQ) αποτελεί ολοκληρωμένο κύκλωμα. Το USB Dongle επικοινωνεί αυστηρά με 115200 bits ανά δευτερόλεπτο ανεξάρτητα αν χρησιμοποιεί 8 ή 16 κανάλια το συστημα EEG.



Εικόνα 2.4*: USB Dongle της Open BCI*



***Εικόνα 2.5**: USB Dongle της Open BCI πάνω στο Raspberry Pi εν λειτουργία*

2.2.1 Αξιολόγηση Αξιοπιστίας του συστήματος της OpenBCI

Καθώς το σύστημα OpenBCI στοχεύει στην επέκταση της δημοφιλίας των συστημάτων EEG, παρέχοντας ένα φθηνό και εύκολο στη χρήση σύστημα με ανοιχτό κώδικα (open source), έχουν γίνει πολλές έρευνες αξιολογώντας και συγκρίνοντας τη λειτουργία του και την ποιότητα των καταγραφών του.

Γενικά, το σύστημα αποδείχθηκε ευέλικτο, κλιμακώσιμο και αποτελεί μια ολοκληρωμένη λύση, κατάλληλη για νευροφυσιολογικά πειράματα δεδομένων EEG. Διαθέτει ρυθμό δειγματοληψίας με αποδεκτά επίπεδα διακύμανσης, ακόμη και σε ασύρματη υλοποίηση.

Το σύστημα εγγυάται ροή δεδομένων σε πραγματικό χρόνο, εντός των απαιτούμενων ορίων καθυστέρησης και διακύμανσης (jitter) για εφαρμογές BCI κλειστού βρόχου (closed-loop).

Η χρονική ακρίβεια και η διακύμανση (jitter) του συστήματος OpenBCI εξαρτώνται σημαντικά από τη διάταξη υλικού και λογισμικού που χρησιμοποιείται. Όταν το σύστημα χρησιμοποιείται με το προεπιλεγμένο πρωτόκολλο Bluetooth (USB dongle) σε συχνότητες δειγματοληψίας 125-250 Hz, η εφαρμογή ειδικών αλγορίθμων διόρθωσης (όπως το 'chunk dejitter') μπορεί να μειώσει το jitter σε εξαιρετικά χαμηλά επίπεδα, μεταξύ 1.69 ms και 3.19 ms, με την καθυστέρηση να κυμαίνεται περίπου στα 527-530 ms. Αλλά, χρησιμοποιώντας τη ρύθμιση “FTDI buffer fix”, η καθυστέρηση μπορεί να μειωθεί στα 18.12 - 21.92 ms.

Μέσω της μεθόδου "sample index", το σύστημα μπορεί να εντοπίσει εύκολα και αποτελεσματικά σημεία όπου έχει αποτύχει η μετάδοση δεδομένων. Περαιτέρω, για την ποιότητα σήματος που προσφέρει, έχει αποδειχθεί σημαντική η κατάλληλη αντίσταση μεταξύ ηλεκτροδίου και δέρματος. Ώστε να αντιμετωπιστεί αυτό και να

ελεγχθεί η ποιότητα της επαφής μεταξύ δέρματος και ηλεκτροδίων, το σύστημα χρησιμοποιεί μια λειτουργία του ADS1299 η οποία διοχετεύει ένα πολύ μικρό ρεύμα 6 nA στα 31.2 Hz, υπολογίζοντας στη συνέχεια την αντίσταση μέσω του νόμου του Ohm.

Παράλληλα, το περιβάλλον του πλαισίου παρέχει ένα πλήρες API (Application Programming Interface), αυτόματες ρυθμίσεις στο παρασκήνιο, καθώς και "widgets" για την παρουσίαση των ερεθισμάτων στον χρήστη. Αυτά τα χαρακτηριστικά το καθιστούν ιδανική πλατφόρμα για την επεξεργασία δεδομένων BCI και τη δημιουργία προσαρμοσμένων λειτουργιών. Το πλαίσιο υποστηρίζει επίσης πολλαπλούς ρυθμούς δειγματοληψίας, διάφορα μεγέθη πακέτων δεδομένων, διαφορετικά πρωτόκολλα επικοινωνίας και ελεύθερη τοποθέτηση ηλεκτροδίων.

Στη σύγκρισή του με το σύστημα υψηλών προδιαγραφών MBrain Train Smarting Mobi, το οποίο κατατάσσεται στον ερευνητικό/ιατρικό εξοπλισμό υψηλών προδιαγραφών (high-end), βρέθηκαν τα εξής: Σε πείραμα ταυτόχρονης καταγραφής στον ίδιο συμμετέχοντα, η συσχέτιση του σήματος μεταξύ γειτονικών ηλεκτροδίων του ίδιου ενισχυτή ήταν 0.89 για το Smarting Mobi και 0.83 για το OpenBCI. Η διάμεση συσχέτιση (median channel correlation) μεταξύ ηλεκτροδίων των δύο διαφορετικών ενισχυτών ήταν πολύ υψηλή, φτάνοντας το r = 0.93, υποδηλώνοντας αντίστοιχη καταγραφή θορύβου (SNR). Επίσης, ο εγγενής θόρυβος (input-referred noise) του OpenBCI υπολογίστηκε περίπου στο ~1 μVpp, λίγο μεγαλύτερος από αυτόν του συστήματος MBrain Train Smarting Mobi.

Να αναφερθεί ότι το εξής αποτέλεσμα προήλθε από καταγραφές γύρω από το αυτί (cEEGrids), Ακουστικά Προκλητά Δυναμικά (P300 ERP), επίδραση Berger (Alpha) και νοητικό φορτίο.

2.3 Λογισμικό που χρησιμοποιήθηκε

Επίσης στην πειραματική διαδικασία χρησιμοποιήθηκε και λογισμικό για ορισμένα στάδια.

Πίνακας 2.2: *Εργαλεία λογισμικού στην πειραματική διαδικασία*

Α/Α	ΛΟΓΙΣΜΙΚΟ	ΠΕΡΙΓΡΑΦΗ
1	Python	
2	Arduino IDE(1.8.19)	Για το λογισμικό του firmware
3	OpenBCI GUI	Προαιρετικά για το έλεγχο του eeg συστήματος
4	Inspire	Πρόγραμμα που αναπτύχθηκε απο την ΕΛ.ΙΔ.Ε.Κ. Χρησιμοποιήθηκε σαν βάση του προγράμματός
5	Raspberry Pi Imager	Για την φόρτωση λογισμικού από τον υπολογιστή στην κάρτα SSD του Raspberry Pi

Θα παρουσιαστεί περιληπτικά η δομή του προγράμματος inspire για την κατανόηση της αλληλεπιδράσεις του λογισμικού που υλοποιήθηκε κατα την εργασία με αυτό. Το πρόγραμμα αυτό χρησιμοποιεί τμηματικό Προγραμματισμό. Το κύριο πρόγραμμα

είναι το inspire_main το οποίο αφού επικαλεστεί από των χρήστη στην λειτουργία του καλεί τα υπόληπτα προγράμματα (τα οποία βρίσκονται στο ίδιο φάκελο).

Το προγραμμα δέχεται πολλές πήγες για εισόδους και εξόδους. Ως είσοδο έχει το σύστημα EEG, ηλεκτρομυογράφημα, δεδομένα κινηματικής ανάλυσης και δεδομένα κίνησης αμαξιδίου. Η έξοδος γίνεται σε συσκευή λειτουργικής ηλεκτρικής διέγερσης ή σε αμαξίδιο. Επίσης, μέσα στον κώδικα παρέχονται και επιλογές για τη χρήση παράλληλης επεξεργασίας.

Στο inspire_main:

1. Φορτώνονται οι απαραίτητες βιβλιοθήκες.
2. Ορίζονται οι μεταβλητές.

```
################################################

# Set session variables
participant = "panagiotis"      # Participant name / identifier
exp_time    = datetime.now() # Save current timestamp for session folder
scale       = True           # Rescale input values or store raw data
guide       = False          # Provide stimulus for movement
task_type   = 2              # 1: walking, 2: wheelchair
process     = True          # Produce real-time output
feedback    = True           # Produce real-time feedback
save_data   = True           # Save session data

################################################
```

Εικόνα 2.6: *Στιγμιότυπο οθόνης του κώδικα python στο inspire_main.*

Αρχικοποιούνται οι παράμετροι (με τη χρήση του inspire_config). Το όνομα της μεταβλητής participant το παίρνει ο φάκελος μέσα στον οποίο αποθηκεύονται όλες οι καταγραφές αυτού του ατόμου. Το task_type αφορά το τι διεργασία θα εκτελέστει. Η διεργασία ορίστηκε στο task type 2, δηλαδή στο αναπηρικό αμαξίδιο.

3. Δημιουργία της διεπαφής με τον χρήστη μέσω του inspire_interface.
4. Ορισμός και έναρξη των διεργασιών με το inspire_threads.
5. Στο τέλος αποθηκεύονται τα δεδομένα της συνεδρίας.
6. Προετοιμασία σύνδεσης και δεδομένων: Το σύστημα συνδέεται με τις εξωτερικές συσκευές και οργανώνει τις μετρήσεις σε μια ενιαία μορφή. Δεδομένα συλλέγονται αποκλειστικά από τους αισθητήρες που το λογισμικό έχει εντοπίσει και με τους οποίους έχει συνδεθεί επιτυχώς.

Στο inspire_threads υλοποιείται η αρχιτεκτονική παραλληλισμού του συστήματος μέσω της δημιουργίας τεσσάρων νημάτων (threads), επιτρέποντας την ταυτόχρονη και ασύγχρονη εκτέλεση διεργασιών στο παρασκήνιο (ως daemon threads). Αυτά τα νήματα καλούνται και εκκινούνται από το inspire_main.

Λειτουργικά, το σύστημα κατανέμεται σε τέσσερα διακριτά νήματα:

Η συνάρτηση monitor_display_task δεσμεύει ένα νήμα για τη διαχείριση συμβάντων (event handling) και την παρακολούθηση της κατάστασης της γραφικής διεπαφής. Καλεί και ελέγχει το inspire_interface.monitor.

Η συνάρτηση record_task ελέγχει τη ροή και την αδιάλειπτη εγγραφή (data logging) της τηλεμετρίας των αισθητήρων. Αντίστοιχα, καλεί και ελέγχει το inspire_datarecord.

Η συνάρτηση feedback_display_task αναλαμβάνει αποκλειστικά την απόδοση (rendering) για την οπτικοποίηση της ανατροφοδότησης. Καλεί και ελέγχει το inspire_interface.feedback_display.

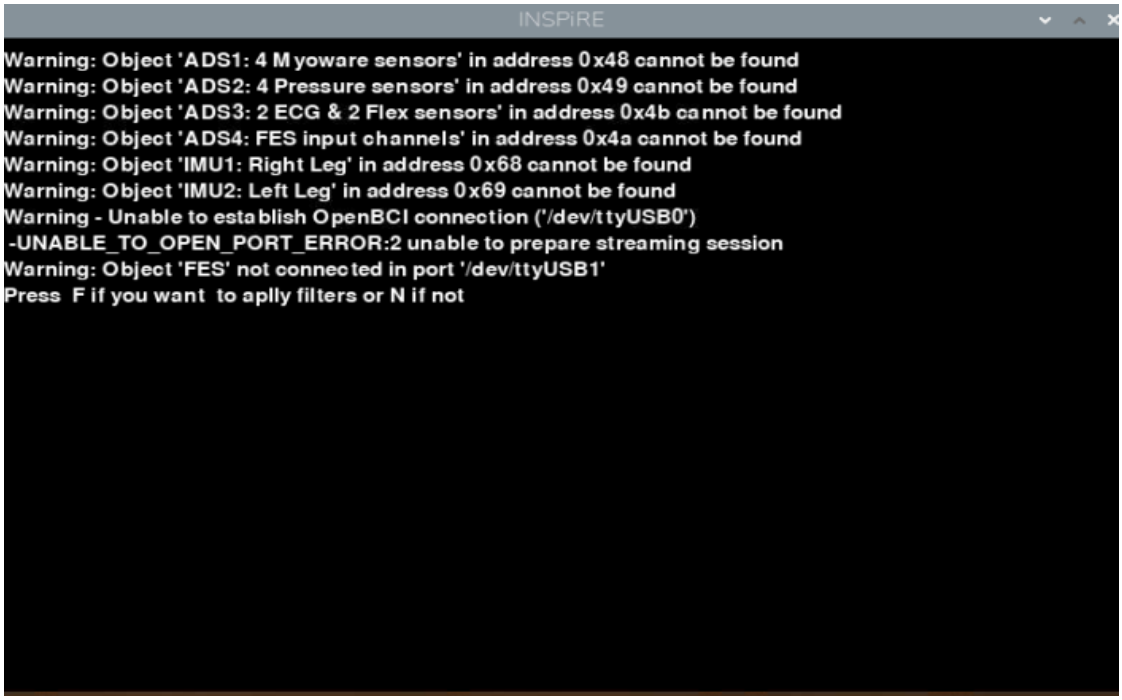
Η συνάρτηση process_task απομονώνει τον υπολογιστικά απαιτητικό φόρτο της ψηφιακής επεξεργασίας σήματος σε πραγματικό χρόνο, καλώντας και ελέγχοντας το inspire_dataprocess.data_process. Αυτός ο αυστηρός διαχωρισμός λειτουργιών (separation of concerns) βελτιστοποιεί την κατανομή των πόρων του συστήματος, διασφαλίζοντας ελαχιστοποιημένη καθυστέρηση (low latency) καθ' όλη τη διαδρομή ελέγχου (I/O, επεξεργασία δεδομένων και προβολή διεπαφής).

Το inspire_datarecord είναι υπεύθυνο για την καταγραφή του συνόλου των δεδομένων από όλες τις πηγές. Συνδυάζει και ενοποιεί όλα τα δεδομένα. Δημιουργεί δύο αρχεία: τα δεδομένα GPIO (τα οποία δεν αφορούν αυτήν την εργασία) και τα δεδομένα OpenBCI.

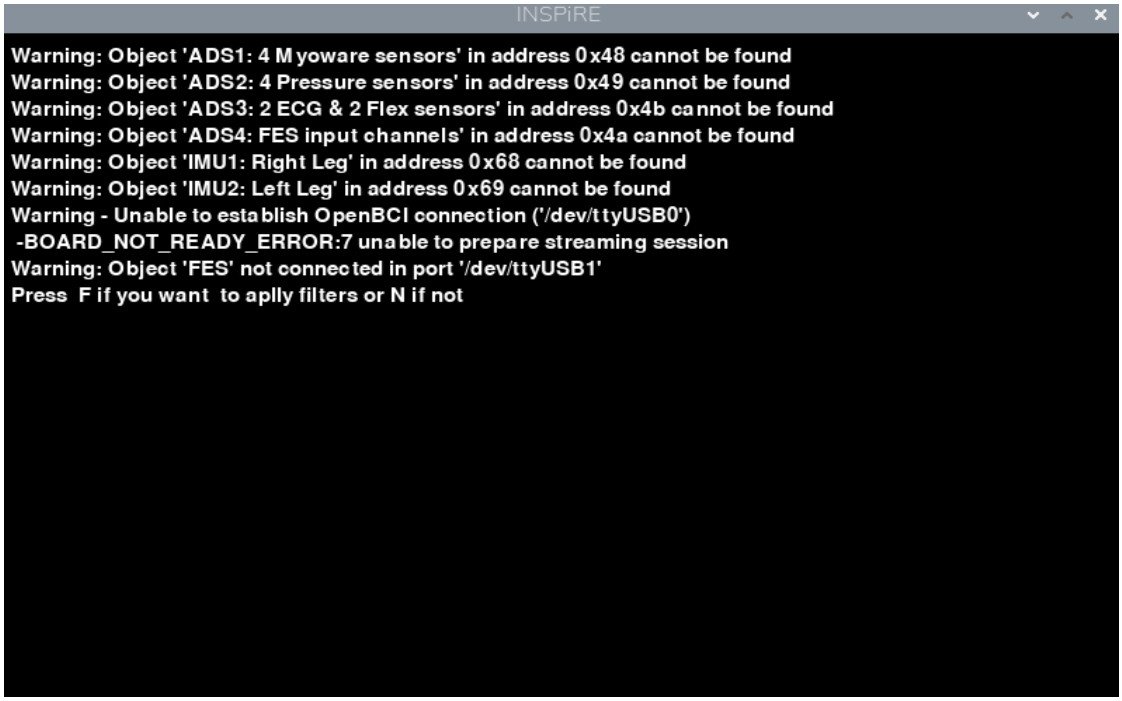
Το inspire_dataprocess λαμβάνει τα δεδομένα σε πραγματικό χρόνο και παράγει μία έξοδο. Στο παρελθόν, αυτή η διαδικασία γινόταν μόνο στο task type 1, δηλαδή στο βάδισμα, και η έξοδος κατευθυνόταν με προορισμό το FES, με τις συναρτήσεις data_process και fes_send αντίστοιχα. Το task type 2 για το ηλεκτρικό αμαξίδιο, στο οποίο θα εργαστούμε, ήταν άδειο ως προς την έξοδο και την επεξεργασία, έχοντας ως είσοδο 130 μετρήσεις του EEG, που αντιστοιχούν σε 1,04 δευτερόλεπτα μετρήσεων.

Το inspire_interface διαχειρίζεται κεντρικά το Γραφικό Περιβάλλον (GUI) του συστήματος συνδυάζοντας τις βιβλιοθήκες Pygame και Matplotlib. Οι τρεις βασικές του λειτουργίες είναι: η αρχικοποίηση της διεπαφής, η δυναμική οπτικοποίηση των μετρήσεων των αισθητήρων σε πραγματικό χρόνο (live feedback ραβδογραμμάτων) και η συνεχής παρακολούθηση των ενεργειών του χρήστη (όπως το πάτημα του Escape) για τον ασφαλή, ελεγχόμενο τερματισμό τόσο του λογισμικού όσο και της καταγραφής δεδομένων.

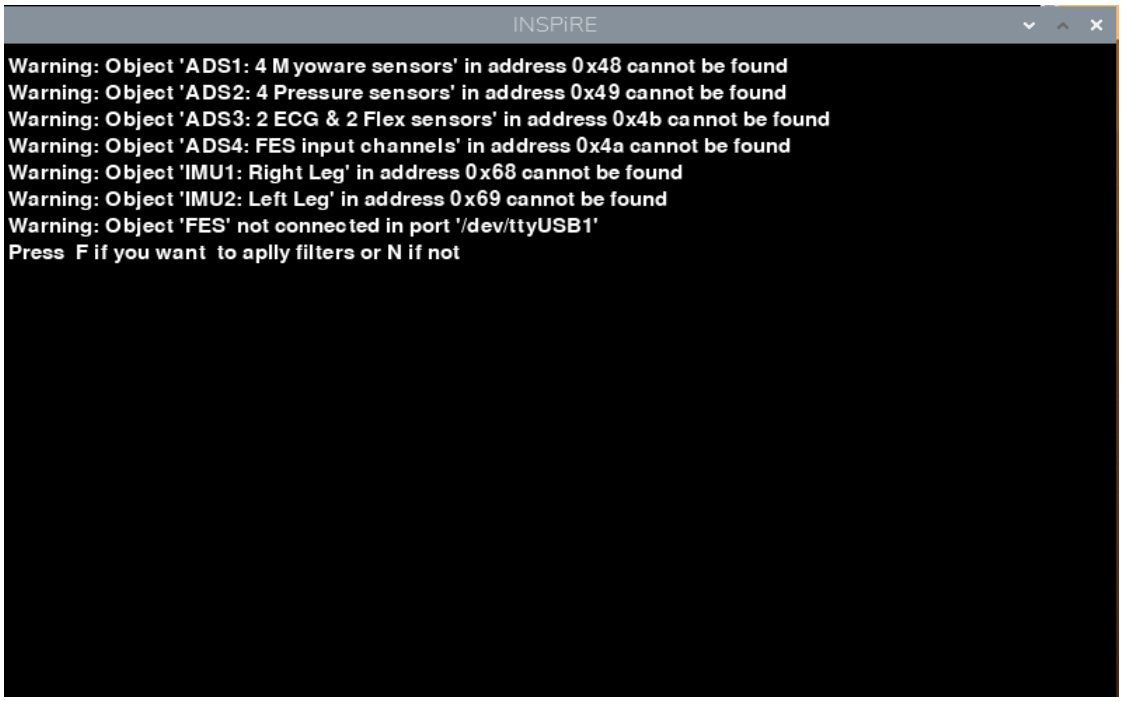
Στην αρχή του προγράμματος, στη διεπαφή με τον χρήστη, εμφανίζονται μηνύματα για το ποιες συνδέσεις εξοπλισμού έχουν επιτευχθεί. Συγκεκριμένα, σε αυτό το έργο χρησιμοποιείται μόνο του συστήματος του EEG. Η θέση που αντιστοιχεί στο dongle στο αρχικό πρόγραμμα ήταν η /dev/ttyUSB1, η οποία τροποποιήθηκε στην /dev/ttyUSB0.



Εικόνα 2.7: *Στιγμιότυπο οθόνης του κώδικα python στο inspire_main. Στιγμιότυπο από προβλήματα διασύνδεσης στη διεπαφή χρήστη (interface) του λογισμικού..Παρατηρείται αποτυχία επικοινωνίας με το USB dongle. Επίσης, σε αυτό το παράδειγμα, η θέση (διαδρομή) που αντιστοιχεί στο dongle ήταν η /dev/ttyUSB0 και όχι η αρχική θέση /dev/ttyUSB1.*



Εικόνα 2.8: *Στιγμιότυπο οθόνης του κώδικα python στο inspire_main. Στιγμιότυπα από προβλήματα διασύνδεσης στη διεπαφή χρήστη (interface) του λογισμικού. Γίνεται επιτυχημένη επικοινωνία με το USB dongle, αλλά υπάρχει αποτυχία σύνδεσης με το σύστημα του EEG. Επίσης, σε αυτό το παράδειγμα, η θέση (διαδρομή) που αντιστοιχεί στο dongle ήταν η /dev/ttyUSB0 και όχι η αρχική θέση /dev/ttyUSB1.*



***Εικόνα 2.9**: Στιγμιότυπο οθόνης του κώδικα python στο inspire_main. Φαίνεται η επιτυχημένη διασύνδεση και επικοινωνία του εξοπλισμού. Επίσης, σε αυτό το παράδειγμα, η θέση (διαδρομή) που αντιστοιχεί στο dongle ήταν η /dev/ttyUSB0 και όχι η αρχική θέση /dev/ttyUSB1.*

Έπειτα, στη διεπαφή χρήστη εμφανίζεται ένα μήνυμα που ρωτά τον χρήστη αν θέλει να συνεχίσει ή να αποχωρήσει από το πρόγραμμα. Αν αυτός επιλέξει να ξεκινήσει, οι μετρήσεις ξεκινούν και ανοίγει ένα παράθυρο με ανάδραση πραγματικού χρόνου, το οποίο παρέχει πληροφορίες σχετικά με το ύψος της μέτρησης κάθε αισθητήρα που έχει συνδεθεί στους ακροδέκτες GPIO του Raspberry Pi. Όπως έχει προαναφερθεί, στο πλαίσιο αυτής της εργασίας αυτοί δεν χρησιμοποιήθηκαν και, κατά συνέπεια, το παράθυρο ήταν κενό.

Ο τερματισμός της καταγραφής γίνεται πατώντας το πλήκτρο Escape στο πληκτρολόγιο. Μετά την ολοκλήρωση, εμφανίζεται μήνυμα στο παράθυρο εντολών με το όνομα και τη θέση των αποθηκευμένων αρχείων, τα οποία έχουν μορφή .csv.

3. Μεθοδολογία

Όπως προαναφέρθηκε, σκοπός της εργασίας είναι η δημιουργία ενός φορητού συστήματος, το οποίο επικοινωνεί με έναν EEGκαι έχει τη δυνατότητα, σε πραγματικό χρόνο, της εφαρμογής φίλτρων, εξαγωγής χαρακτηριστικών και, τέλος, λήψης αποφάσεων. Παράλληλα, το σύστημα αποθηκεύει τα δεδομένα της καταγραφής, ως μια βάση για μετέπειτα έρευνες.

Στόχος είναι η λήψη δεδομένων από φορητό EEG σύστημα, η εφαρμογή τεχνικών ψηφιακής επεξεργασίας σήματος σε πραγματικό χρόνο και η εξαγωγή χρήσιμων νευροφυσιολογικών χαρακτηριστικών, χρησιμοποιώντας έναν μικροεπεξεργαστή Raspberry Pi με έναν EEGOpenBCI 16 καναλιών. Το σύστημα αυτό έχει ευελιξία ως προς την εφαρμογή στην επεξεργασία του σήματος, την εξαγωγή των χαρακτηριστικών και τη λήψη των αποφάσεων. Παράλληλα, αποθηκεύει τα δεδομένα κάθε συνεδρίας, τα οποία μπορούν να χρησιμοποιηθούν για την εκπαίδευση μοντέλων. Επίσης, μπορεί να γίνει άμεση απεικόνιση των αποτελεσμάτων για την αξιολόγηση της λειτουργίας του συστήματος στο φιλτράρισμα, στα χαρακτηριστικά και στη λήψη της απόφασης. Επιπρόσθετα, γίνεται προβολή του πόσο χρόνο παίρνει αυτή η διαδικασία και το πόσο σκληρά δουλεύει ο επεξεργαστής του Raspberry Pi, για την αποφυγή μελλοντικών προβλημάτων που μπορούν να προκύψουν.

3.1 Βήματα υλοποίησης:

Η υλοποίηση έγινε σε δύο κομμάτια. Αρχικά, έπρεπε να στηθεί ξανά το ήδη υπάρχον σύστημα INSPiRE, ως μια βάση. Έγιναν αλλαγές ώστε να μπορεί να ξαναλειτουργήσει αυτό το σύστημα με τις πιο πρόσφατες εκδόσεις των προγραμμάτων.

Έγινε η επικοινωνία του Raspberry Pi με τον υπολογιστή για την απεικόνιση μέσω PuTTY και RealVNC Viewer, η σύνδεση του Raspberry Pi με το διαδίκτυο και η επικοινωνία του Raspberry Pi με το OpenBCI USB Dongle, καθώς επίσης και η φόρτωση του κατάλληλου λογισμικού στην πλακέτα του EEG με το Arduino IDE, από τον υπολογιστή μέσω του Dongle.

Έπειτα, στο δεύτερο κομμάτι, στο ήδη υπάρχον σύστημα προστέθηκαν τα εργαλεία που δίνουν τη δυνατότητα του φιλτραρίσματος του σήματος σε πραγματικό χρόνο και, μετέπειτα, με ανεξάρτητο τρόπο, την εξαγωγή χαρακτηριστικών από το σήμα και, με βάση τα εν λόγω χαρακτηριστικά, τη λήψη μιας απόφασης.

Για τις εν λόγω διαδικασίες, οι επιλογές ήταν ενδεικτικές και όχι για την πραγματική λήψη αποφάσεων, αλλά για την επίδειξη ότι αυτό μπορεί να στηθεί εύκολα. Βοηθητικά για αυτήν τη διαδικασία δημιουργήθηκαν και 7 ανεξάρτητα προγράμματα για την οπτικοποίηση των φίλτρων, πραγματικού και μη πραγματικού χρόνου, με σκοπό τον έλεγχο της σωστής λειτουργίας τους.

Ακόμη, οι τιμές των εξαγόμενων χαρακτηριστικών και η απόφαση μπορούν εύκολα να εκτυπώνονται στο command window (παράθυρο εντολών). Για την αποφυγή μελλοντικών προβλημάτων, εκτυπώνεται και η καθυστέρηση, ο φόρτος του επεξεργαστή και η θερμοκρασία στη διαδικασία «φίλτρο - εξαγωγή χαρακτηριστικών - απόφαση», καθώς μελλοντικά πιο σύνθετα μοντέλα πιθανόν να αντιμετωπίσουν

προβλήματα με τις περιορισμένες δυνατότητες ενός Raspberry Pi σε αυτήν τη σχετικά δύσκολη διαδικασία.

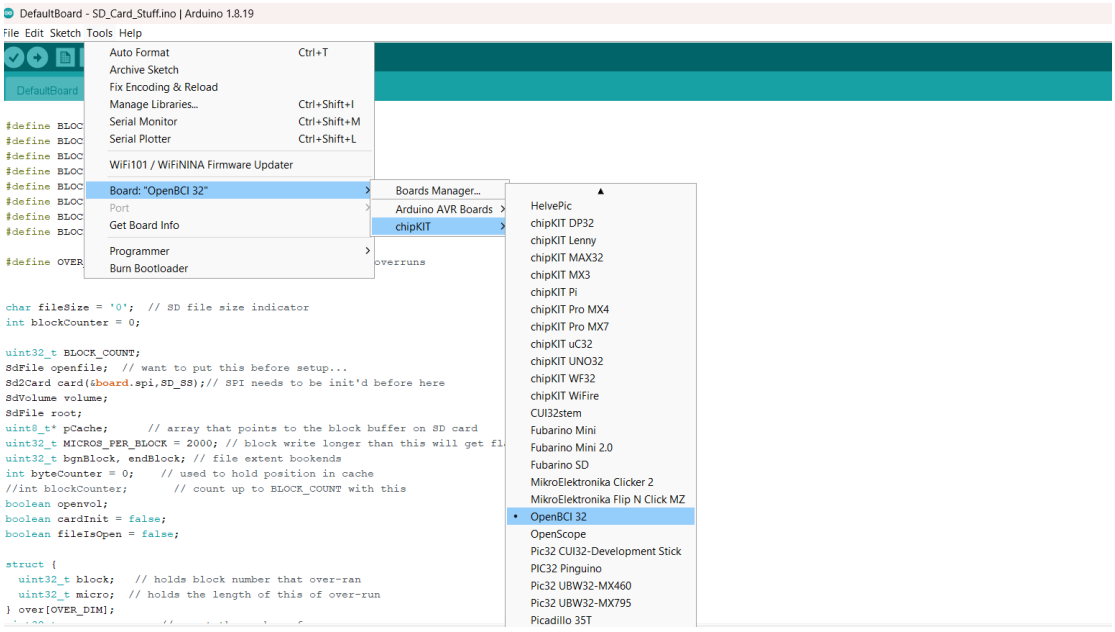
3.2 ΠΡΑΚΤΙΚΗ ΥΛΟΠΟΙΗΣΗ:

Θα αναλυθεί ξεχωριστά καθεμία από τις διαδικασίες ως οδηγίες για την αναδημιουργία του έργου και θα αναφερθούν μερικά προβλήματα που αντιμετωπίστηκαν κατά τη διαδικασία αυτή.

3.2.1 Φόρτωση firmware

Αρχίζοντας με την φόρτωση του κατάλληλου λογισμικού στην πλακέτα Cyton. Στην επίσημη ιστοσελίδα της OpenBCI δίνονται αναλυτικές οδηγίες για την εγκατάσταση του υλικολογισμικού, άρα για λόγους πληρότητας και αναφοράς προβλημάτων, θα αναφερθεί και εδώ περιληπτικά.

Θα χρησιμοποιηθούν τα πακέτα OpenBCI_32bit_Library, OpenBCI_32bit_SD και OpenBCI_Wifi_Master. Για την εγκατάσταση αυτών των πακέτων χρησιμοποιήθηκε το Arduino IDE (1.8.19).



***Εικόνα 3.1**: Στιγμιότυπο οθόνης του Arduino IDE (1.8.19) κατά την εγκατάσταση του λογισμικού στο firmware.*

Ο διακόπτης της πλακέτας cyton πρέπει να βρίσκεται στο υπολογιστή και ο διακόπτης του dongle στο GPIO6 mode. Η πλακέτα Cyton πρέπει να μπει σε λειτουργία «Bootloader». Αυτό γίνεται με τον εξής τρόπο

- Απενεργοποίηση πλακέτας (Power OFF)
- Ταυτόχρονο παρατεταμένο πάτημα κουμπιών RST και PROG
- Ενεργοποίηση πλακέτας (Power ON)
- Απελευθέρωση του κουμπιού RST
- Απελευθέρωση του κουμπιού PROG

Εφόσον η διαδικασία ολοκληρωθεί επιτυχώς, θα αρχίσει να αναβοσβήνει η μπλε ενδεικτική λυχνία (LED) στην πλακέτα.

Στο σημείο αυτό αξίζει να σημειωθεί ότι αντιμετωπίστηκε ένα πρόβλημα σύνδεσης και επικοινωνίας για αδιευκρίνιστο λόγο. Το πρόβλημα επιλύθηκε όταν πραγματοποιήθηκε σύνδεση για έλεγχο μέσω του επίσημου λογισμικού OpenBCI GUI. Εκτιμάται ότι έλαβε χώρα κάποια διαδικασία αρχικοποίησης (initialization) της σύνδεσης μεταξύ των συστημάτων, η οποία επέτρεψε εν συνεχεία την ορθή επικοινωνία με το περιβάλλον ανάπτυξης του Arduino IDE.

Το OpenBCI GUI δεν αποτελεί μέρος του παρόντος έργου, ωστόσο συνιστά ένα ιδιαίτερα χρήσιμο εργαλείο για την επαλήθευση της ορθής λειτουργίας του συστήματος EEG πριν από τη διασύνδεσή του με το Raspberry Pi. Παρέχει εργαλεία τα οποία θα ήταν σκόπιμο να ενσωματωθούν μελλοντικά στο προτεινόμενο πρόγραμμα, κυρίως όσον αφορά τον έλεγχο της ορθής λειτουργίας (π.χ. ποιότητα επαφής) των αισθητήρων.

3.2.2 Επικοινωνία Raspberry Pi με τον υπολογιστή

Αρχικά, με το Raspberry Pi Imager φορτώνεται το λογισμικό Raspberry Pi OS στην κάρτα SD. Επιπρόσθετα, προσθέτεται στην κάρτα ένα αρχείο με ονομασία SSH (χωρίς κατάληξη) για τη σύνδεση με το διαδίκτυο· συγκεκριμένα επιλέχθηκε ένα ασύρματο σημείο πρόσβασης μέσω κινητού (hotspot) στα 2.4GHz. Μέσω των ρυθμίσεων των Windows, ενεργοποιούμε την κοινή χρήση στις επιλογές σύνδεσης: ενεργοποίηση σύνδεσης άλλων χρηστών, απενεργοποίηση άδειας για έλεγχο, απενεργοποίηση κοινόχρηστης σύνδεσης και επιλογή θύρας σύνδεσης του Raspberry Pi από τη λίστα «Επιλογή σύνδεσης ιδιωτικού δικτύου». Το όνομα ορίστηκε ως raspberry, ο κωδικός ως pi και το hostname ως raspberrypi.

Για την επιβεβαίωση της επικοινωνίας, συνιστάται στο cmd να χρησιμοποιηθεί η εντολή “ping raspberrypi.local”. Σχολιάζεται ότι σε αυτό το βήμα υπήρχε πρόβλημα με την επικοινωνία σε κάποιο μη γνωστό επίπεδο, το οποίο επιλύθηκε μετά από τη χρήση ενός USB-Ethernet adapter για τη δημιουργία της αρχικής επικοινωνίας, και η σύνδεση έγινε χειροκίνητα στο interface του RealVNC Viewer. Έπειτα από αυτό, δεν ήταν απαραίτητη η χρήση του USB-Ethernet adapter.

Αρχικά, έγινε σύνδεση SSH μέσω του PuTTY για την ενεργοποίηση του VNC interface. Έπειτα, έγινε η σύνδεση μέσω του VNC Viewer για τη μεγαλύτερη άνεση που προσφέρει το περιβάλλον στη χρήση.

Μετά την επιβεβαίωση της σωστής επικοινωνίας του συστήματος, το επόμενο βήμα είναι η εγκατάσταση του λογισμικού που είναι απαραίτητο για τη λειτουργία του INSPiRE. Αρχικά, εκτελείται εκτενής ενημέρωση και αναβάθμιση των αποθετηρίων του λειτουργικού συστήματος και του διαχειριστή πακέτων της Python (pip/setuptools). Στη συνέχεια, υλοποιούνται εξειδικευμένες ρυθμίσεις που αφορούν τη διαχείριση του υλικού και των σημάτων, όπως η τροποποίηση των παραμέτρων του πρωτοκόλλου ALSA για την αποσιώπηση σφαλμάτων, η ενημέρωση της βιβλιοθήκης wiringPi για την ορθή διαχείριση των ακροδεκτών GPIO και η καθολική ενεργοποίηση των διαθέσιμων διεπαφών επικοινωνίας (interfaces). Η αρχιτεκτονική του συστήματος επεκτείνεται περαιτέρω μέσω της εγκατάστασης του συστήματος διαχείρισης σχεσιακών βάσεων δεδομένων MariaDB και της διασφάλισης του αυτοματοποιημένου χρονικού συγχρονισμού μέσω πρωτοκόλλου NTP. Τέλος, πραγματοποιείται η εγκατάσταση της σουίτας λογισμικού LibreOffice και η ενσωμάτωση του περιβάλλοντος διαχείρισης πακέτων Miniconda. Η χρήση του Miniconda απαιτεί τη

δημιουργία και ενεργοποίηση ενός απομονωμένου εικονικού περιβάλλοντος (virtual environment). Αυτό διασφαλίζει την εγκατάσταση όλων των προαναφερθέντων εξαρτήσεων χωρίς να προκαλούνται διενέξεις με τα προεγκατεστημένα πακέτα (PEP 668) του Raspberry Pi OS.

Σε συνέχεια της αρχιτεκτονικής του συστήματος, η επεξεργασία των βιοσημάτων και η εύρυθμη λειτουργία της διάταξης εξασφαλίζονται μέσω ενός ολοκληρωμένου οικοσυστήματος βιβλιοθηκών της Python, το οποίο καλύπτει γραμμικά τη ροή δεδομένων από τη λήψη έως την παραγωγή φυσικού έργου. Αρχικά, η βιβλιοθήκη BrainFlow αποτελεί τον κρίσιμο πυρήνα της διασύνδεσης υλικού, αναλαμβάνοντας την αδιάλειπτη λήψη και μετάφραση των ακατέργαστων (raw) πακέτων Bluetooth από τις μονάδες OpenBCI σε μετρήσιμες αριθμητικές τιμές. Ακολούθως, οι βιβλιοθήκες NumPy και SciPy λειτουργούν ως η υπολογιστική μηχανή της διεργασίας ψηφιακής επεξεργασίας σήματος (DSP), εκτελώντας προηγμένες αναλύσεις, όπως Ταχείς Μετασχηματισμούς Fourier (FFT), για το φιλτράρισμα του θορύβου και την απομόνωση των εγκεφαλικών συχνοτήτων σε εξειδικευμένες ζώνες (όπως τα κύματα Θήτα και Βήτα). Τα επεξεργασμένα αυτά δεδομένα οπτικοποιούνται αυστηρά μέσω των Matplotlib και PyLab, ενώ η βιβλιοθήκη Pygame διαχειρίζεται το κεντρικό γραφικό περιβάλλον (GUI) και καταγράφει τα ασύγχρονα συμβάντα ελέγχου (όπως εντολές πληκτρολογίου για την εφαρμογή φίλτρων). Τέλος, το τελικό στάδιο της διεπαφής εγκεφάλου-υπολογιστή (BCI) ολοκληρώνεται μέσω της RPi.GPIO (ή gpiozero), η οποία επιτρέπει την εξαγωγή της τελικής απόφασης του συστήματος ως αποτέλεσμα (λογικό 0 ή 1) προς τους φυσικούς ακροδέκτες του μικροελεγκτή (π.χ. Pin 17), επιτρέποντας την εξαγωγή απτών εντολών ελέγχου σε φυσικό υλικό.

Η διασύνδεση λογισμικού-υλικού σε περιβάλλον Linux προϋποθέτει την υπέρβαση εγγενών περιορισμών του συστήματος. Αφενός, για την απρόσκοπτη επικοινωνία της βιβλιοθήκης BrainFlow με τον δέκτη USB του OpenBCI, απαιτείται η εκχώρηση δικαιωμάτων σειριακής πρόσβασης μέσω της ένταξης του χρήστη στις ομάδες dialout ή tty. Αφετέρου, για την ορθή χωρική τοποθέτηση του γραφικού περιβάλλοντος (Pygame/Matplotlib) που εκτελείται σε διεργασίες παρασκηνίου, είναι απαραίτητη η ρύθμιση μεταβλητών περιβάλλοντος (π.χ. os.environ['SDL_VIDEO_CENTERED'] = '1'), εξαναγκάζοντας τον διαχειριστή παραθύρων να δρομολογήσει την απεικόνιση στην επιθυμητή θέση.

Η μετάβαση στο πρωτόκολλο απεικόνισης X11 καθίσταται επιβεβλημένη λόγω των αυστηρών πολιτικών ασφαλείας του νεότερου Wayland, το οποίο απαγορεύει ρητά την προβολή γραφικών περιβαλλόντων από διεργασίες με δικαιώματα υπερχρήστη (root/sudo). Καθώς η επικοινωνία του λογισμικού με το υλικό χαμηλού επιπέδου (σειριακές θύρες USB και ακροδέκτες GPIO) απαιτεί προνόμια διαχειριστή, το Wayland μπλοκάρει τη διεπαφή Pygame. Η επιστροφή στο παλαιότερο και πιο ευέλικτο σύστημα X11 επιτρέπει την ελεγχόμενη παράκαμψη αυτού του περιορισμού (π.χ. μέσω της εντολής `xhost +`), διασφαλίζοντας την ταυτόχρονη και απρόσκοπτη εκτέλεση του κώδικα ελέγχου υλικού και της οπτικής απεικόνισης.

Επιπλέον, για την πλήρη λειτουργία του συστήματος είναι απαραίτητη η εγκατάσταση των παρακάτω βοηθητικών βιβλιοθηκών Python: pyserial, adafruit-blinka, smbus2, adafruit-circuitpython-ads1x15, pylsl, pylab και pyasyncore (λόγω αφαίρεσης του asyncore από τις νεότερες εκδόσεις της Python), mariadb, matplotlib και scipy, οι οποίες δεν ήταν απολύτως απαραίτητες για τη βασική λειτουργία, αλλά χρησιμοποιήθηκαν μετέπειτα.

3.2.3 Περιβάλλον διεργασίας

Στο υποκεφάλαιο αυτό θα παρουσιαστεί το περιβάλλον διεργασίας που βρίσκεται μέσα στο Raspberry Pi και η δομικη λειτουργίας του προγράμματος. Το περιβάλλον αυτό δημιουργήθηκε για την πιο εύκολη ανάπτυξη του λογισμικού μέσα στο Raspberry Pi ώστε με RealVNC Viewer να προβάλλεται μια επιφάνια εργασίας παροιμία με αυτή του υπολογιστή.

Τα προγράμματα του INSPiRE μπήκαν μέσα στο /home/raspberry/Desktop/inspire και δημιουργήθηκε εικονικό περιβάλλον με τα πακέτα που χρειάζονται ως brainflow_env. Οι καταγραφές αποθηκεύονται μέσα στον φάκελο /home/raspberry/Desktop/Documents, ο οποίος περιέχει υποφακέλους με τα ονόματα του κάθε χρήστη. Μέσα σε αυτούς του φακέλους με τα ονόματα χρήστη υπάρχουν αριθμημένα με σειρά τα αρχεία των καταγραφών. Επίσης, υπάρχουν 2 βοηθητικά προς τους χρήστες αρχεία κειμένου: ένα για την εκκίνηση του προγράμματος (commands) και ένα για τα προγράμματα πειραματισμού-ελέγχου (inspiret).



***Εικόνα 3.2**: Στιγμιότυπο οθόνης της επιφάνειας εργασίας του Raspberry Pi*

Θα επεξηγηθεί περιληπτικά η βασική δομή του λογισμικού INSPiRE, για την κατανόηση των προσθηκών που υλοποιήθηκαν σε αυτό, καθώς και των αλλαγών που έγιναν ώστε να μπορέσει να λειτουργήσει σε σύγχρονο λογισμικό. Θα αναλυθούν περιληπτικά μόνο αυτά που αφορούν το έργο, σύμφωνα με τη σειρά που έχουν μέσα στο πρόγραμμα.

3.2.4 Λειτουργικές τροποποιήσεις

Για την εργασία αυτή χρησιμοποιήθηκαν κυρίως οι τελευταίες εκδόσεις των προγραμμάτων και, κατά συνέπεια, αυτό είχε ως επίπτωση να πρέπει να αλλάξει η μορφή του κύριου προγράμματος πριν αυτό μπορέσει να λειτουργήσει. Αυτές οι αλλαγές ήταν κυρίως μικρές, με το μόνο μεγάλο πρόβλημα να προκύπτει στην ανανέωση του γραφικού περιβάλλοντος.

Αρχικά, μετά την έκδοση 3.0.0 της βιβλιοθήκης της Adafruit, η ονοματολογία των ακροδεκτών (pins) απλοποιήθηκε: από ADS.P0, ADS.P1, ADS.P2, ADS.P3 έγινε απλά

0, 1, 2, 3. Παράλληλα, στις νεότερες εκδόσεις της βιβλιοθήκης Matplotlib (έκδοση 3.8+), η εντολή raw_data = renderer.tostring_rgb() πρέπει να αντικατασταθεί από την raw_data = renderer.tostring_argb() και, αντίστοιχα, η εντολή surf = pygame.image.fromstring(raw_data, size, "RGB") σε surf = pygame.image.fromstring(raw_data, size, "ARGB"). Ακόμη, υπήρχε ένα συντακτικό λάθος διάταξης στο inspire_datarecord σχετικά με τις εσοχές του κώδικα (indentation) και τις εξαρτώμενες εντολές.

Το μεγαλύτερο πρόβλημα και, συνεπώς, η σημαντικότερη αλλαγή που έγινε, οφειλόταν στο γεγονός ότι στις σύγχρονες εκδόσεις του Pygame δεν επιτρέπεται η ενημέρωση ενός παραθύρου γραφικών που χρησιμοποιεί επιτάχυνση υλικού (hardware acceleration) από ένα δευτερεύον νήμα εκτέλεσης (background thread). Παρά τις προσπάθειες, δεν κατέστη δυνατή η παράκαμψη αυτού του μέτρου ασφαλείας. Στο inspire_interface απενεργοποιήθηκε η εντολή pygame.display.update() και μεταφέρθηκε στο inspire_main, όπου και προστέθηκε στο κομμάτι του κώδικα στο οποίο γίνεται ο έλεγχος λειτουργίας των νημάτων.

Επιπλέον, όπως προαναφέρθηκε, η θύρα USB που αντιστοιχεί στον EEGάλλαξε ονομασία, ανταλλάσσοντας θέση με αυτήν του FES. Το dongle μεταφέρθηκε από τη θέση /dev/ttyUSB1 στη θέση /dev/ttyUSB0 και το FES, αντίστοιχα, από τη θέση /dev/ttyUSB0 στη θέση /dev/ttyUSB1.

Ακόμη μία αλλαγή που έγινε αφορούσε τα ονόματα των επικεφαλίδων των αρχείων Excel (.csv). Οι επικεφαλίδες (όπως Fp1, Fp2, C3, C4, P7, P8, O1...) διαχωρίζονταν με κόμμα (,) αντί για ελληνικό ερωτηματικό (;), με αποτέλεσμα να καταχωρούνται όλα στο ίδιο κελί στο Excel. Αυτό συνέβη πιθανώς λόγω της διαφορετικής έκδοσης του Excel που χρησιμοποιήθηκε και ίσως να μην είναι απαραίτητο για όλους τους χρήστες.

Αφότου έγιναν αυτές οι αλλαγές και το πρόγραμμα μπόρεσε να λειτουργήσει επιτυχώς, δημιουργήθηκε ένα αντίγραφο ασφαλείας του λογισμικού με τις αλλαγές που ήταν αναγκαίες για τη λειτουργία του.

Αφότου επιβεβαιώθηκε ότι όλη η λειτουργία του προγράμματος δούλευε σωστά, ξεκίνησε το προγραμματιστικό έργο το οποίο αφορά τα δεδομένα του EEG. Οι προσθήκες στο πρόγραμμα έγιναν με γνώμονα τη δυνατότητα μελλοντικών προσθηκών και επεκτάσεων, όπως προαναφέρθηκε. Η δομή κρατήθηκε παρόμοια με αυτή του αρχικού προγράμματος για τη συνοχή στη μορφή του κώδικα και την αποφυγή σύγχυσης των μελλοντικών προγραμματιστών που θα εργαστούν πάνω στο έργο.

3.2.5 Λογισμικό φιλτραρίσματος

Αρχικά, στη διεπαφή χρήστη, ο χρήστης ερωτάται αν επιθυμεί να χρησιμοποιηθούν φίλτρα. Η επιλογή αυτή έγινε ορίζοντας μια μεταβλητή filt, ως True για τη χρήση φίλτρων και ως False για τη μη χρήση τους. Μέσα στη συνάρτηση επιβεβαίωσης της έναρξης της καταγραφής (inspire_interface.confirm), πριν γίνει η ερώτηση για την καταγραφή, προστέθηκε κώδικας ο οποίος προτρέπει τον χρήστη: «Αν επιθυμείς να χρησιμοποιήσεις φίλτρα πάτα F, και αν όχι πάτα N», θέτοντας την αντίστοιχη μεταβλητή filt σε True ή False. Έπειτα, το αν θα εφαρμοστούν τα φίλτρα ή όχι

προβάλλεται με ένα μήνυμα στη διεπαφή του χρήστη. Αυτή η επιλογή, βέβαια, θα μπορούσε εύκολα, με λίγες αλλαγές, να ενσωματωθεί στις υπόλοιπες επιλογές που ορίζονται στο inspire_main, όπου έχει ήδη τεθεί προληπτικά ως προεπιλογή η τιμή False.

Το επόμενο βήμα είναι η εφαρμογή των φίλτρων. Επιλέχθηκε να χρησιμοποιηθούν δύο ανεξάρτητα φίλτρα για τα δεδομένα: ένα κατά την επεξεργασία τους σε πραγματικό χρόνο και ένα κατά την αποθήκευση. Η επιλογή αυτή έγινε για πρακτικούς λόγους, και συγκεκριμένα για:

1. Την εξοικονόμηση των υπολογιστικών πόρων του συστήματος κατά τη λειτουργία της καταγραφής.
2. Την αποφυγή ριζικών αλλαγών στη δομή του προγράμματος.
3. Την καλύτερη ποιότητα φιλτραρίσματος στα δεδομένα μη πραγματικού χρόνου (offline) και τη δυνατότητα για πιο περίπλοκες μορφές επεξεργασίας.

Αυτό, βέβαια, μπορεί να αλλάξει μελλοντικά σχετικά εύκολα, αν αποδειχθεί πειραματικά ότι είναι πιο χρήσιμο να υπάρχει ακριβώς η ίδια επεξεργασία στα δεδομένα τα οποία χρησιμοποιούμε για την εκπαίδευση με αυτά του πραγματικού χρόνου, τα οποία χρησιμοποιούνται για τη λήψη μιας απόφασης.

Αφού γίνει η αποθήκευση των αρχικών αρχείων στο κεντρικό πρόγραμμα με την εντολή inspire_datarecord.save_sessiondata, καλείται η συνάρτηση inspire_datarecord.save_filteredeegdata, εφόσον, προφανώς, έχει θέτει τη μεταβλητή filt ως αληθή (True). Αυτή η διαδικασία ακολουθεί τα εξής βήματα:

- **Φάση 1 (Ανάγνωση και Διαχωρισμός):** Διαβάζει το αρχείο εξόδου (.csv) των ακατέργαστων (raw) καταγραφών και προβαίνει σε διαχωρισμό (parsing) των επικεφαλίδων (headers) από τον κύριο όγκο των ακατέργαστων αριθμητικών δεδομένων.
- **Φάση 2:** Τροφοδοτεί τα αριθμητικά δεδομένα στη ρουτίνα ψηφιακής επεξεργασίας inspire_filters.apply_frequency_filters, η οποία θα αναλυθεί στη συνέχεια. Αυτή εφαρμόζει ένα φίλτρο συχνοτήτων στα ακατέργαστα δεδομένα και τα επιστρέφει στο inspire_datarecord.
- **Φάση 3:** Τα πλέον καθαρά δεδομένα ανασυνδυάζονται με τις επικεφαλίδες σε μία ενιαία δομή.
- **Φάση 4:** Αποθηκεύονται σε ένα καινούργιο, ξεχωριστό αρχείο (ορισμένο στη μεταβλητή pars.config.fopenbcidatafile).
- Η μεταβλητή pars.config.fopenbcidatafile περιέχει την ονομασία filteredopenbcirecordings.csv, η οποία έχει οριστεί μέσα στο configparams του inspire_config. Το αρχείο λαμβάνει αυτήν την ονομασία και αποθηκεύεται στον ίδιο φάκελο μαζί με τα υπόλοιπα, διατηρώντας το αρχικό αρχείο της καταγραφής ανέπαφο.

Το **Inspire_filters** δημιουργήθηκε ώστε οι συναρτήσεις (functions) των φίλτρων να βρίσκονται σε μια συγκεντρωμένη δομή. Επί του παρόντος, περιλαμβάνει δύο συναρτήσεις: την προαναφερόμενη apply_frequency_filters, η οποία δρα στα

αποθηκευμένα δεδομένα, και την filter_live_chunk, η οποία δρα στα δεδομένα πραγματικού χρόνου. Τα αποτελέσματα αυτών των φίλτρων μπορούν να εξεταστούν στο κεφάλαιο 4.

Η apply_frequency_filters αποτελεί ένα απλό φίλτρο. Αρχικά, μετατρέπει τα ακατέργαστα δεδομένα σε πίνακα της βιβλιοθήκης NumPy για την εκτέλεση αποδοτικών μητρωικών υπολογισμών. Βάσει του ορισμένου ρυθμού δειγματοληψίας (125 Hz), εφαρμόζει σειριακά σε κάθε καταγεγραμμένο κανάλι (στήλες 1 έως 16) κρίσιμα ψηφιακά φίλτρα IIR (Infinite Impulse Response), τα οποία είναι κατάλληλα για την εξοικονόμηση των υπολογιστικών πόρων. Η εφαρμογή γίνεται μέσω της συνάρτησης μηδενικής καθυστέρησης φάσης filtfilt, η οποία αποτρέπει τη χρονική παραμόρφωση του βιοσήματος, διατηρώντας κάθε πιθανή αντίδραση στον ακριβή χρόνο που συνέβη, για τη σωστή αναγνώρισή της.

Εφαρμόζεται ένα φίλτρο αποκοπής ζώνης (notch filter) στα 50 Hz, με υψηλό συντελεστή ποιότητας (Q=30), για τη στοχευμένη εξάλειψη της παρεμβολής του δικτύου ηλεκτροδότησης. Επίσης, λόγω ισχυρού θορύβου στα 25 Hz, που μάλλον οφείλεται σε υποαρμονική συχνότητα των 50 Hz του δικτύου, προστέθηκε ακόμη ένα IIR notch φίλτρο, με κέντρο τα 25 Hz και συντελεστή ποιότητας Q=52.

Ακολούθως, εφαρμόζεται ένα ζωνοπερατό φίλτρο (Bandpass Butterworth 8ης τάξης) στο αυστηρό εύρος 1-50 Hz, το οποίο απομονώνει τις ωφέλιμες εγκεφαλικές συχνότητες, απορρίπτοντας ταυτόχρονα τις χαμηλόσυχνες κινήσεις των ματιών (κάτω από 1 Hz) και τον υψίσυχνο μυϊκό θόρυβο (πάνω από 50 Hz).

Ακόμη, τα πρώτα και τα τελευταία 2 δευτερόλεπτα της καταγραφής αφαιρούνται, καθώς περιέχουν πολύ θόρυβο λόγω της «προθέρμανσης» (warming up) και της «επαναφοράς» (cooling down) του συστήματος και των φίλτρων. Τέλος, το καθαρό σήμα επανασυντίθεται και επιστρέφεται στη βασική δομή λίστας της Python.

Η συνάρτηση filter_live_chunk είναι το φίλτρο που εφαρμόζαιτε στα δεδομένα σε πραγματικό χρόνο και, για αυτόν τον λόγο, σχεδιάστηκε έτσι ώστε να λειτουργεί αποδοτικά ως προς τους υπολογιστικούς πόρους και να μπορεί να δράσει σε μικρά, μεμονωμένα τμήματα (chunks των 130 τιμών).

Αρχικά, για τη βέλτιστη χρήση των υπολογιστικών πόρων, έγινε χρήση των ήδη έτοιμων φίλτρων της βιβλιοθήκης BrainFlow, τα οποία έχουν βελτιστοποιηθεί για αυτόν τον σκοπό.

Το πρόβλημα, όταν εφαρμόζοντες φίλτρα συχνοτήτων σε μικρά τμήματα (chunks), είναι ότι αν εφαρμοστούν απευθείας, αυτό θα προκαλέσει παραμορφώσεις στα άκρα του σήματος (φαινόμενο γνωστό ως edge effects). Για να αντιμετωπιστεί αυτό το πρόβλημα, χρησιμοποιήθηκε η τεχνική που είναι γνωστή ως κυλιόμενη μνήμη (rolling buffer). Κατά την τεχνική αυτή, η συνάρτηση διατηρεί αποθηκευμένα 600 δείγματα (που αντιστοιχούν σε 4,8 δευτερόλεπτα), στα οποία εφαρμόζει τα φίλτρα συχνοτήτων. Εφαρμόζει τα φίλτρα αποκοπής του ηλεκτρικού δικτύου (Bandstop 48-52 Hz), της υποαρμονικής συχνότητας (Bandstop 24.5-25.5 Hz) και το ζωνοπερατό φίλτρο (Bandpass 1-50 Hz) στο σύνολο αυτής της μνήμης. Αυτό εξασφαλίζει ότι η μαθηματική εξομάλυνση του σήματος γίνεται βέλτιστα. Έπειτα, εξάγεται το τμήμα (chunk) που εισήχθη αρχικά, στο οποίο θα γίνει η εξαγωγή των χαρακτηριστικών, ενώ τα

προηγούμενα μη φιλτραρισμένα δεδομένα διατηρούνται στη μνήμη για να επαναχρησιμοποιηθούν στον επόμενο κύκλο.

Ο έλεγχος της σωστής λειτουργίας των φίλτρων του πραγματικού και μη πραγματικού χρόνου μπορεί να γίνει με άλλα προγράμματα τα οποία αναπτύχθηκαν και θα αναλυθούν μετέπειτα.

Το προηγούμενο φίλτρο εφαρμόζεται στο inspire_dataprocess στο task type 2 (αναπηρικό αμαξίδιο), καθώς εκεί μόνο χρησιμοποιείται το EEG systhma για την εξαγωγή απόφασης. Θα μπορούσε, βέβαια, να ενσωματωθεί και στο άλλο task type. Προφανώς, αυτό γίνεται μόνο όταν ο χρήστης έχει επιλέξει να χρησιμοποιηθούν τα φίλτρα. Έπειτα, τα πλέον καθαρισμένα δεδομένα αποθηκεύονται στη μεταβλητή clean_data, Προκειμένου να γίνει μετάβαση στο επόμενο βήμα, την εξαγωγή χαρακτηριστικών.

3.2.6 Εξαγωγή χαρακτηριστικών

Στο inspire_features βρίσκονται συγκεντρωμένες οι συναρτήσεις για την εξαγωγή των χαρακτηριστικών, με σκοπό την καλύτερη οργάνωση του κώδικα. Προς το παρόν, υπάρχει μόνο μία συνάρτηση η οποία εξάγει την ισχύ της ζώνης (band power), η extract_band_power.

Αυτή η συνάρτηση λαμβάνει ως είσοδο τα δεδομένα, καθώς και την ελάχιστη και μέγιστη συχνότητα της επιλεγμένης ζώνης, και επιστρέφει την ισχύ σε αυτήν τη ζώνη. Ως μέτρο ασφαλείας, σε περίπτωση που υπάρξει κάποιο σφάλμα (error), εκτυπώνεται ένα μήνυμα στο παράθυρο εντολών (command window) και επιστρέφεται η τιμή 0.0. Με αυτήν τη μορφή, καθίσταται εύκολη η μελλοντική επιλογή διαφορετικών συχνοτήτων για την εξαγωγή των χαρακτηριστικών.

Επί του παρόντος, η συνάρτηση καλείται δύο φορές στο inspire_dataprocess για την εξαγωγή της ισχύος στα κύματα Θήτα (5.0 - 8.0 Hz) και Βήτα (12.0 - 25.0 Hz). (Σημειώνεται ότι τα όρια των ζωνών ορίστηκαν ελαφρώς πιο στενά από τα θεωρητικά, προκειμένου να απομονωθεί καλύτερα το καθαρό σήμα.) Όπως έχει αναφερθεί, η επιλογή τους έγινε για την επίδειξη της λειτουργίας του συστήματος και όχι για την πραγματική λήψη αποφάσεων κίνησης.

3.2.7 Ταξινόμηση (Classification)

Έπειτα, συνεχίζοντας στο inspire_dataprocess, οι δύο μεταβλητές τις ισχύος των ζωνών που έχουν εξαχθεί συγκρίνονται μεταξύ τους. Όταν η ισχύς των κυμάτων Θήτα είναι διπλάσια από την ισχύ των κυμάτων Βήτα, λαμβάνεται η απόφαση ως αληθής (True / command 1) ως προς την έξοδο. Σε αυτήν την εργασία, η συγκεκριμένη έξοδος αντιστοιχεί στο άναμμα ενός κόκκινου LED, για την οπτικοποίηση της απόφασης.

3.2.8 Διεπαφή Ελέγχου

Η έξοδος αυτής της απόφασης υλοποιείται μέσω μιας απλής συνάρτησης, της wheelchair_send, η οποία βρίσκεται στο τέλος του inspire_dataprocess. Όταν η εντολή εισόδου είναι 1, η συνάρτηση αυτή τροφοδοτεί με ρεύμα έναν ακροδέκτη (pin) του Raspberry Pi, ο οποίος στην αρχή του προγράμματος έχει οριστεί ως ο ακροδέκτης 17.

Για την πρακτική εξέταση αυτού του συστήματος, στο παράθυρο εντολών (command window) εκτυπώνονται μηνύματα με την ισχύ των κυμάτων Βήτα και Θήτα, καθώς και τα μηνύματα "power on" ή "power off", όταν εξάγεται αποτέλεσμα εξόδου 1 ή 0 αντίστοιχα. Αυτά δεν είναι πλέον απολύτως απαραίτητα, καθώς η οπτικοποίηση γίνεται πρακτικά μέσω του LED και με έναν πίνακα απεικόνισης πραγματικού χρόνου, που θα αναλυθεί στη συνέχεια.

3.2.9 Έλεγχος κατάστασης συστήματος

Στο inspire_utils υπάρχει μια απλή συνάρτηση (print_performance), η οποία εκτελείται στο τέλος κάθε βρόχου (loop) του dataprocess. Αυτή ελέγχει και εκτυπώνει στο παράθυρο εντολών (command window) τον χρόνο που απαιτήθηκε για να ολοκληρωθεί η διαδικασία (από το φιλτράρισμα μέχρι την εξαγωγή της απόφασης), τη χρήση της CPU και τη θερμοκρασία του συστήματος.

Σε αυτό το στάδιο, προφανώς, ο χρόνος για την εξαγωγή της απόφασης και ο φόρτος εργασίας της CPU είναι μικροί. Ωστόσο, ο κίνδυνος για την υψηλή αύξηση της θερμοκρασίας του συστήματος, όπως παρατηρήθηκε στις δοκιμές, ήταν πραγματικός.

Σκοπός αυτής της συνάρτησης είναι ο μελλοντικός έλεγχος του συστήματος για προβλήματα που μπορεί να προκύψουν σε πιο πολυσύνθετες επεξεργασίες και διαδικασίες λήψης αποφάσεων.

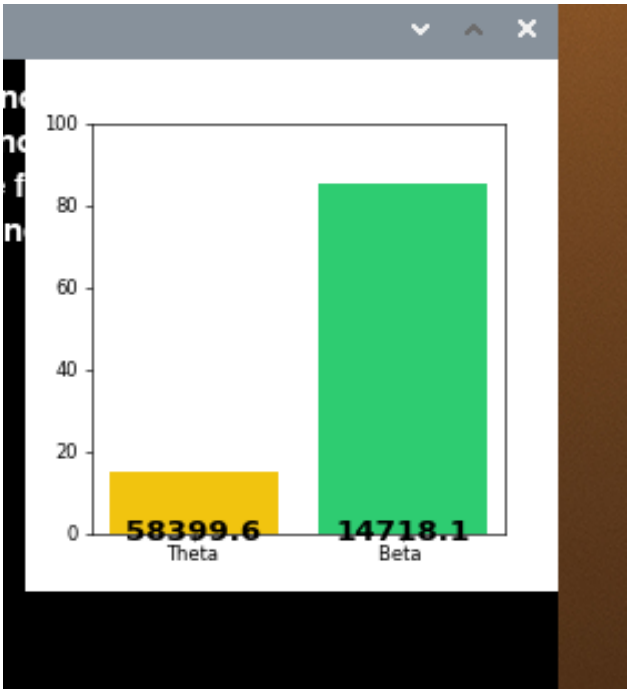


***Εικόνα 3.3**: Στιγμιότυπο οθόνης απο το παράθυρο εντολών κατά την καταγραφή δεδομένων, του μηνήματος του inspire_utils*

3.2.10 Απεικονίσεις

Αρχικά, το παλιό γράφημα (figure) απεικόνισης των αισθητήρων αφαιρέθηκε (ο κώδικας μετατράπηκε σε μορφή σχολίων), καθώς στο πλαίσιο αυτής της εργασίας δεν χρησιμοποιήθηκαν καθόλου αυτοί οι αισθητήρες, οι θέσεις των οποίων απεικονίζονταν. Η μορφή της απεικόνισης άλλαξε ριζικά, εστιάζοντας στην απεικόνιση του εύρους της ισχύος των ζωνών. Στην παρούσα εργασία υπάρχουν μόνο δύο στήλες, οπότε η υλοποίηση είναι σχετικά απλή, αλλά υπάρχει η δυνατότητα επέκτασης με την προσθήκη περαιτέρω χαρακτηριστικών που θα εξάγονται στο μέλλον. Ο κώδικας της απεικόνισης βρίσκεται στο inspire_interface και στο inspire_dataprocess.

Πρόκειται για δύο στήλες με κλειδωμένο άξονα Y. Το ύψος της μίας απεικονίζεται συγκριτικά με το ύψος της άλλης, καθώς καμία από τις δύο δεν έχει κάποια απόλυτη σταθερή τιμή και η απόφαση λαμβάνεται με βάση τη μεταξύ τους σύγκριση. Επίσης, μέσα στην απεικόνιση προβάλλεται και η ακριβής αριθμητική τιμή, η οποία αντιστοιχεί στο εκάστοτε μέγεθος. Απεικονίζονται στην εικόνα 3.4 στο κεφάλαιο των αποτελεσμάτων. Αναγνωρίζεται, βέβαια, ότι σε περίπτωση προσθήκης περαιτέρω χαρακτηριστικών, ενδεχομένως να μην έχει νόημα η συγκριτική απεικόνιση του ύψους τους, όπως συμβαίνει στη συγκεκριμένη περίπτωση.



***Εικόνα 3.4**: στιγμιότυπο οθόνης της απεικονίσεις των χαρακτηριστικών στην διεπαφή (interface) χρήστη*

3.3 Προγράμματα πειραματισμού-ελέγχου

Εκτός του κύριου προγράμματος, δημιουργήθηκαν και 9 ανεξάρτητα προγράμματα για την απεικόνιση του σήματος και την επαλήθευση της ορθής λειτουργίας των φίλτρων και του EEG. Τα ονόματά τους ξεκινούν με inspiret_... αντί για inspire_..., καθώς δεν αποτελούν μέρος του κύριου προγράμματος και δεν καλούνται μέσα σε αυτό. Το «t» στην ονομασία υποδηλώνει τη λέξη δοκιμή (testing). Τα αρχεία αυτά βρίσκονται σε έναν υποφάκελο μέσα στον κεντρικό φάκελο όπου βρίσκονται και τα υπόλοιπα στοιχεία του INSPiRE.

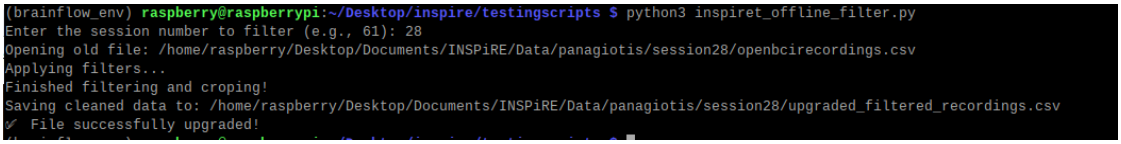
Κατά την εκκίνησή τους, τα προγράμματα αυτά ζητούν από τον χρήστη να επιλέξει τον αριθμό της συνεδρίας (session). Όλα χρησιμοποιούν ως μοναδική διαδρομή (path) την /home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis. Όπως αναφέρθηκε, ο υποφάκελος panagiotis προκύπτει από τον ορισμό στο inspire_main και πρέπει να προσαρμόζεται όταν αλλάζει και η αντίστοιχη μεταβλητή.

Στη συνέχεια παρουσιάζονται τα προγράμματα τα οποία απεικονίζουν με διαγράμματα τα δεδομένα που παρείχε το σύστημα EEG, καθώς και το αποτέλεσμα των φίλτρων που χρησιμοποιούνται σε αυτά. Τα τελικά τους αποτελέσματα θα παρουσιαστούν εκτενώς στο κεφαλαίο 4 τα αποτελέσματα καθώς απεικονίζουν και την λειτουργία των φίλτρων.

- inspiret_singlefiltersvisual.py: Απεικονίζει ένα κανάλι (το οποίο επιλέγεται μέσα στο πρόγραμμα) πριν και μετά την εφαρμογή του φίλτρου συχνοτήτων (μη πραγματικού χρόνου / offline). Για καλύτερη οπτική παρουσίαση δεν απεικονίζεται όλο το σήμα, αλλά μόνο τα 500 πρώτα στοιχεία (δείγματα). Έχει επιλεγεί αυθαίρετα να εμφανίζεται το πρώτο κανάλι του EEG, αλλά αυτό μπορεί να αλλάξει εύκολα.
- inspiret_multyvisual.py: Απεικονίζει όλα τα κανάλια συγχρόνως, πριν και μετά την εφαρμογή του φίλτρου συχνοτήτων. Για καλύτερη οπτική παρουσίαση

απεικονίζονται μόνο τα 200 πρώτα στοιχεία. Αυτή η απεικόνιση μπορεί επίσης να χρησιμοποιηθεί για την οπτική εξέταση της λειτουργίας όλων των καναλιών.

- inspiret_frequency.py: Απεικονίζει τα σήματα όλων των καναλιών στο πεδίο των συχνοτήτων, εξάγοντας τον μέσο όρο από το κάθε κανάλι. Τα πρώτα και τα τελευταία 2 δευτερόλεπτα της καταγραφής δεν χρησιμοποιούνται, καθώς εισάγουν θόρυβο στην καταγραφή.
- inspiret_testrealtime.py: Εφαρμόζει το φίλτρο πραγματικού χρόνου όπως ακριβώς θα εφαρμοζόταν στο πραγματικό σήμα και απεικονίζει μόνο ένα κανάλι, πριν και μετά το φίλτρο συχνοτήτων. Όπως και προηγουμένως, για καλύτερη οπτική παρουσίαση απεικονίζονται τα 500 πρώτα στοιχεία (με προεπιλεγμένο το πρώτο κανάλι).
- inspiret_realtimefft_visualizer.py: Εφαρμόζει το φίλτρο πραγματικού χρόνου όπως ακριβώς θα εφαρμοζόταν στο πραγματικό σήμα και απεικονίζει μόνο ένα κανάλι, πριν και μετά το φίλτρο συχνοτήτων, αλλά σε αυτήν την περίπτωση η απεικόνιση γίνεται στο πεδίο των συχνοτήτων. Για να εξαλειφθεί η αρχική παροδική βαθμιδωτή απόκριση των φίλτρων Butterworth IIR και να παρουσιαστεί το φάσμα συχνοτήτων σταθερής κατάστασης, αφαιρέθηκαν από την απεικόνιση τα πρώτα 3 δευτερόλεπτα της καταγραφής. Έχει επιλεγεί αυθαίρετα να εμφανίζεται το φάσμα συχνοτήτων του πρώτου καναλιού του EEG, αλλά αυτό μπορεί να αλλάξει εύκολα. Εξετάζοντας το φάσμα συχνοτήτων, μπορούμε να αντλήθουν διαφορετικές πληροφορίες και να προσαρμόστουν το φίλτρο ξεχωριστά για κάθε κανάλι. Αυτό θα σχολιαστεί μετέπειτα στο κεφάλαιο 5.3.
- inspiret_realtimefft_visualizermulty.py: Είναι παρόμοιο με το inspiret_realtimefft_visualizer.py, με τη διαφορά ότι, αντί να απεικονίζει μόνο ένα κανάλι, υπολογίζεται ο μέσος όρος και απεικονίζονται όλα τα κανάλια στο πεδίο των συχνοτήτων.
- inspiret_offlinefilter.py: Για τον εύκολο πειραματισμό, δημιουργήθηκε ένα πρόγραμμα που επεξεργάζεται τα δεδομένα και δημιουργεί ένα νέο αρχείο .csv (Excel), χρησιμοποιώντας το φίλτρο μη πραγματικού χρόνου. Τα νέα, φιλτραρισμένα δεδομένα αποθηκεύονται στον ίδιο φάκελο με τα υπόλοιπα δεδομένα της συνεδρίας, με την ονομασία upgraded_filtered_recordings.csv. Η συνάρτηση του φίλτρου δεν καλείται από το κύριο πρόγραμμα, αλλά από ένα αντίγραφό της που βρίσκεται μέσα στο αρχείο, παρέχοντας τη δυνατότητα για εύκολες αλλαγές. Παράλληλα, τα υπόλοιπα προγράμματα απεικόνισης έχουν την επιλογή μέσα στον κώδικα να προβάλουν αυτά τα δεδομένα.



Εικόνα 3.5*: στιγμιότυπο οθόνης με την λειτουργία του nspiret_offlinefilter.py στο παράθυρο εντολών.*

- inspiret_present.py: Απεικονίζει και τα 16 κανάλια, παρόμοια με το inspiret_multyvisual.py, με τη διαφορά ότι δεν απεικονίζεται το μη φιλτραρισμένο σήμα και ο χρήστης, μέσω ενός widget, μπορεί να κάνει κύλιση (scroll) στον άξονα X (του χρόνου). Ο λόγος που δεν απεικονίζεται παράλληλα και το ακατέργαστο σήμα είναι ότι άνοιγε υπερβολικά τον άξονα Y, με αποτέλεσμα να μην καθίσταται δυνατή η καλή οπτικοποίηση των

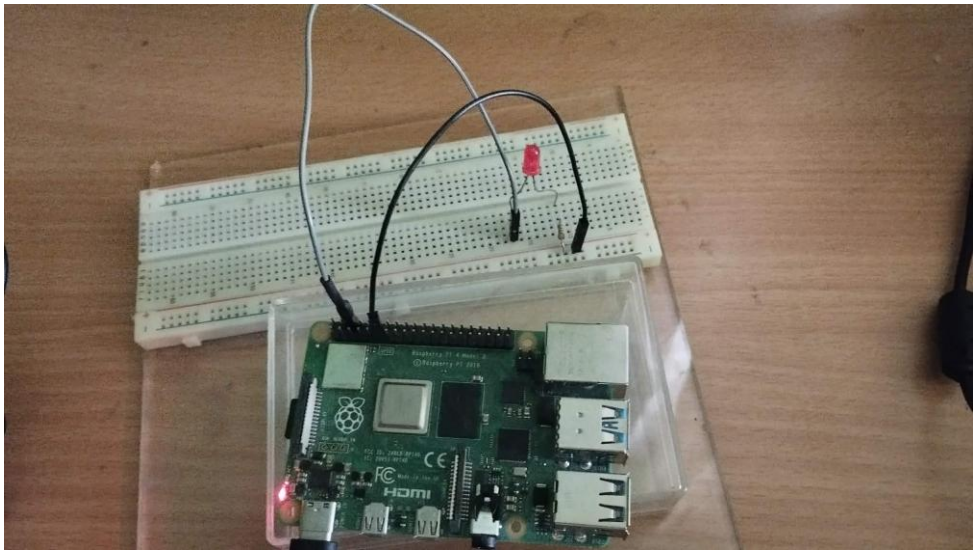
διακυμάνσεων του σήματος. Το πρόγραμμα απεικονίζει τρεις τιμές για το εύρος του σήματος σε κάθε κανάλι, καθώς η κλίμακα σε κάθε κανάλι έχει οριστεί διαφορετικά βάσει της μέγιστης και της ελάχιστης τιμής του. Οι τρεις τιμές που προβάλλονται είναι η μέγιστη, η ελάχιστη και η ενδιάμεσή τους.

- inspiret_presentrealtime.py: Λειτουργεί ακριβώς με τον ίδιο τρόπο όπως το inspiret_present.py, με τη διαφορά ότι παρουσιάζει το σήμα που έχει φιλτραριστεί από το φίλτρο πραγματικού χρόνου. Παράλληλα, τα πρώτα 3 δευτερόλεπτα της καταγραφής —η προαναφερθείσα φάση «προθέρμανσης» (warm-up)— δεν απεικονίζονται, καθώς το πλάτος τους είναι τόσο μεγάλο που καθιστά τις πραγματικές μεταβολές αόρατες με γυμνό μάτι.

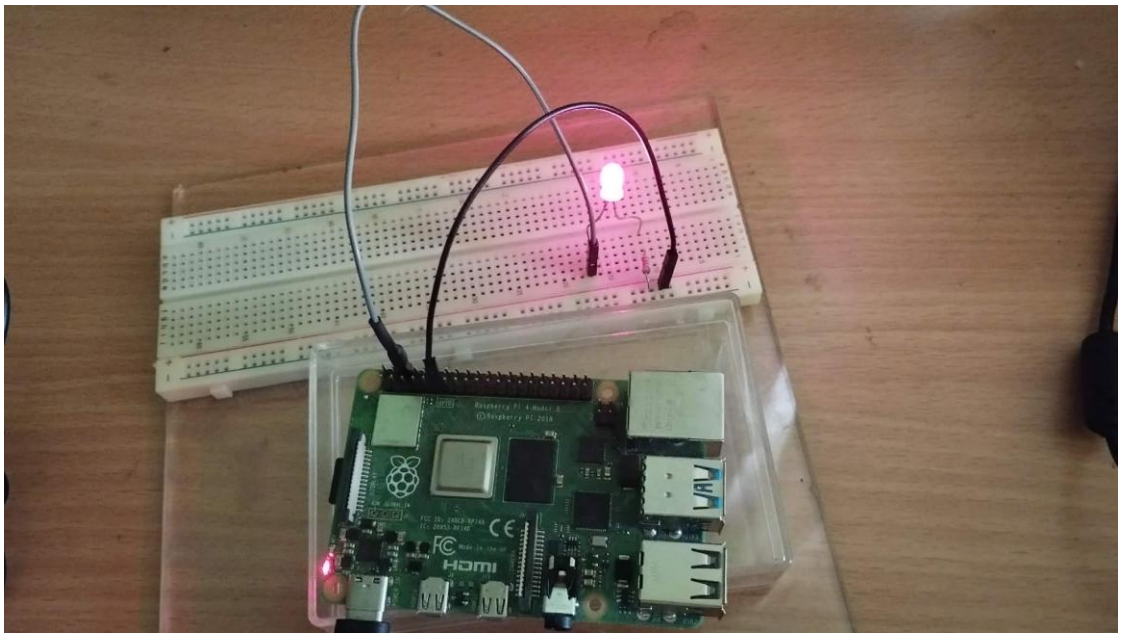
3.4 Κύκλωμα led

Δημιουργήθηκε ένα πολύ απλό κύκλωμα για την απεικόνιση της εξόδου που αναφέρθηκε προηγουμένως. Το κύκλωμα αυτό αποτελείται από μία αντίσταση 120 kΩ, ένα κόκκινο LED και δύο καλώδια σύνδεσης (jump wires) για τη σύνδεση με τους ακροδέκτες (pins) του Raspberry Pi.

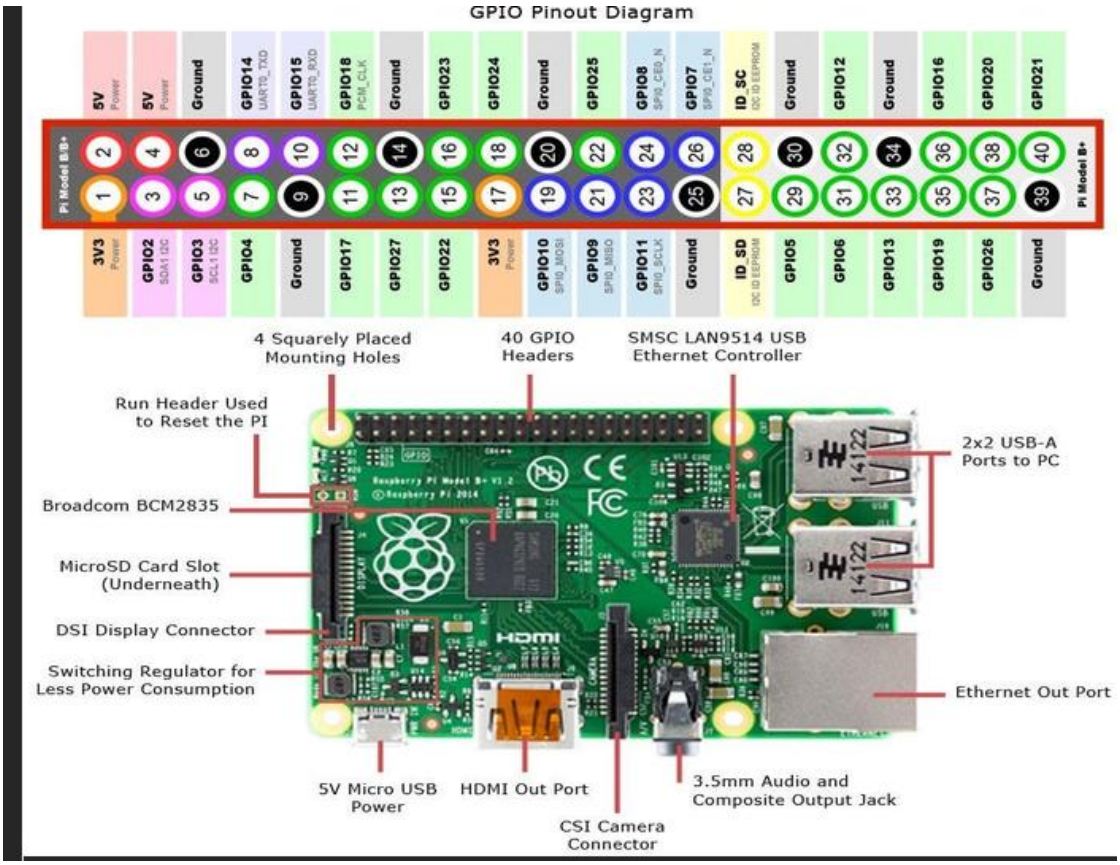
Ο ακροδέκτης 6 χρησιμοποιήθηκε ως γείωση (GND) και ο ακροδέκτης 11 (GPIO17) χρησιμοποιήθηκε ως τροφοδοσία για το κύκλωμα, παρέχοντας τα 3.3V που προσφέρει το Raspberry Pi.



Εικόνα 3.6: *Το κύκλωμα led ελεγχόμενο απο το Raspberry Pi κλειστό.*



Εικόνα 3.7*: Το κύκλωμα led ελεγχόμενο απο το Raspberry Pi ανοιχτό.*



***Εικόνα 3.8**: Διάταξη ακροδεκτών (GPIO) του Raspberry Pi και αντιστοίχιση λειτουργιών (https://www.hackster.io/julio-gonzalez/raspberry-pi-3-not-your-average-pie-62935a)*

3.5 Προβλήματα και Προκλήσεις που Αντιμετωπίστηκαν

Σε αυτήν την υποενότητα θα σχολιαστούν κάποια πρακτικά προβλήματα υλοποίησης, για την καλύτερη αντιμετώπιση των ίδιων ή παρόμοιων προβλημάτων που μπορεί να ανακύψουν μελλοντικά.

Προβλήματα δικτύου και επικοινωνίας του Raspberry Pi Υπάρχουν δύο προβλήματα που αφορούν τη σύνδεση του υπολογιστή με το Raspberry Pi, η οποία υλοποιήθηκε στην αρχή της εργασίας. Αρχικά, παρά τη φαινομενικά σωστή φόρτωση των στοιχείων του δικτύου στην κάρτα SD του Raspberry Pi μέσω του Raspberry Pi Imager, το σύστημα δεν συνδεόταν στο δίκτυο. Η επίλυση αυτού του προβλήματος έγινε χρησιμοποιώντας ένα καλώδιο USB-Ethernet από τον υπολογιστή στο Raspberry Pi. Το δεύτερο πρόβλημα αφορά τη σύνδεση μέσω του καλωδίου Ethernet, όπου μετά από μερικά λεπτά η διασύνδεση διακοπτόταν. Για την αντιμετώπιση αυτού του προβλήματος έγιναν κάποιες ενέργειες, οι οποίες όμως δεν ήταν επιτυχημένες. Η πιο εύκολη και πρακτική λύση αποδείχθηκε η αποσύνδεση και η επανασύνδεση του καλωδίου με τον υπολογιστή, όποτε συνέβαινε αυτό.

Προβλήματα λογισμικού και συμβατότητας (Arduino IDE) Όσον αφορά τη φόρτωση του λογισμικού στην πλακέτα του EEG, κατά τη λήψη των απαραίτητων πακέτων στη νεότερη έκδοση του Arduino IDE (2.3.8) αντιμετωπίστηκαν κάποια προβλήματα. Εικάζεται ότι αυτά οφείλονταν σε ζητήματα συμβατότητας των συγκεκριμένων

πακέτων με τις νέες εκδόσεις. Τα προβλήματα αυτά εξαλείφθηκαν όταν χρησιμοποιήθηκε η παλαιότερη έκδοση (legacy version) του προγράμματος (1.8.19).

Τροφοδοσία EEG Ακόμη ένα πρακτικό πρόβλημα που παρουσιάστηκε στην υλοποίηση και προκάλεσε σύγχυση είναι το ακόλουθο: Παρότι η επικοινωνία μεταξύ Raspberry Pi, dongle και του EEG φαινόταν να είναι σταθερή, παρουσιάστηκε πρόβλημα στην τροφοδοσία του EEG. Οι τέσσερις (4) μπαταρίες καταναλώνονται σχετικά γρήγορα από τη συσκευή, και όταν η τάση τους πέσει —ενώ το EEG φαίνεται να λειτουργεί— χάνεται η επικοινωνία με το dongle.

Δυσλειτουργία καναλιών και φίλτρα Κατά τις δοκιμές για τη λειτουργία των φίλτρων, παρατηρήθηκε ότι αρκετά κανάλια δεν λειτουργούσαν σωστά, γεγονός που ίσως προκάλεσε διαφορετικά παρασιτικά σήματα. Καθώς τα φίλτρα που δημιουργήθηκαν βασίστηκαν στο φάσμα των συχνοτήτων, επισημαίνεται ότι τα φίλτρα αυτά δεν αποτελούν παρά μια βάση η οποία πρόκειται να τροποποιηθεί μελλοντικά.

Υπερθέρμανση (Thermal Throttling) Επίσης, κατά τη διάρκεια των δοκιμών και έπειτα από μεγάλες διάρκειες λειτουργίας, το Raspberry Pi υπερθερμαινόταν. Αυτό προκαλούσε θερμικό στραγγαλισμό (thermal throttling) των επιδόσεων. Για να αντιμετωπιστεί το φαινόμενο, έγινε χρήση ενός ψυχρού αντικειμένου ως βάση κάτω από το Raspberry Pi, λειτουργώντας ως αυτοσχέδια ψύκτρα. Λόγω της παρατήρησης αυτού του προβλήματος, προστέθηκε στο inspire_utils η δυνατότητα εμφάνισης και παρακολούθησης της θερμοκρασίας.

Τέλος, τα μόνα άλλα προβλήματα αφορούσαν τη συμβατότητα του προγράμματος INSPiRE με τις νέες εκδόσεις των λογισμικών, τα οποία προαναφέρθηκαν και επιλύθηκαν επιτυχώς.

3.6 Ανάλυση κίνδυνων και περιορισμών

Στο υποκεφάλαιο αυτό παρουσιάζονται παράγοντες που ενδέχεται να επηρεάσουν την ορθή λειτουργία της δομής που δημιουργήθηκε στο πλαίσιο αυτής της εργασίας. Καθώς η εστίαση της παρούσας εργασίας ήταν αρκετά περιορισμένη, οι άμεσοι περιορισμοί και οι κίνδυνοι των στοιχείων που αναπτύχθηκαν ήταν ελάχιστοι. Οι περισσότεροι περιορισμοί και κίνδυνοι προέρχονται από τα περιφερειακά συστήματα που χρησιμοποιήθηκαν γύρω από την εργασία.

Αρχικά, σε ορισμένες περιπτώσεις, η επικοινωνία του Raspberry Pi με τον υπολογιστή διακόπτεται τυχαία. Αν αυτό συμβεί σε περίπτωση εξέτασης ή συλλογής δεδομένων, δεν επιτρέπει στον χρήστη να τερματίσει την καταγραφή, καθιστώντας πιθανώς αναγκαία την επανάληψη της διαδικασίας.

Μελλοντικά, το σύστημα ενδέχεται να αντιμετωπίσει προβλήματα απόδοσης λόγω των περιορισμών του Raspberry Pi 4. Συγκεκριμένα, ο επεξεργαστής του (Cortex-A72 στα 1.5GHz) είναι επιρρεπής σε υπερθέρμανση (thermal throttling) όταν εκτελεί συνεχώς βαριά μαθηματικά φίλτρα ψηφιακής επεξεργασίας σήματος (DSP) για 16 κανάλια σε πραγματικό χρόνο, κάτι που μπορεί να προκαλέσει επικίνδυνη καθυστέρηση (lag) στην οδήγηση του αμαξιδίου. Παράλληλα, η μνήμη RAM μπορεί να υπερφορτωθεί αν δεν αδειάζουν σωστά τα δεδομένα (memory leaks), ενώ η κάρτα microSD των 16GB είναι όχι μόνο πολύ μικρή για τον τεράστιο όγκο των αρχείων .csv, αλλά διαθέτει και πολύ αργές ταχύτητες εγγραφής (I/O bottleneck), καθώς και υψηλό κίνδυνο μόνιμης βλάβης (corruption) από τις συνεχείς εγγραφές του καταγραφικού. Κατά τη διάρκεια των

δοκιμών, η μνήμη της κάρτας γέμισε, υποδεικνύοντας έναν πολύ πιθανό κίνδυνο και την άμεση ανάγκη για μεταφορά των δεδομένων. Όπως προαναφέρθηκε, το πρόγραμμα INSPiRE δημιουργήθηκε με τη δυνατότητα παράλληλης επεξεργασίας, αλλά στο πλαίσιο αυτής της εργασίας δεν αξιοποιήθηκε.

Επιπλέον, όσον αφορά τις δοκιμές των καταγραφών, θα πρέπει να αναφερθεί ότι δεν λειτουργούσαν όλα τα κανάλια, γεγονός που μπορεί να επηρεάσει τα αποτελέσματα των φίλτρων που θα παρουσιαστούν παρακάτω. Αυτό αφορά τον EEGκαι την απουσία εξειδικευμένου προσωπικού για την ορθή τοποθέτησή του σε κάποιο υποκείμενο (χρήστη).

Περαιτέρω, όπως προαναφέρθηκε, τα φίλτρα που εφαρμόζονται στις καταγραφές σε πραγματικό χρόνο και έπειτα για την αποθήκευση των δεδομένων είναι διαφορετικά, αν και παρόμοια. Όταν μελλοντικά εκπαιδευτεί ένα μοντέλο μηχανικής μάθησης με αυτά τα δεδομένα, τα εξαγόμενα αποτελέσματα μπορεί να είναι χειρότερα από αυτά που θα παρουσιαστούν στην εκπαίδευση. Επιπρόσθετα, υπάρχει μια μικρή καθυστέρηση στον χρονισμό του LED και του γραφικού περιβάλλοντος λόγω καθυστέρησης του συστήματος στην απεικόνιση, αν και αυτό το πρόβλημα δεν εμφανίζεται πάντα. Ακόμη, πρέπει να αναφερθεί ένα μικρό πρακτικό ζήτημα: αν το LED είναι ενεργοποιημένο στο τέλος της καταγραφής, παραμένει ενεργοποιημένο μέχρι τη λήξη του προγράμματος.

Επιβάλλεται να αναφερθεί η πολύ μεγάλη πρακτική δυσκολία λήψης αποφάσεων μέσω ενός EEG σε πραγματικές συνθήκες, καθώς πολλοί παράγοντες μπορεί να παρεμβληθούν στη σωστή καταγραφή των δεδομένων από τους αισθητήρες. Η διαδικασία λήψης 4 αποφάσεων (5 μαζί με την κατάσταση ηρεμίας), καθώς προορίζεται για αναπηρικό αμαξίδιο, είναι εξαιρετικά πιο δύσκολη από αυτή των 2 αποφάσεων, όσον αφορά τη φανταστική κίνηση (motor imagery). Αυτό συμβαίνει επειδή στην περίπτωση των 2 αποφάσεων μπορεί να εφαρμοστεί εύκολα διαχωρισμός της δραστηριότητας του ενός ημισφαιρίου από το άλλο, με φανταστικές κινήσεις χεριών ή ποδιών σε διαφορετικές μεριές. Επίσης, η συσκευή της OpenBCI είναι εκπληκτική αναλογικά με το κόστος της, αλλά παραμένει καταναλωτικός εξοπλισμός και όχι ιατρικός, γεγονός που σημαίνει ότι ο λόγος σήματος προς θόρυβο (SNR - Signal to Noise Ratio) είναι σχετικά χαμηλός.

Παράλληλα, όσον αφορά τον χρόνο διάρκειας κάθε απόφασης, ο βρόχος (loop) μέσα στο πρόγραμμα παραμένει ίδιος για 0,5 δευτερόλεπτα, συν τον χρόνο που απαιτείται για να γίνει η επεξεργασία για την επόμενη λήψη απόφασης. Ο χρόνος αυτός προς το παρόν θεωρείται πολύ μικρός και μελλοντικά η επεξεργασία θα μπορούσε να γίνεται ενώ το σύστημα βρίσκεται μέσα σε αυτή την παύση των 0,5 δευτερολέπτων (ή οποιασδήποτε άλλης χρονικής καθυστέρησης).

Οι περιορισμοί αυτοί αφορούν κυρίως μελλοντικά βήματα, αλλά η πρόβλεψή τους πρέπει να γίνει εκ των προτέρων, ώστε να αντιμετωπιστούν με αποτελεσματικό και σωστότερο τρόπο κατά τη δημιουργία της υποδομής του έργου. Οι περιορισμοί αυτοί λαμβάνονται υπόψη στη συζήτηση των αποτελεσμάτων και στις προτάσεις για μελλοντική βελτίωση του συστήματος.

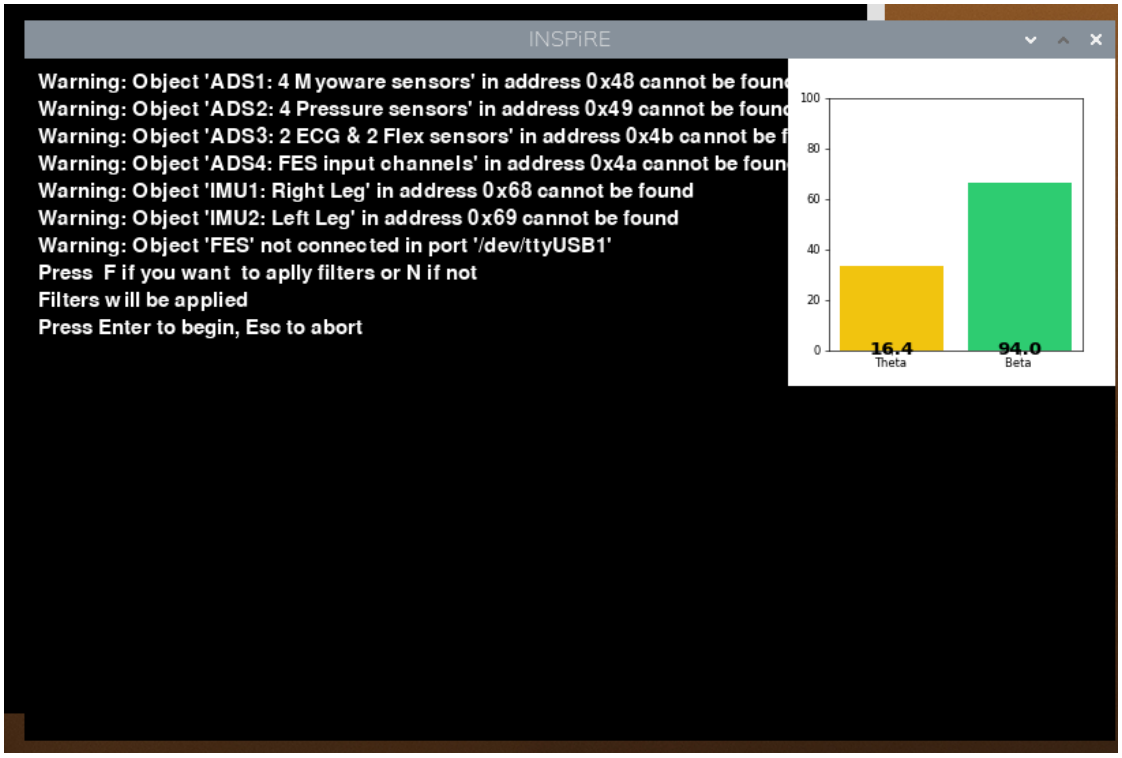
4. Αποτελέσματα

Στο προηγούμενο κεφάλαιο παρουσιάστηκε η μεθοδολογία υλοποίησης και η θεωρητική θεμελίωση του προτεινόμενου συστήματος. Στο παρόν κεφάλαιο παρουσιάζονται τα πειραματικά αποτελέσματα που προέκυψαν από την κατασκευή και τον έλεγχο των επιμέρους υποσυστημάτων, καθώς και από τη συνολική λειτουργία της διάταξης. Τα αποτελέσματα παρατίθενται χωρίς εκτενή σχολιασμό, με στόχο την αντικειμενική αποτύπωση της λειτουργίας της διάταξης. Η παρουσίαση των αποτελεσμάτων θα γίνει με τη σειρά που παρουσιάστηκαν στο κεφάλαιο της υλοποίησης του συστήματος.

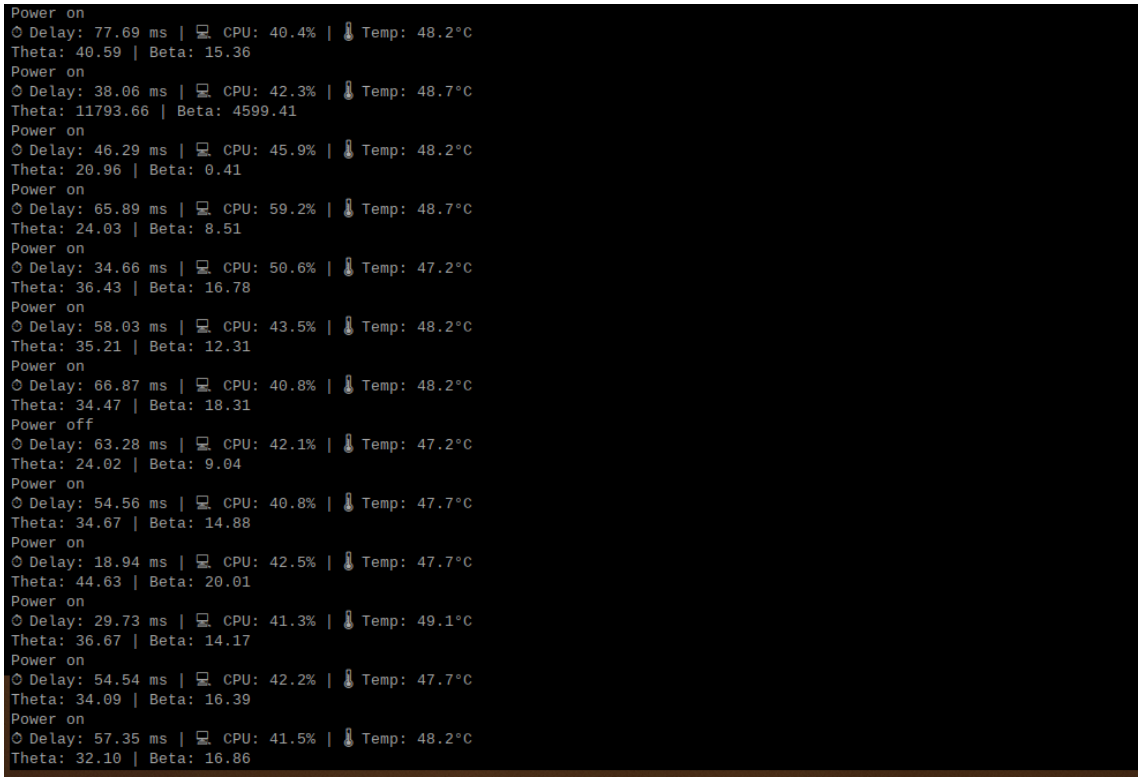
Αρχικά, θα περιγραφεί η ευκολία με την οποία μπορεί να λειτουργήσει το σύστημα ως σύνολο. Το USB dongle συνδέεται στη μαύρη θύρα (USB 2.0) του Raspberry Pi και το σύστημα του EEG ενεργοποιείται στη λειτουργία επικοινωνίας μέσω dongle (πάνω θέση του διακόπτη).
Εφόσον δεν υπάρχει κάποιο πρόβλημα στη σύνδεση με το τοπικό δίκτυο, η σύνδεση μπορεί να επιτευχθεί απευθείας χρησιμοποιώντας το RealVNC Viewer. Σε περίπτωση, όμως, που υπάρξει κάποιο πρόβλημα ή είναι επιθυμητό να συνδεθεί το Raspberry Pi σε κάποιο καινούριο δίκτυο, μπορεί να χρησιμοποιηθεί το καλώδιο USB-to-Ethernet.
Εφόσον ανοίξει το RealVNC Viewer και προβληθεί κανονικά το γραφικό περιβάλλον του Raspberry Pi, ανοίγει το αρχείο κειμένου commands και αντιγράφεται τον κώδικα. Έπειτα, ανοίγει το παράθυρο εντολών (command window), εκτελείτε επικόλληση (Ctrl+Shift+V) και πατάμε Enter.
Ελέγχαιτε ότι υπάρχει επικοινωνία με το dongle και τον EEGμέσω των μηνυμάτων που προβάλλονται. Έπειτα, προχωράμε πατώντας 'F' ή 'N' για την επιλογή ή μη της χρήσης φίλτρου αντίστοιχα, και στη συνέχεια πατάμε Enter για την εκκίνηση του προγράμματος.
Κατά την εκκίνηση, ανοίγει η ζωντανή απεικόνιση της ισχύος των ζωνών Βήτα και Θήτα στην πάνω δεξιά γωνία (εικόνα 4.1). Οι τιμές παρουσιάζονται ως ραβδογράμματα, το ύψος των οποίων συγκρίνεται μεταξύ τους, ενώ η ακριβής αριθμητική τους τιμή αναγράφεται στο διάγραμμα.
Στο παράθυρο εντολών εκτυπώνονται τα εξής:

- Η ισχύς των ζωνών Βήτα και Θήτα.
- Η κατάσταση που έχει ως έξοδο (Power On / Power Off).
- Ο χρόνος που απαιτήθηκε για την επεξεργασία των σημάτων, την εξαγωγή χαρακτηριστικών και τη λήψη απόφασης.
- Το μήνυμα καταστάσης απο το inspire_utils.

Παράλληλα, ανάβει και το ενδεικτικό LED, εφόσον αυτές οι τιμές αντιστοιχούν σε κατάσταση Power On. Όλα τα παραπάνω έχουν οριστεί να ανανεώνονται (ως βρόχος) κάθε 0,5 δευτερόλεπτα, μια ρύθμιση που μπορεί να αλλάξει, αν θεωρηθεί ωφέλιμο στο μέλλον.



***Εικόνα 4.1**: Στιγμιότυπο οθόνης που παρουσιάζεται η διεαπαφή χρηστή (interface) του προγράμματος καθώς εκτελείται καταγραφή, κάθε 0.5 δευτερόλεπτα γίνεται ανανέωση των τιμών και του ύψους το στήλων.*



***Εικόνα 4.2**: Παραθυρο εντολών κατά την καταγραφή δεδομένων*

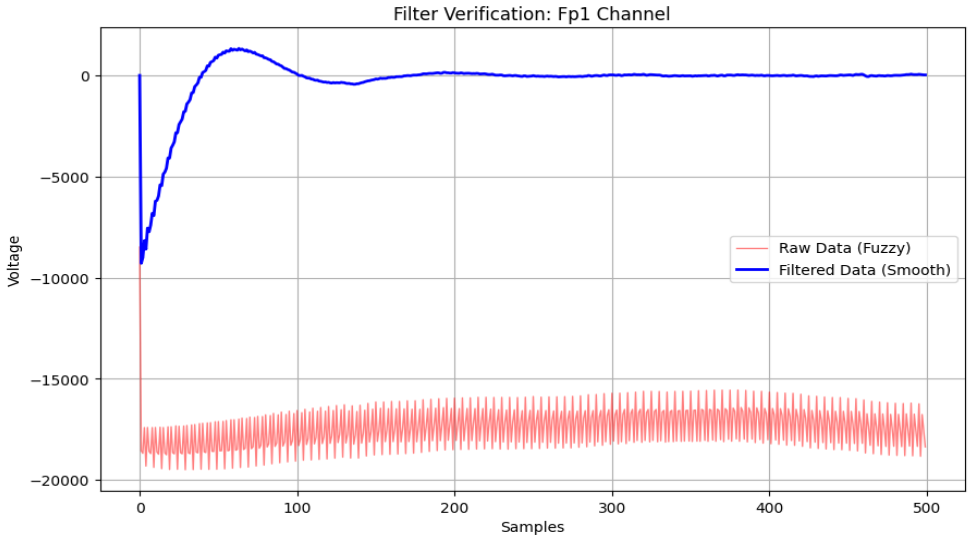
Στο παράθυρο εντολών (command window) όπως φαίνεται στην εικόνα 4.2 παρουσιάζονται 3 μηνύματα σε κάθε βρόχο (loop):

1. Οι τιμές ισχύος των ζωνών Βήτα και Θήτα από τη συνάρτηση inspire_dataprocess.data_process.
2. Αν η έξοδος του συστήματος είναι θετική (κατάσταση Power On), γεγονός που σημαίνει ότι εφαρμόζεται τάση και το ενδεικτικό LED λαμπάκι ανάβει, μέσω της συνάρτησης inspire_dataprocess.wheelchair_send.
3. Ο χρόνος που απαιτήθηκε για την επεξεργασία, την εξαγωγή χαρακτηριστικών και τη λήψη απόφασης, καθώς και η χρήση του επεξεργαστή (CPU) και η θερμοκρασία, μέσω της συνάρτησης inspire_utils.print_performance.

Για τον τερματισμό της καταγραφής πρέπει να πατηθεί το πλήκτρο Esc. Όταν γίνει αυτό, κλείνουν τα γραφικά παράθυρα και στο παράθυρο εντολών εμφανίζονται μηνύματα για τη λήξη του προγράμματος, μαζί με τη θέση (διαδρομή) όπου αποθηκεύτηκαν οι καταγραφές.

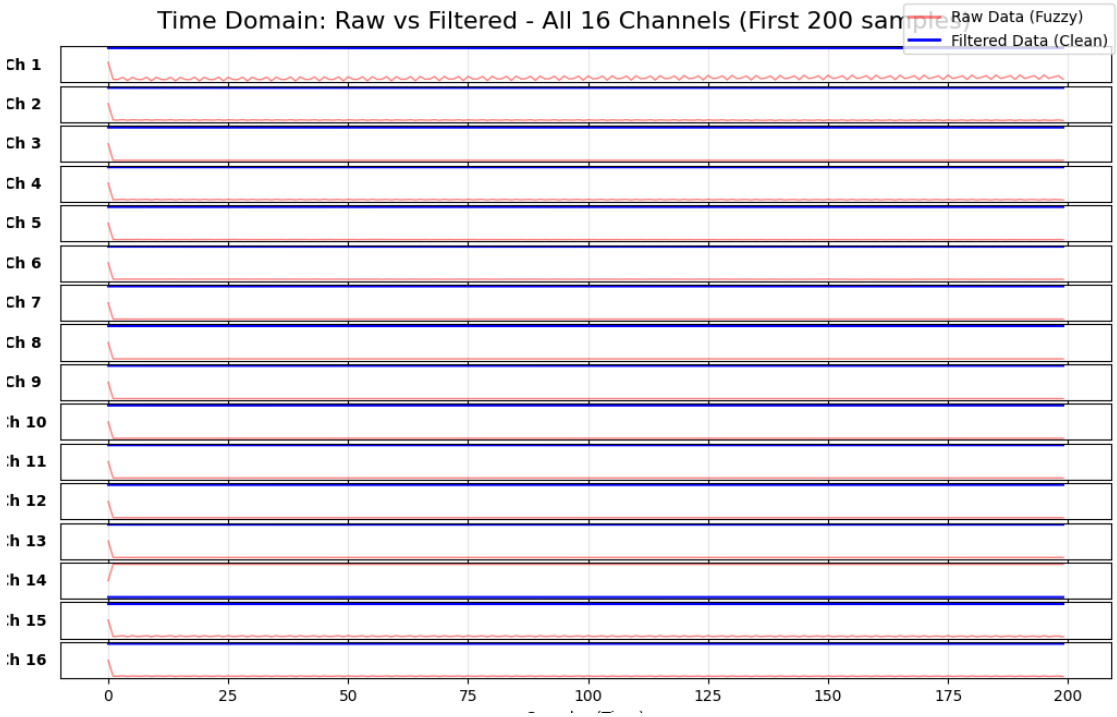
Έπειτα, οι καταγραφές μπορούν να εξεταστούν περαιτέρω χρησιμοποιώντας τα 7 προγράμματα που προαναφέρθηκαν, για την ανάλυση και απεικόνιση της καταγραφής που διεξήχθη, καθώς και της λειτουργίας των φίλτρων πάνω σε αυτήν.

Στη συνέχεια θα παρουσιαστούν οι καταγραφές και τα αποτελέσματα του φιλτραρίσματός τους, χρησιμοποιώντας τα απεικονιστικά προγράμματα (inspiret_...) που δημιουργήθηκαν σε αυτήν την εργασία. Τα επακόλουθα αποτελέσματα (4.3-13) προκύπτουν από μία συγκεκριμένη καταγραφή που πραγματοποιήθηκε. Σε αυτήν την καταγραφή, όπως θα διαφανεί και από τα διαγράμματα, δεν λειτουργούσαν τα κανάλια 3, 7, 8, 9, 10, 11, 12, 13 και 15. Αυτό είχε επιβεβαιωθεί και από το λογισμικό OpenBCI_GUI πριν ξεκινήσει η εν λόγω καταγραφή.



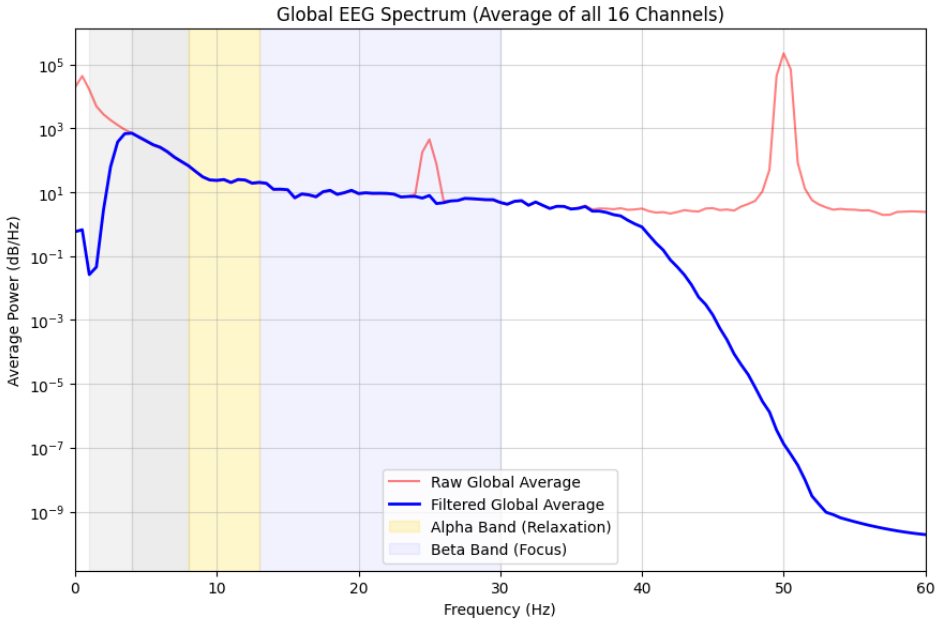
***Εικόνα 4.3**: Διάγραμμα που παρήγαγε το inspiret_singlefiltersvisual.py*

Στην εικόνα 4.3 παρατηρείται ότι η συνεχής συνιστώσα (DC) φιλτράρεται, καθώς το σήμα κεντράρεται γύρω από το 0, και ο έντονος υψίσυχνος θόρυβος αποκόπτεται. Στην αρχή του σήματος, βέβαια, παρουσιάζεται το μεταβατικό φαινόμενο (transient response) του ψηφιακού φίλτρου.



***Εικόνα 4.4**: Διάγραμμα που παρήγαγε το inspiret_multyvisual.py*

Στην εικόνα 4.4 παρατηρείται παρόμοια δράση του φίλτρου σε όλα τα κανάλια. Δυστυχώς, όπως προαναφέρθηκε, είναι δύσκολο να διακριθεί τις μεταβολές στο σήμα, καθώς το εύρος του άξονα Υ είναι πολύ μεγάλο.



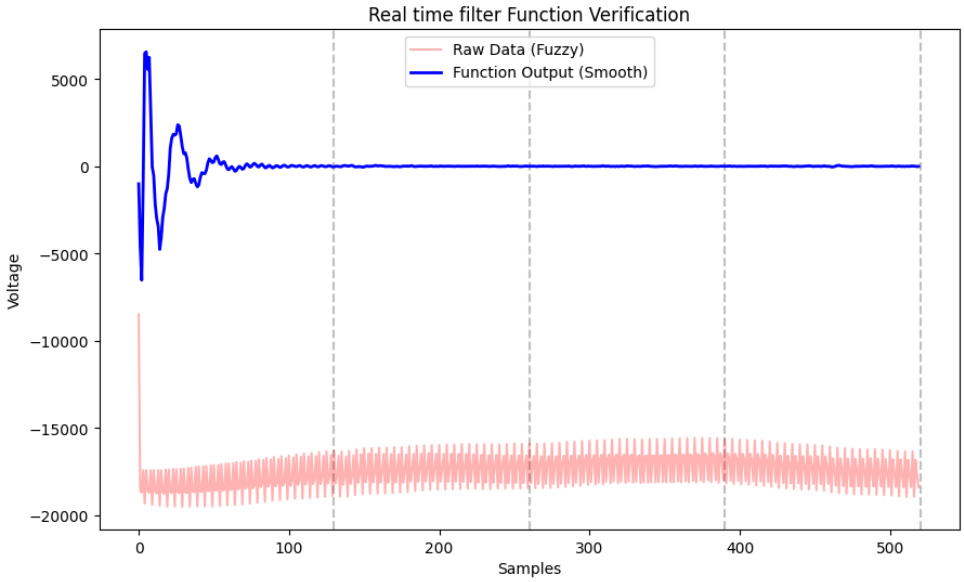
***Εικόνα 4.5**: Διάγραμμα που παρήγαγε το inspiret_frequency.py*

Όπως είναι εμφανές στην εικόνα 4.5 εύκολα εξετάζεται το φάσμα των συχνοτήτων του σήματος πριν και μετά το φιλτράρισμα. Όπως έχει αναφερθεί, στο μη φιλτραρισμένο σήμα παρατηρείτε έντονες αιχμές στα 50 Hz λόγω παρεμβολών του δικτύου ηλεκτροδότησης, στα 25 Hz μια μικρότερη αιχμή λόγω της υποαρμονικής συχνότητας στα 100hz (λόγω άλλον συσκευών και μετασχηματιστών) , καθώς και θόρυβο στις

χαμηλές συχνότητες (0-1 Hz). Αυτός ο θόρυβος χαμηλών συχνοτήτων οφείλεται σε κινήσεις, στο ανοιγοκλείσιμο των ματιών, και κυρίως στα γαλβανικά δυναμικά επαφής μεταξύ του δέρματος και του ηλεκτροδίου (συνεχής συνιστώσα / DC offset).

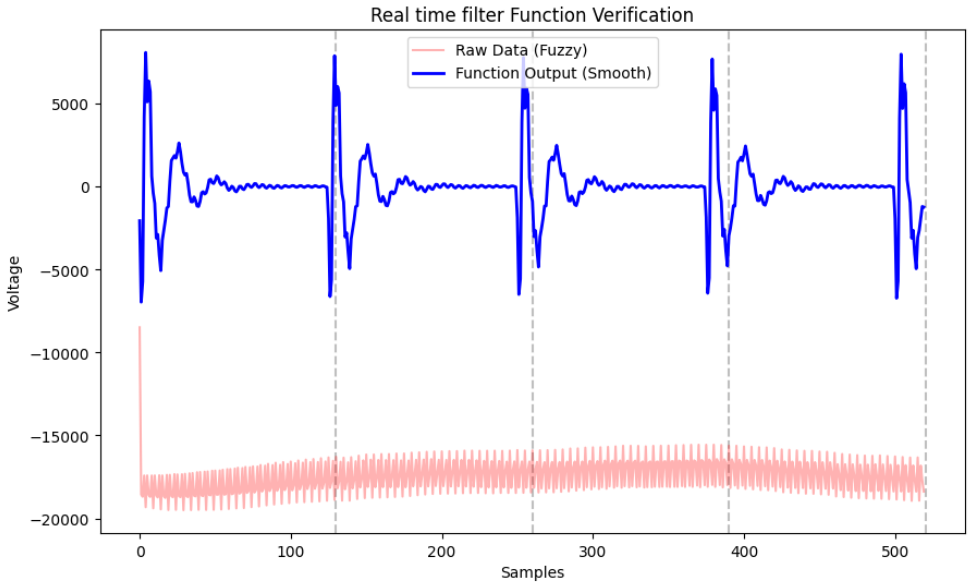
Στο φιλτραρισμένο σήμα διακρίναιτε ότι οι συχνότητες πέρα από το επιθυμητό εύρος αποκόπτονται. Το ίδιο συμβαίνει και με τις αιχμές του θορύβου, οι οποίες εξομαλύνονται προκειμένου να αποφευχθούν πιθανές ψευδενδείξεις (artifacts). Οι συχνότητες μεταξύ 0 και 1 Hz, λόγω τεχνικών ζητημάτων, δεν αποκόπηκαν πλήρως, γεγονός το οποίο θα σχολιαστεί μετέπειτα.

Στη συνέχεια, θα παρουσιαστούν τα προγράμματα για την εξέταση των φίλτρων πραγματικού χρόνου. Το μη φιλτραρισμένο σήμα δεν θα σχολιαστεί εκ νέου, αφού η ανάλυσή του έχει γίνει ήδη.



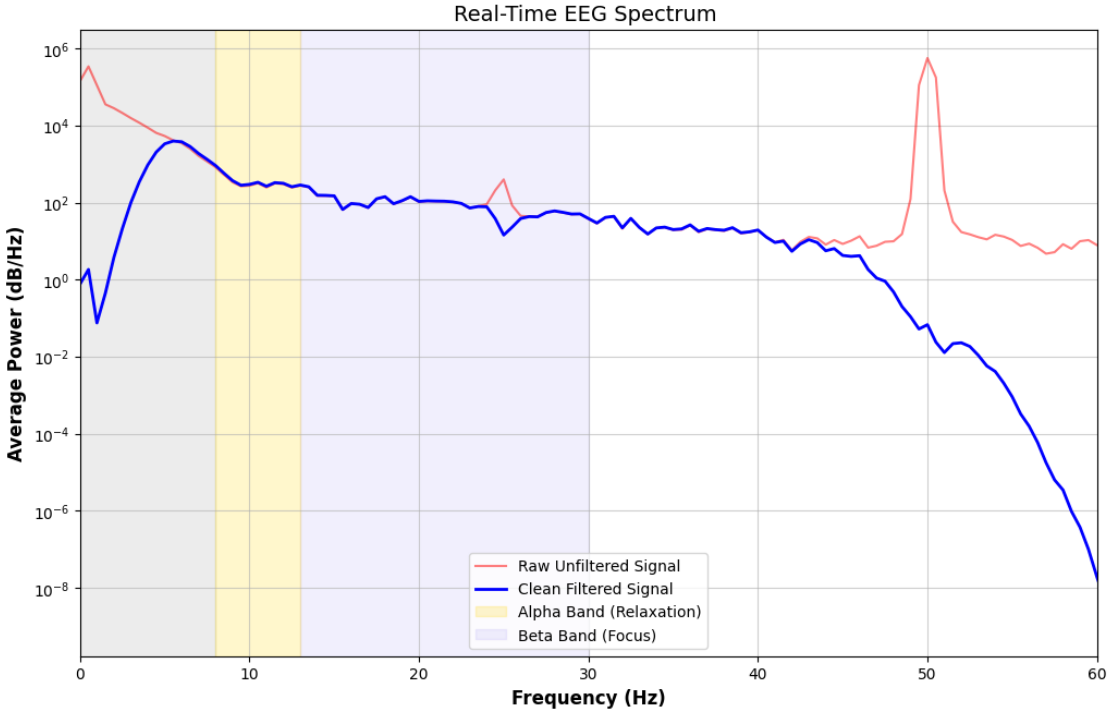
***Εικόνα 4.6**: Διάγραμμα που παρήγαγε το inspiret_testrealtime.py*

Στην εικόνα 4.6 παρατηρείτε ότι το φίλτρο αποκόπτει τη συνεχή συνιστώσα (DC), καθώς το σήμα κεντράρεται στο 0, ενώ παράλληλα εξαλείφεται και ο υψίσυχνος θόρυβος. Και πάλι, βέβαια, κάνει την εμφάνισή του το μεταβατικό φαινόμενο (transient response).



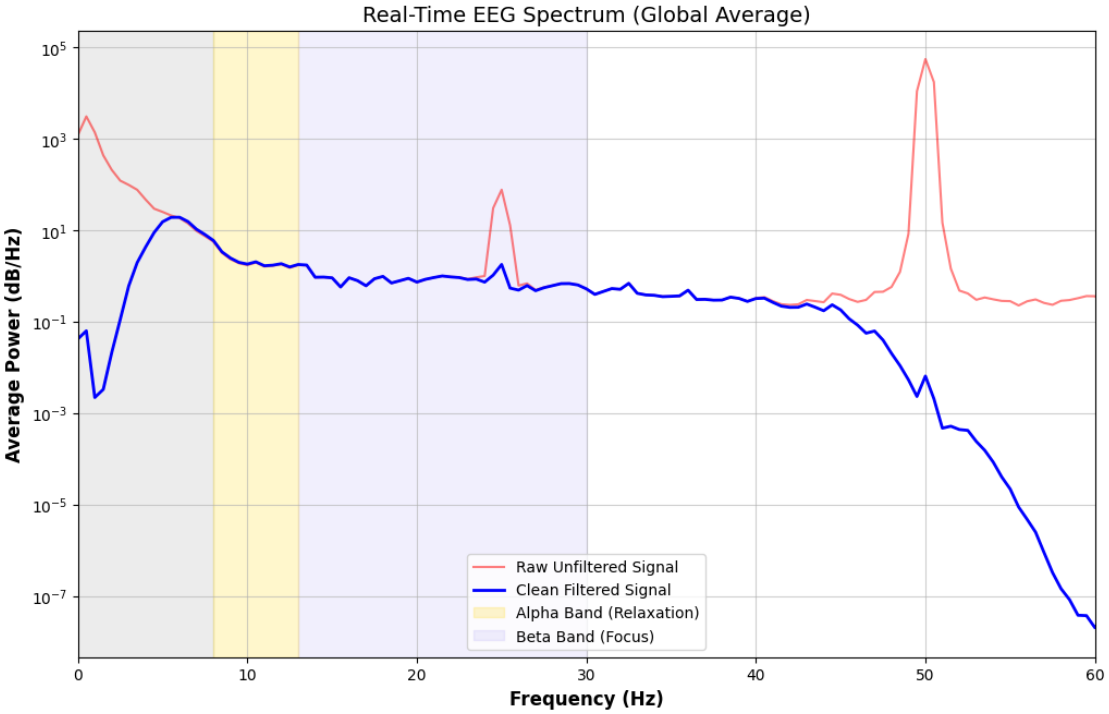
***Εικόνα 4.7**: Διάγραμμα που παρήγαγε το inspiret_testrealtime.py του αποτελέσματος του φιλέτου πραγματικού χρόνου χωρίς history buffer.*

Για περαιτέρω εξέταση στην εικόνα 4.7 παρουσιάζεται το σήμα χωρίς την εσωτερική μνήμη. Διακρίνεται ότι στην αρχή κάθε νέο δείγματος το σήμα έχει έντονο θόρυβο.



***Εικόνα 4.8**: Διάγραμμα που παρήγαγε το inspiret_realtimefft_visualizer.py*

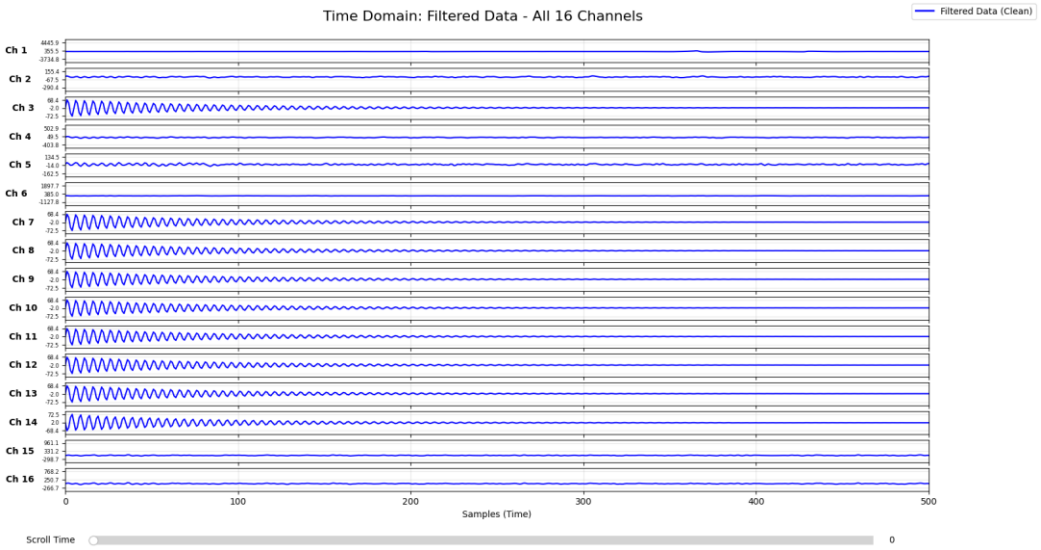
Μέσω του αρχείου inspiret_realtimefft_visualizer.py απεικονίζεται το φάσμα συχνοτήτων ενός μόνο καναλιού, μετά την εφαρμογή του φίλτρου πραγματικού χρόνου, όπως παρουσιάζεται στην εικόνα 4.8. Παρατηρείται ότι το φίλτρο στα 25 Hz είναι ελαφρώς πιο ισχυρό από ό,τι θα έπρεπε (προκαλώντας μεγαλύτερη εξασθένηση από την επιθυμητή).



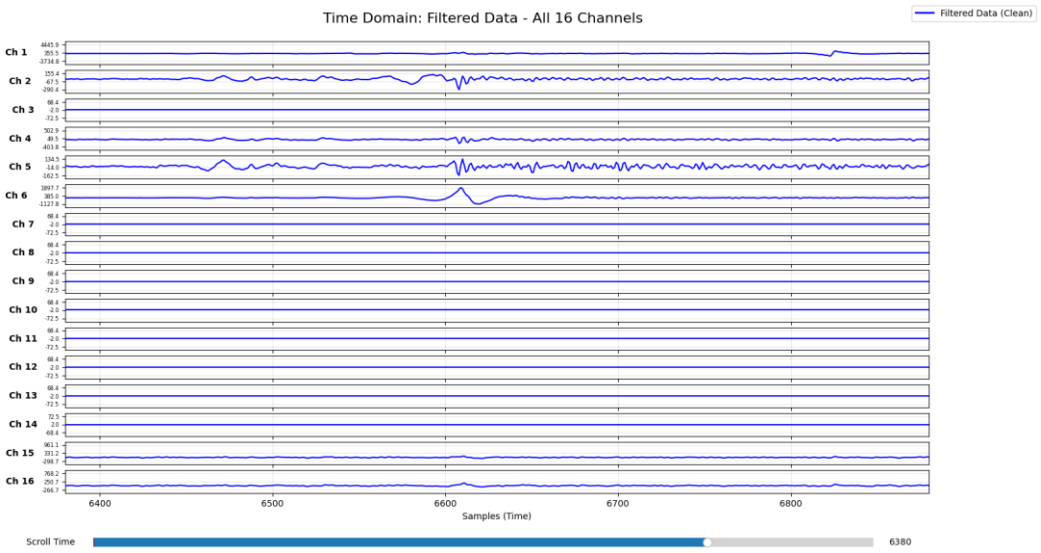
***Εικόνα 4.9**: Διάγραμμα που παρήγαγε το inspiret_realtimefft_visualizermulty.py*

Μέσω του αρχείου inspiret_realtimefft_visualizermulty.py απεικονίζεται το φάσμα συχνοτήτων σε όλα τα κανάλια, μετά την εφαρμογή του φίλτρου πραγματικού χρόνου, όπως παρουσιάζεται στην εικόνα 4.9. Σε αντίθεση με την προηγούμενη περίπτωση, παρατηρείται ότι το φίλτρο στα 25 Hz είναι ελαφρώς πιο «αδύναμο» (προκαλεί μικρότερη εξασθένηση) από ό,τι θα έπρεπε. Επίσης, διακρίνετε διαφορετική ισχύ στον θόρυβο μεταξύ των καναλιών.

Ακόμη, τα προγράμματα που δεν παρουσιάζουν το μη φιλτραρισμένο σήμα (όπως το inspiret_present.py) διαθέτουν τη δυνατότητα οριζόντιας κύλισης (scrolling) μέσω ενός widget, ώστε να μπορεί να αναδειχθεί και να εξεταστεί ολόκληρο το σήμα. Αυτή η λειτουργία είναι παρόμοια με πολλά εξειδικευμένα προγράμματα εξέτασης ηλεκτροεγκεφαλογραφημάτων.

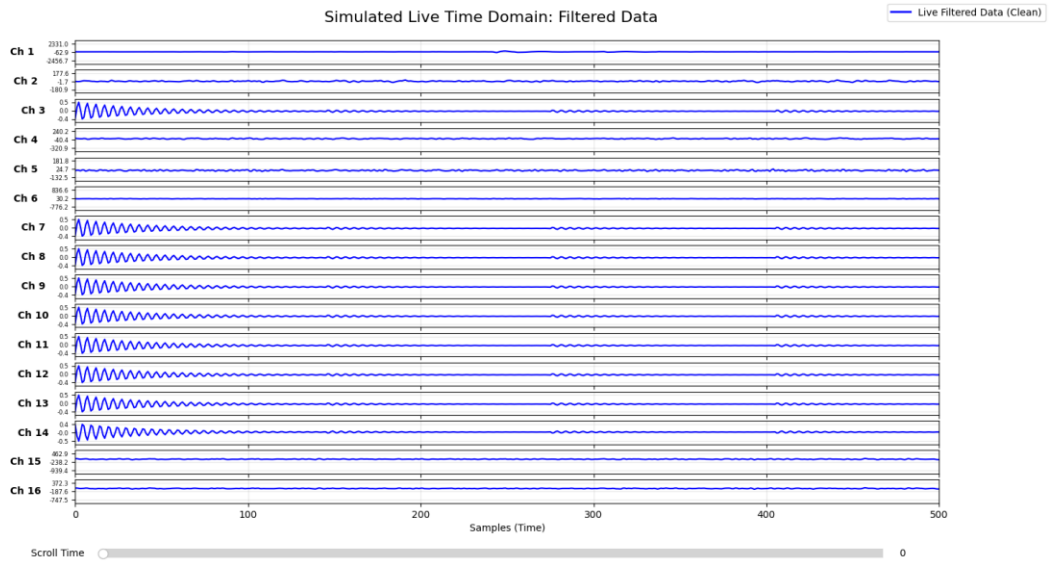


Εικόνα 4.10: *Διάγραμμα που παρήγαγε το inspiret_present.py.*

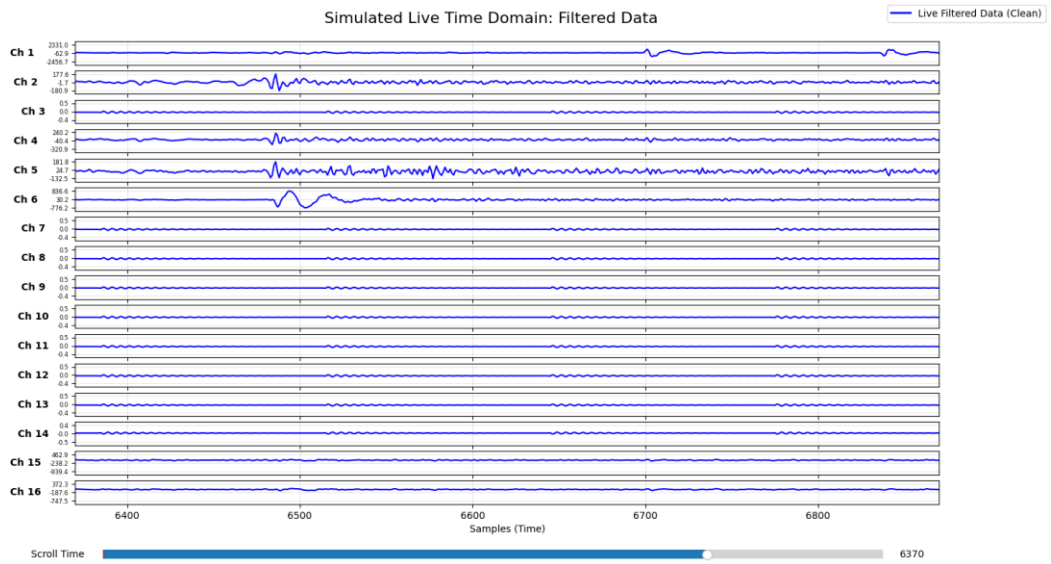


Εικόνα 4.11*: Διάγραμμα που παρήγαγε το inspiret_present.py.*

Στην εικόνα 4.10 παρουσιάζεται το σήμα (φιλτραρισμένο με το φίλτρο μη πραγματικού χρόνου) στην αρχή της καταγραφής, καθώς και σε ένα σημείο κατά το οποίο υπήρξε κίνηση του κράνους (εικόνα 4.11). Παρατηρείτε ότι κάθε κανάλι παρουσιάζει διαφορετικό πλάτος, ενώ τα κανάλια τα οποία δεν λειτουργούσαν εμφανίζουν στην αρχή της καταγραφής ένα χαρακτηριστικό ημιτονοειδές κύμα, με σχετικά μικρό πλάτος, το οποίο σταδιακά φθίνει.



Εικόνα 4.12: *Διάγραμμα που παρήγαγε το inspiret_presentrealtime.py.*



Εικόνα 4.13*: Διάγραμμα που παρήγαγε το inspiret_presentrealtime.py.*

Στις εικόνες 4.12 και 4.13 παρουσιάζεται το σήμα (φιλτραρισμένο με το φίλτρο πραγματικού χρόνου) στην αρχή της καταγραφής, καθώς και σε ένα σημείο κατά το οποίο υπήρξε κίνηση του κράνους, όπως και στην προηγούμενη περίπτωση. Παρατηρείτε ότι κάθε κανάλι παρουσιάζει διαφορετικό πλάτος κύματος. Τα κανάλια τα οποία δεν λειτουργούσαν εμφανίζουν στην αρχή της καταγραφής ένα χαρακτηριστικό ημιτονοειδές κύμα, με σχετικά μικρό πλάτος, το οποίο σταδιακά φθίνει. Σε αντίθεση με πριν, διακρίνεται ότι υπάρχει μια μικρή επανεμφάνιση αυτού του παρασιτικού σήματος κατά την αρχή του εκάστοτε νέου τμήματος δεδομένων (chunk). Επίσης, σε σχέση με το φίλτρο μη πραγματικού χρόνου, το πλάτος του σήματος είναι διαφορετικό, γεγονός που θα σχολιαστεί στη συνέχεια.

5. ΣΥΖΗΤΗΣΗ & ΣΥΜΠΕΡΑΣΜΑΤΑ

5.1 ΣΥΖΗΤΗΣΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ

Στο υποκεφάλαιο αυτό συζητούνται κριτικά τα πειραματικά ευρήματα που παρουσιάστηκαν στο Κεφάλαιο 4. Σε αντίθεση με την απλή περιγραφή των αποτελεσμάτων, εδώ επιχειρείται η ερμηνεία τους, η σύγκριση με τη θεωρητική προσέγγιση, η ανάλυση τυχόν αποκλίσεων και η αξιολόγηση της συνολικής αρχιτεκτονικής του συστήματος.

5.1.1 Ερμηνεία βασικών λειτουργικών ευρημάτων

Επικοινωνία Raspberry Pi-υπολογιστή

Η επικοινωνία μεταξύ του φορητού υπολογιστή (laptop) στην προκειμένη εργασία — αλλά και οποιασδήποτε άλλης υπολογιστικής μονάδας— επιτυγχάνεται εύκολα. Επιπλέον, εξασφαλίζεται πλήρως η φορητότητα του συστήματος, καθώς η σύνδεση μπορεί να πραγματοποιηθεί μέσω ενός φορητού σημείου πρόσβασης (hotspot) ενός κινητού τηλεφώνου ή, εναλλακτικά, μέσω ενός καλωδίου USB-to-Ethernet.

Επικοινωνία Raspberry Pi-dongle-EEG

Μέσω του dongle, το σύστημα του EEG επικοινωνεί ικανοποιητικά και σταθερά με το Raspberry Pi, εφόσον βρίσκεται σε ακτίνα περίπου 40 εκατοστών. Ωστόσο, το σύστημα παρουσιάζει αστάθεια στην επικοινωνία όταν η τάση των μπαταριών του EEG μειωθεί, παρότι δεν παρέχεται κάποια σχετική (οπτική) ένδειξη στην ίδια τη συσκευή.

Σε γενικές γραμμές, το σύστημα του EEG φαίνεται να λειτουργεί επαρκώς, με εξαίρεση το πρόβλημα τροφοδοσίας που μόλις προαναφέρθηκε. Εντούτοις, λόγω της απουσίας εξειδικευμένου χειριστή κατά τις δοκιμές, ήταν δυσχερής η ταυτόχρονη και ορθή λήψη σήματος από πολλά κανάλια μαζί, γεγονός που διαφαίνεται και στα παραδείγματα των καταγραφών.

Raspberry Pi και εσωτερικο λογισμικο

Συνολικά, το υπολογιστικό σύστημα λειτουργεί ικανοποιητικά. Τα μοναδικά προβλήματα προκύπτουν έπειτα από πολλές ώρες συνεχούς λειτουργίας, λόγω της υπερθέρμανσης του επεξεργαστή, όπως προαναφέρθηκε. Κατά την εκκίνηση του προγράμματος εμφανίζονται ορισμένες προειδοποιήσεις (warnings) στην κονσόλα, οι οποίες ωστόσο είναι αμελητέες.

Για τους σκοπούς της παρούσας εργασίας, το συγκεκριμένο Raspberry Pi κρίνεται επαρκές. Ωστόσο, σε μια ενδεχόμενη μελλοντική μετεξέλιξη του συστήματος, εκτιμάται ότι θα απαιτηθεί μεγαλύτερη επεξεργαστική ισχύς. Ακόμη, ο διαθέσιμος αποθηκευτικός χώρος για τις καταγραφές δεν είναι ιδιαίτερα μεγάλος, γεγονός που επιβάλλει τη μεταφορά των δεδομένων σε κάποιο εξωτερικό μέσο έπειτα από μικρό αριθμό συνεδριών.

Φιλτράρισμα και απεικόνιση

Παρότι το φιλτράρισμα του σήματος δεν αποτελούσε το κύριο αντικείμενο της παρούσας εργασίας, φαίνεται πως, παρά την ύπαρξη ορισμένων μικροπροβλημάτων, λειτουργεί ικανοποιητικά. Συγκεκριμένα, αποκόπτει επιτυχώς τη συνεχή συνιστώσα

(DC), τις μη επιθυμητές συχνότητες, καθώς και τις παρεμβολές από τα 50 Hz και 25 Hz.

Υπάρχουν δύο μορφές απεικόνισης του σήματος: αυτή του γραφικού περιβάλλοντος (interface), το οποίο αναδεικνύει τα χαρακτηριστικά σε πραγματικό χρόνο, και αυτή των απεικονιστικών προγραμμάτων (αρχεία inspiret_...) έπειτα από τη λήψη της καταγραφής.

Στο γραφικό περιβάλλον σε πραγματικό χρόνο, τα χαρακτηριστικά που εξάγονται από το σήμα απεικονίζονται μέσω δύο στηλών (ραβδογραμμάτων), όπου το ύψος τους αντιστοιχεί στην αριθμητική τους τιμή. Στο πλαίσιο της παρούσας εργασίας, τα χαρακτηριστικά αυτά είναι η ισχύς στις ζώνες Βήτα και Θήτα. Η κλίμακά τους δεν είναι σταθερή, αλλά αναπροσαρμόζεται (μεταβάλλεται δυναμικά) ανάλογα με τις τιμές τους, καθώς η λήψη της τελικής απόφασης βασίζεται στη μεταξύ τους σύγκριση.

Το σήμα, όπως απεικονίζεται μετά το φιλτράρισμα, είναι πολύ πιο ευανάγνωστο και μπορεί να αναλυθεί οπτικά με μεγαλύτερη ευκολία. Μπορεί να πραγματοποιηθεί έλεγχος των λειτουργικών και μη λειτουργικών καναλιών σε μια καταγραφή, παρότι δεν έχει ενσωματωθεί ακόμη κάποιο εξειδικευμένο τμήμα κώδικα (ρουτίνα) αποκλειστικά για αυτήν τη διαδικασία. Επιπλέον, παρέχεται η δυνατότητα εξέτασης του φάσματος συχνοτήτων της καταγραφής και του τρόπου με τον οποίο τα φίλτρα επιδρούν σε αυτό. Η ανάλυση του φάσματος συχνοτήτων μπορεί να πραγματοποιηθεί είτε συνολικά (ως μέσος όρος) είτε εξατομικευμένα για κάθε κανάλι.

Εξαγωγή χαρακτηριστικών και αποφάσεις

Σε πειραματικό στάδιο, φαίνεται εφικτό το πρόγραμμα να εξάγει χαρακτηριστικά και να λαμβάνει αποφάσεις με βάση αυτά, θεωρώντας βέβαια ότι η ποιότητα των καταγραφών είναι αρκετά καλή και ότι επαρκεί η υπολογιστική ισχύς του συστήματος.

Η λήψη της απόφασης εκτελείται με μια πολύ απλή λογική σύγκρισης των τιμών των χαρακτηριστικών. Συγκεκριμένα, αν η ισχύς της ζώνης Θήτα είναι διπλάσια από την ισχύ της ζώνης Βήτα, τότε η απόφαση είναι θετική (κατάσταση Power On).

Έξοδος Led

Το σύστημα οπτικής ένδειξης της εξόδου με τη μορφή LED λειτουργεί ορθά. Η θετική απόφαση του συστήματος αντιστοιχεί στην ενεργοποίηση (άναμμα) του LED. Προς το παρόν, το LED αποτελεί μια μονή έξοδο (single output). Στην πραγματική εφαρμογή, ωστόσο, θα πρέπει να χρησιμοποιηθούν περισσότεροι ακροδέκτες (GPIO pins) του Raspberry Pi. Επιπλέον, το σήμα που τώρα καταλήγει στο LED μπορεί να οδηγηθεί σε οποιοδήποτε άλλο κύκλωμα εισόδου για την ενεργοποίηση ενός εξωτερικού συστήματος (όπως, για παράδειγμα, στον ελεγκτή ενός αναπηρικού αμαξιδίου).

5.1.2 Σύγκριση με τον αρχικό στόχο της εργασίας

Τα αποτελέσματα δείχνουν ότι επιτεύχθηκε ο βασικός στόχος της εργασίας, δηλαδή η ανάπτυξη μίας πειραματικής διάταξης ικανής να επεξεργάζεται και να αναλύει τα σήματα από ένα σύστημα EEG για τη λήψη μιας απόφασης, χωρίς, ωστόσο, αυτό να

έχει υλοποιηθεί ακόμη στο πλαίσιο μιας πλήρως πρακτικής εφαρμογής. Η αρχιτεκτονική του προγράμματος δύναται να αποτελέσει το θεμέλιο για την εύκολη δημιουργία ενός ολοκληρωμένου συστήματος λογισμικού, το οποίο θα μπορεί να λαμβάνει αποφάσεις με βάση τις καταγραφές του συγκεκριμένου EEG, ενδεχομένως και σε συνδυασμό με δεδομένα από άλλους αισθητήρες.

5.2 ΠΡΟΤΑΣΕΙΣ ΓΙΑ ΜΕΛΛΟΝΤΙΚΗ ΕΞΕΛΙΞΗ

Η διάταξη που παρουσιάστηκε στην παρούσα εργασία αποτελεί τη μετέπειτα εξέλιξη ενός ευρύτερου συστήματος. Παραθέτει ένα δομημένο πλαίσιο στο οποίο μπορούν εύκολα να ενσωματωθούν περαιτέρω τμήματα για την ανάπτυξή του, ενώ παράλληλα παρέχει τα απαραίτητα εργαλεία για τη δημιουργία αυτών των μελλοντικών τμημάτων. Βέβαια, για την υπέρβαση των υφιστάμενων υπολογιστικών περιορισμών, θεωρείται πολύ πιθανό ότι θα απαιτηθεί αναβάθμιση και στο υλικό (hardware). Αναλυτικότερα, προτείνονται οι εξής μελλοντικές βελτιώσεις και επεκτάσεις του συστήματος:

Αναβάθμιση υπολογιστικού συστήματος: Το τρέχον σύστημα βρίσκεται πολύ κοντά στο όριο της επεξεργαστικής του ισχύος, χωρίς να υπάρχουν πολλά περιθώρια εξοικονόμησης πόρων και περαιτέρω βελτιστοποίησης. Προτείνεται, μελλοντικά, να γίνει αναβάθμιση της κεντρικής μονάδας σε Raspberry Pi 5.
Βελτιστοποίηση τοποθέτησης ηλεκτροδίων: Είναι πολύ εύκολο να μετακινηθούν τα ηλεκτρόδια στις θέσεις μέγιστου ενδιαφέροντος στο κρανίο, με σκοπό τον ακριβέστερο εντοπισμό των εκάστοτε υπό εξέταση συμβάντων (π.χ. φανταστική κίνηση).
Βελτίωση φιλτραρίσματος: Το τρέχον φιλτράρισμα του συστήματος είναι βασικό. Θα μπορούσε να αναπτυχθεί πολύ περισσότερο, εφόσον αυτό κριθεί ωφέλιμο για την εξαγωγή χαρακτηριστικών. Ακόμη, θα μπορούσε να εφαρμοστεί εξατομικευμένο φιλτράρισμα σε κάθε κανάλι ξεχωριστά, καθώς οι παρεμβολές που λαμβάνει το καθένα διαφέρουν ελαφρώς.
Μηχανική Μάθηση (Machine Learning) και λήψη αποφάσεων: Η εκπαίδευση ενός μοντέλου για τη λήψη αποφάσεων αποτελεί το κυριότερο μελλοντικό βήμα στην εξέλιξη του συστήματος. Καθώς η διάταξη είναι εύχρηστη, μπορεί να συλλεχθεί μεγάλος αριθμός καταγραφών για τη δημιουργία μιας εκτενούς βάσης δεδομένων. Περαιτέρω, θεωρείται ότι τεχνικές όπως η Μεταφερόμενη Μάθηση (Transfer Learning) θα ήταν ιδιαίτερα χρήσιμες. Ο χρήστης θα μπορούσε, κατά την πρώτη χρήση της συσκευής, να πραγματοποιεί μερικές καταγραφές βαθμονόμησης (calibration), ώστε το σύστημα να προσαρμόζεται εξατομικευμένα στα δικά του σήματα.
Ενσωμάτηση δεδομένων από άλλους αισθητήρες (Sensor Fusion): Δεν υπάρχει περιορισμός στη χρήση αποκλειστικά του EEG για τη λήψη μιας απόφασης (και κατά συνέπεια, για την εκπαίδευση ενός μοντέλου). Το λογισμικό INSPiRE διαθέτει ήδη τη δυνατότητα σύνδεσης και εξαγωγής δεδομένων συνεργατικά από άλλα συστήματα και αισθητήρες.
Επέκταση εξόδων ελέγχου: Το σύστημα, όπως δίνει μία έξοδο ελέγχου τώρα (π.χ. το LED), μπορεί να παρέχει άλλες 26 (μέσω των ακροδεκτών GPIO). Αυτές μπορούν να αντιστοιχούν σε διαφορετικές εντολές προς ένα άλλο υποσύστημα, ελέγχοντας την εναλλαγή (toggle) ενός ακροδέκτη με χρόνο απόκρισης 10 έως 25 μικροδευτερόλεπτα (microseconds). Το πρόγραμμα INSPiRE αρχικά προοριζόταν για τον έλεγχο ενός συστήματος Λειτουργικής Ηλεκτρικής Διέγερσης (FES - Functional Electrical

Stimulation) και ενός αναπηρικού αμαξιδίου, ωστόσο δεν υπάρχει κανένας περιορισμός για περαιτέρω χρήσεις.
Φορητότητα και ευελιξία τροφοδοσίας: Για την πλήρη ευελιξία του συστήματος (ώστε να μπορεί να μεταφερθεί οπουδήποτε), πρέπει να ενσωματωθεί μια φορητή πηγή τροφοδοσίας. Το Raspberry Pi, η δική του τροφοδοσία, καθώς και η τροφοδοσία του EEG θα μπορούσαν να ενσωματωθούν είτε πάνω στον σκελετό του EEG, είτε σε κάποιο προστατευτικό περίβλημα το οποίο να μπορεί να τοποθετηθεί, χωρίς κίνδυνο βλάβης, σε ένα σακίδιο ή στο αμαξίδιο. Η πηγή τροφοδοσίας θα μπορούσε να είναι κοινή, κάνοντας χρήση κατάλληλων μετατροπέων τάσης (voltage regulators) για κάθε συσκευή ξεχωριστά. Παράλληλα, λόγω των προβλημάτων που προαναφέρθηκαν, θα ήταν σημαντικό να ενσωματωθεί ένας δείκτης ελέγχου της σωστής τάσης προς το EEG.
Εναλλακτική χρήση φορητού υπολογιστή: Το Raspberry Pi μπορεί να αντικατασταθεί από έναν φορητό υπολογιστή (laptop) με την κατάλληλη προσαρμογή του λογισμικού, εφόσον αυτός διαθέτει τα απαιτούμενα τεχνικά χαρακτηριστικά. Η χρήση ενός laptop, για κάποιον χρήστη που ήδη το κατέχει, μπορεί να μειώσει ακόμη περισσότερο το κόστος του συστήματος, περιορίζοντάς το αποκλειστικά στο κόστος αγοράς του EEG.

5.3 ΓΕΝΙΚΑ ΣΥΜΠΕΡΑΣΜΑΤΑ

Η παρούσα διπλωματική εργασία είχε ως αντικείμενο τον σχεδιασμό και την υλοποίηση ενός συστήματος ανάλυσης και επεξεργασίας σημάτων από έναν φορητό σύστημα EEG με τη χρήση μικροϋπολογιστή Raspberry Pi. Στόχος ήταν να αποδειχθεί ότι με ένα σχετικά οικονομικό σύστημα μπορεί να λειτουργήσει μια ολοκληρωμένη διάταξη BCI, η οποία εκτελείται εξ ολοκλήρου τοπικά και αυτόνομα.

Το σύστημα διακρίνεται για τη φορητότητα και την ευχρηστία του, προσφέροντας μεγάλη ευελιξία στη χρήση και δυνατότητα απρόσκοπτης συλλογής δεδομένων για την εκχώρηση και εκπαίδευση μελλοντικών μοντέλων μηχανικής μάθησης. Παράλληλα, περιλαμβάνει βοηθητικά εργαλεία για τη διευκόλυνση μελλοντικών επεκτάσεων και αναβαθμίσεων. Η βασική καινοτομία της παρούσας εργασίας έγκειται στην ανάπτυξη μιας χαμηλού κόστους πειραματικής υποδομής η οποία, λαμβάνοντας δεδομένα από ένα συστημα EEG, επεξεργάζεται τα σήματα, τα αναλύει και εξάγει απόφαση σε πραγματικό χρόνο, επιτρέποντας την εύκολη ενσωμάτωση εξωτερικών συσκευών που αξιοποιούν άμεσα τα αποτελέσματα αυτών των αποφάσεων.

Σύγκριση με πραγματικές συνθήκες:
Όπως έχει προαναφερθεί, η χρήση των EEG αντιμετωπίζει αρκετά προβλήματα όταν αυτοί χρησιμοποιούνται εκτός εργαστηριακού περιβάλλοντος. Οι καταγραφές είναι ιδιαίτερα ευάλωτες σε διάφορες ψευδενδείξεις, γεγονός που καθιστά απαραίτητο το λογισμικό να διαθέτει τη δυνατότητα αποτελεσματικής αντιμετώπισής τους. Παρ' όλα αυτά, ο έλεγχος εξωτερικών συσκευών αποκλειστικά μέσω σημάτων EEG είναι απολύτως εφικτός και εφαρμόζεται ευρέως στη σύγχρονη τεχνολογία.

Τελική αποτίμηση:
Η εργασία απέδειξε ότι είναι δυνατή η υλοποίηση ενός λειτουργικού συστήματος προσομοίωσης ελέγχου με χαμηλό κόστος και εργαλεία ανοιχτού κώδικα. Η συνεισφορά της είναι κυρίως μεθοδολογική και τεχνολογική, ενώ προσφέρει τη δυνατότητα για την αφομοίωση και ανάπτυξη διάφορων εφαρμογών. Το σύστημα θεωρείται κατάλληλο για πολύ εύκολη εκπαίδευση, χάρη στην απλότητα συλλογής

δεδομένων. Περαιτέρω, για ακόμη καλύτερα αποτελέσματα, η διαδικασία θα μπορούσε να εξατομικευτεί για κάθε χρήστη της συσκευής, είτε με την εκπαίδευση του μοντέλου με αποκλειστικά δικά του δεδομένα είτε με τη χρήση transfer learning πάνω σε ένα προεκπαιδευμένο γενικό μοντέλο. Ένα ακόμη μέσο που θα μπορούσε να ενσωματωθεί για τη βελτίωση της λήψης αποφάσεων είναι η χρήση άλλων μορφών καταγραφών, όπως το ηλεκτρομυογράφημα (EMG), το οποίο μάλιστα είναι ήδη ενσωματωμένο στο πρόγραμμα.

Η ευελιξία αυτής της εργασίας τής επιτρέπει να αποτελεί ένα ακόμη βήμα προς την ανάπτυξη προσιτών και προσαρμόσιμων υποστηρικτικών τεχνολογιών, ικανών να καλύπτουν τις εξατομικευμένες ανάγκες κάθε χρήστη. Παρόλο που μελλοντικά ενδέχεται να απαιτηθεί αναβάθμιση στην υπολογιστική ισχύ της διάταξης, τα ευρήματα υποδεικνύουν σαφώς πως η δομή που αναπτύχθηκε προσφέρει ένα βιώσιμο θεμέλιο για τη δημιουργία μελλοντικών συστημάτων BCI για καθημερινή χρήση.

Αναφορές - Πηγές

[1] Abhang, P. A., Gawali, B. W., & Mehrotra, S. C. (2016). *Introduction to EEG- and speech-based emotion recognition*. Academic Press.

[2] Akor, P. A., Enemali, G., Muhammad, U., Singh, R. R., & Larijani, H. (2025). An attention-residual convolutional network for real-time seizure classification on edge devices. *Sensors*, *25*(22), Article 6855. https://doi.org/10.3390/s25226855

[3] Arns, M., Conners, C. K., & Kraemer, H. C. (2013). A decade of EEG theta/beta ratio research in ADHD: A meta-analysis. *Journal of Attention Disorders*, *17*(5), 374–383. https://doi.org/10.1177/1087054712460087

[4] Artinis Medical Systems. (n.d.). *fNIRS in brain-computer interface and neurofeedback*. Artinis. https://artinis.com/blogpost-all/fnirs-in-brain-computer-interface-and-neurofeedback

[5] Bandes Rodriguez, S. I., & Koike, Y. (2025). Hybrid EEG-EMG control scheme for multiple degrees of freedom upper-limb prostheses. *Actuators*, *14*(8), Article 397. https://doi.org/10.3390/act14080397

[6] Barry, R. J., Clarke, A. R., & Johnstone, S. J. (2003). A review of electrophysiology in attention-deficit/hyperactivity disorder: I. Qualitative and quantitative electroencephalography. *Clinical Neurophysiology*, *114*(2), 171–183. https://doi.org/10.1016/S1388-2457(02)00362-0

[7] Bear, M. F., Connors, B. W., & Paradiso, M. A. (2020). *Neuroscience: Exploring the brain* (4th ed.). Jones & Bartlett Learning.

[8] Bronzino, J. D. (2000). *The biomedical engineering handbook* (2nd ed.). CRC Press.

[9] Buzsáki, G. (2006). *Rhythms of the brain*. Oxford University Press.

[10] Cardona-Álvarez, Y. N., Álvarez-Meza, A. M., Cárdenas-Peña, D. A., Castaño-Duque, G. A., & Castellanos-Dominguez, G. (2023). A novel OpenBCI framework for EEG-based neurophysiological experiments. *Sensors*, *23*(7), Article 3763. https://doi.org/10.3390/s23073763

[11] Chen, B., Jia, Z., & Mark, R. (2026). *Multimodal brain-computer interface grand challenge: EEG-fNIRS based handwriting-trajectory classification* [Conference challenge proposal]. OpenReview. https://openreview.net/forum?id=u2SnYEmWIK

[12] Cohen, M. X. (2014). *Analyzing neural time series data: Theory and practice*. MIT Press.

[13] Engel, A. K., & Fries, P. (2010). Beta-band oscillations—signalling the status quo? *Current Opinion in Neurobiology*, *20*(2), 156–165. https://doi.org/10.1016/j.conb.2010.02.015

[14] Gonzalez, J. (n.d.). *Raspberry Pi 3 not your average pie*. Hackster.io. https://www.hackster.io/julio-gonzalez/raspberry-pi-3-not-your-average-pie-62935a

[15] Ilyas, M. Z., Saad, P., Ahmad, M. I., & Ghani, A. R. I. (2016, November). Classification of EEG signals for brain-computer interface applications: Performance comparison. In *2016 International Conference on Robotics, Automation and Sciences (ICORAS)* (pp. 1–5). IEEE. https://doi.org/10.1109/ICORAS.2016.7872610

[16] Kandel, E. R., Schwartz, J. H., Jessell, T. M., Siegelbaum, S. A., & Hudspeth, A. J. (2012). *Principles of neural science* (5th ed.). McGraw-Hill. https://ftp.columbia.edu/itc/gsas/g9600/2003/HenReadings/KandellCh3.pdf

[17] Klimesch, W. (1999). EEG alpha and theta oscillations reflect cognitive and memory performance: A review and analysis. *Brain Research Reviews*, *29*(2–3), 169–195. https://doi.org/10.1016/S0165-0173(98)00056-3

[18] Knierim, M. T., Bleichner, M. G., & Reali, P. (2023). A systematic comparison of high-end and low-cost EEG amplifiers for concealed, around-the-ear EEG recordings. *Sensors*, *23*(9), Article 4559. https://doi.org/10.3390/s23094559

[19] Knyazev, G. G., Savostyanov, A. N., & Levin, E. A. (2004). EEG correlates of trait anxiety in the normal ECG/EEG baseline. *Neuroscience Letters*, *363*(1), 16–19. https://doi.org/10.1016/j.neulet.2004.03.051

[20] Kosnacova, H., Horvath, D., Vitazkova, D., Foltan, E., Pecik, M., & Vavrinsky, E. (2026). Smart wearable EEG devices: A review of lightweight, multi-sensor systems for sleep and everyday neurophysiology. *Biosensors*, *16*(7), Article 374. https://doi.org/10.3390/bios16070374

[21] Lal, S. K., & Craig, A. (2001). A critical review of the psychophysiology of driver fatigue. *Biological Psychology*, *55*(3), 173–194. https://doi.org/10.1016/S0301-0511(00)00085-5

[22] Li, R. (2026). Multimodal learning in brain-computer interfaces: A research review on applications. In *Proceedings of the 2025 International Conference on Electronics, Electrical*

and Grid Technology (ICEEGT 2025) (pp. 546–554). Atlantis Press. https://doi.org/10.2991/978-94-6463-986-5_56

[23] Liang, T., & Damoah, A. (2024). *EEGMobile: Enhancing speed and accuracy in EEG-based gaze prediction with advanced mobile architectures* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2408.03449

[24] López-Larraz, E., Sarasola-Sanz, A., Birbaumer, N., & Ramos-Murguialday, A. (2024). *Unveiling movement intention after stroke: Integrating EEG and EMG for motor rehabilitation* [Preprint]. bioRxiv. https://doi.org/10.1101/2024.02.22.581596

[25] Mitchell, P., Moxon, K. A., et al. (2021). Motor neuroprosthesis implanted with neurointerventional surgery improves capacity for activities of daily living tasks in severe paralysis. *Journal of NeuroInterventional Surgery*, *13*(7), 647–651. https://doi.org/10.1136/neurintsurg-2020-016853

[26] Nguyen, M.-D., Do, T., Tran, X.-T., Nguyen, Q.-T., & Lin, C.-T. (2025). Edge AI-brain–computer interfaces system: A survey. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, *33*, 4051–4066. https://doi.org/10.1109/TNSRE.2025.3618688

[27] Niedermeyer, E., & da Silva, F. L. (2005). *Electroencephalography: Basic principles, clinical applications, and related fields* (5th ed.). Lippincott Williams & Wilkins.

[28] Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-time signal processing* (3rd ed.). Prentice Hall.

[29] PatSnap. (2026, March 5). *Evaluating brain-computer interface integration with edge computing platforms*. PatSnap Eureka. https://eureka.patsnap.com/report-evaluating-brain-computer-interface-integration-with-edge-computing-platforms

[30] Proakis, J. G., & Manolakis, D. G. (2006). *Digital signal processing: Principles, algorithms, and applications* (4th ed.). Pearson Prentice Hall.

[31] Qiu, L., Zhong, Y., He, Z., & Pan, J. (2022). Improved classification performance of EEG-fNIRS multimodal brain-computer interface based on multi-domain features and multi-level progressive learning. *Frontiers in Human Neuroscience*, *16*, Article 973959. https://doi.org/10.3389/fnhum.2022.973959

[32] Raspberry Pi. (n.d.-a). *GPIO and the 40-pin header*. Retrieved July 25, 2026, from https://www.raspberrypi.com/documentation/computers/os.html#gpio-and-the-40-pin-header

[33] Raspberry Pi. (n.d.-b). *Raspberry Pi hardware*. Retrieved July 25, 2026, from https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[34] Raspberry Pi. (n.d.-c). *Remote access*. Retrieved July 25, 2026, from https://www.raspberrypi.com/documentation/computers/remote-access.html

[35] Ray, W. J., & Cole, H. W. (1985). EEG alpha activity reflects attentional demands, and beta activity reflects emotional and cognitive processes. *Science*, *228*(4700), 750–752. https://doi.org/10.1126/science.3992243

[36] Sanei, S., & Chambers, J. A. (2013). *EEG signal processing and feature extraction*. John Wiley & Sons.

[37] Schomer, D. L., & Lopes da Silva, F. H. (Eds.). (2017). *Niedermeyer's electroencephalography: Basic principles, clinical applications, and related fields* (7th ed.). Oxford University Press.

[38] Shafin, A. H., & Rahman, M. S. (2025, December). Design and implementation of a hybrid BCI using EEG and EOG. In *2025 28th International Conference on Computer and Information Technology (ICCIT)* (pp. 3224–3229). IEEE. https://doi.org/10.1109/ICCIT66964.2025.11490421

[39] Solé-Casals, J., Caiafa, C. F., Zhao, Q., & Cichocki, A. (2018). Brain-computer interface with corrupted EEG data: A tensor completion approach. *Cognitive Computation*, *10*(6), 1062–1074. https://doi.org/10.1007/s12559-018-9574-9

[40] Sun, J., & Liu, Y. (2022). A hybrid asynchronous brain–computer interface based on SSVEP and eye-tracking for threatening pedestrian identification in driving. *Electronics*, *11*(19), Article 3171. https://doi.org/10.3390/electronics11193171

[41] Tragoudaras, A., Antoniadis, C., & Massoud, Y. (2023). TinyML for EEG decoding on microcontrollers. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)* (pp. 1–5). IEEE. https://doi.org/10.1109/iscas46773.2023.10181950

[42] Wascher, E., Rasch, B., & Sänger, J. (2014). Frontal theta activity reflects distinct aspects of mental fatigue. *Biological Psychology*, *96*, 57–65. https://doi.org/10.1016/j.biopsycho.2013.11.007

[43] Widmann, A., Schröger, E., & Maess, B. (2015). Digital filter design for electrophysiological data–a practical approach. *Journal of Neuroscience Methods*, *250*, 34–46. https://doi.org/10.1016/j.jneumeth.2014.08.002

[44] Wolpaw, J. R., & Wolpaw, E. W. (2012). *Brain-computer interfaces: Principles and practice*. Oxford University Press.

[45] Yi, Z., Pan, J., Chen, Z., Lu, D., Cai, H., Li, J., & Xie, Q. (2024). A hybrid BCI integrating EEG and eye-tracking for assisting clinical communication in patients with disorders of consciousness. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, *32*, 2759–2771. https://doi.org/10.1109/TNSRE.2024.3435016

[46] Zhang, J., Li, J., Huang, Z., Huang, D., Yu, H., & Li, Z. (2023). Recent progress in wearable brain-computer interface (BCI) devices based on electroencephalogram (EEG) for medical applications: A review. *Health Data Science*, *3*, Article 0096. https://doi.org/10.34133/hds.0096

[47] Πανεπιστήμιο Δυτικής Αττικής. (2020). *INSPiRE – Υλοποίηση ολοκληρωμένου συστήματος υποστήριξης και αποκατάστασης κινητικών λειτουργιών μέσω υβριδικής διεπαφής εγκεφάλου-υπολογιστή* [Ερευνητικό έργο]. RDE Hub. https://rdehub.uniwa.gr/feng/systima-ypostirixis-kinisis-me-chrisi-egkefalikon-simaton-inspire/

Παράρτημα 1: Κώδικας

A.1 Αρχείο inspire_main.py

```
Import path
import sys,multiprocessing
import csv sys.path.insert(1, '/home/pi/Desktop/Documents/INSPiRE/Package')
################################################
Import modules (Python & custom modules)
import time from datetime
import datetime
import inspire_config, inspire_objects, inspire_threads,
inspire_datarecord, inspire_interface,inspire_filters
Commands for reloading modules in case of an update during debugging
#from importlib import reload #reload(inspire_objects) # declare which
module you wish to reload
################################################
Set session variables
participant = "panagiotis" # Participant name / identifier exp_time =
datetime.now() # Save current timestamp for session folder scale = True #
Rescale input values or store raw data guide = False # Provide stimulus for
movement task_type = 2 # 1: walking, 2: wheelchair process = True # Produce
real-time output feedback = True # Produce real-time feedback save_data =
True # Save session data filt = True # Apply frequency filters in the data
################################################
Initialise parameters
pars =
inspire_config.params(participant,exp_time,save_data,scale,guide,process,ta
sk_type) # Experiment parameters
print(pars._fields)
################################################
Create objects & Define data headers
objects, msg = inspire_objects.objects(pars) pars.session.gpio_headers,
pars.session.openbci_headers = inspire_objects.data_headers(objects,pars)
################################################
Create Shared variables
NOTE: Needed if multiprocessing library (vs threading) is used
#pygame = multiprocessing.Array('i',[0]) #screen =
multiprocessing.Array('i',[0]) #canvas = multiprocessing.Array('i',[0])
################################################
Initialise interface
(NOTE: use 1st instead of 2nd command if threading is used instead of
multiprocessing)
[pygame,screen,pos,canvas] =
inspire_interface.interface(objects,pars.config,msg)
#[pygame.value,screen.value,pos,canvas.value] =
inspire_interface.interface(objects,pars.config,msg)
###########asking if filteres shoulld be applied
filt = False # Default to False #the filtering choice is made with confirm
shared_powers = {"theta": 0.0, "beta": 0.0}
#######
Ask for confirmation
(NOTE: use 1st instead of 2nd command if threading is used instead of
multiprocessing)
[cont,pos,filt] =
inspire_interface.confirm(pygame,screen,pars.config,pos,filt)
#[cont,pos] =
inspire_interface.confirm(pygame.value,screen.value,pars.config,pos)
if not cont: exit()
```

```
################################################
Create Pipe for data exchange among parallel processes
NOTE: Only if multiprocessing is used
-parent_conn: data sender
-child_conn : data receiver
#conn1,conn2 = multiprocessing.Pipe(duplex=True)
################################################
Create Threads
Thread for monitoring display
thread1 = inspire_threads.monitor_display_task(pygame) #thread1 =
inspire_threads.monitor_display_task()
Recording thread
NOTE: use 1st instead of 2nd command if threading is used
thread2 = inspire_threads.record_task(objects, pars, process) #thread2 =
inspire_threads.record_task(objects, pars, process, conn1)
Live feedback thread
NOTE: use 1st instead of 2nd command if threading is used
thread3 = inspire_threads.feedback_display_task(pygame, screen, pars,
canvas) #thread3 = inspire_threads.feedback_display_task(pygame, screen,
pars, canvas, conn2)
Real-time data processing
NOTE: use 1st instead of 2nd command if threading is used
thread4 = inspire_threads.process_task(objects, pars, process, feedback,
task_type, guide,filt,shared_powers) #thread4 =
inspire_threads.process_task(objects, channels, pars, process, task_type,
guide, conn2)
Start OpenBCI Stream (DEPRECATED)
if objects.openbci[0].obj == []:
print("OpenBCI stream not started")
thread0 = 0
else:
thread0 = inspire_threads.openbci_stream(objects.openbci[0].obj)
thread0.start()
time.sleep(2) # Wait until stream starts and vars are created
################################################
Test variables for multiprocessing (1st 2 commands for array, 2nd 2
commands for value)
#inspire_datarecord.gpiodata = multiprocessing.Array('i',[1,1])
#inspire_datarecord.gpiodata.value=[] #inspire_datarecord.test =
multiprocessing.Value('i') #inspire_datarecord.test.value = 1
################################################
Start Threads
thread1.start() # monitor_display_task thread2.start() # record_task if
feedback: thread3.start() # feedback_display_task if process:
thread4.start() # process_task
Command for testing, 1st for threading, 2nd for multiprocessing
#print(inspire_datarecord.recording)
#print(inspire_datarecord.recording.value)
################################################
Monitor state of threads
import pygame while True: time.sleep(0.05) try: pygame.event.pump()
pygame.display.update() except pygame.error: pass threads =
[thread1.is_alive(), thread2.is_alive(), thread3.is_alive(),
thread4.is_alive() ] #print(threads) if not any(threads): if
any(pars.session.openbci_headers): objects.openbci.stop_stream()
objects.openbci.release_session() objects.openbci.release_all_sessions()
   # DEPRECATED

if not isinstance(thread0, int):
```

```
objects.openbci.obj.stop_stream()
print("OpenBCI stream ended")
#thread0.join()
   break

thread1.join() # monitor_display_task thread2.join() # record_task if
feedback: thread3.join() # feedback_display_task if process: thread4.join()
# process_task print("Finished")
################################################
Get recorded data from connection
NOTE: Only needed if multiprocessing is used
#gpiodata = conn2.recv()
################################################
Save session data
NOTE: use 1st instead of 2nd command if threading is used
inspire_datarecord.save_sessiondata(pars,objects,inspire_datarecord.gpiodat
a) #inspire_datarecord.save_sessiondata(pars,objects,gpiodata)
if filt == True:
#the filtering happens inside the saving function
inspire_datarecord.save_filteredeegdata(pars,objects,inspire_datarecord.gpi
odata)

#inspire_filters.visualize_filter_results(pars.config.openbcidatafile,
pars.config.fopenbcidatafile)

################################################
Close
input("Press Enter to close...") #thread1.join() # monitor_display_task
#thread2.join() # record_task
```

A.2 Αρχείο inspire_config.py

```
Import modules
import os,collections,sys,serial
################################################
Call functions to define all parameters
def params(participant,exp_time,save_data,scale,guide,process,task_type):
inspire_pars = collections.namedtuple('inspire_pars', \
    ['config','inaddr','outaddr','exp',

'preprocess','featextract','process','feedback','protocol','session','guide
']])

inspire_pars.config = config_params(participant)
inspire_pars.inaddr = i2c_addr_in()
inspire_pars.outaddr = out_addr()
inspire_pars.exp = exp_params()
inspire_pars.preprocess = preprocess_params()
inspire_pars.featextract = feature_params()
inspire_pars.process = process_params()
inspire_pars.feedback = feedback_params()
inspire_pars.protocol = protocol_stimuli(task_type)

inspire_pars.session = collections.namedtuple('session',\
    ['participant','exp_time','scale','process','task_type',
     'gpio_headers','openbci_headers'])
inspire_pars.session.participant = participant
inspire_pars.session.exp_time = exp_time
inspire_pars.session.scale = scale
inspire_pars.session.process = process
```

```
inspire_pars.session.guide = guide
inspire_pars.session.task_type = 'walking' if task_type == 1 else
'wheelchair'

# Return output
return inspire_pars

################################################
Configuration parameters (directories & interfacing)
def config_params(participant):
config = collections.namedtuple('config',
    ['sessionfolder','genpath','folderout','datafile',
     'sessionfile','original_stdout',
     'gui_title','gui_width','gui_height',
     'gui_font','gui_fontsize',
     'gui_fontbgr','gui_fontcolor','gui_startpos','fopenbcidatafile'])

# Data output
config.original_stdout = sys.stdout # Save a reference to the original
standard output
config.genpath = "/home/raspberry/Desktop/Documents/INSPiRE/Data" # General
path
config.subjectfolder = config.genpath + "/" + participant

gpiodatafile = "gpiorecordings.csv" # Output file for recordings
openbcidatafile = "openbcirecordings.csv" # Output file for recordings
fopenbcidatafile = "filteredopenbcirecordings.csv" # Output file for
recordings
sessionfile = "sessiondata.csv" # Output file for session variables
logfile = "logfile.txt" # Output logfile

config.sessionfolder,config.gpiodatafile,config.openbcidatafile,\
                            config.sessionfile,config.logfile = \
    output_dirs(config.genpath,config.subjectfolder,\
                gpiodatafile,openbcidatafile,sessionfile,logfile)
config.fopenbcidatafile = config.sessionfolder + "/" + fopenbcidatafile
# Interfacing
config.gui_title, config.gui_width, config.gui_height  = "INSPiRE", 800,
500
config.gui_font, config.gui_fontsize = 'freesansbold.ttf', 15
config.gui_fontbgr, config.gui_fontcolor = [0, 0, 0], [255, 255, 255]
config.gui_startpos = [10,10]
config.fig_size, config.fig_dpi = [4, 4], 60
config.ylim_min , config.ylim_max = -0.2, 3.5

# Return output
return config

################################################
Define output folder & files
def output_dirs(genpath, subjectfolder, gpiodatafile, openbcidatafile,
sessionfile, logfile):
# Create folder for subject if needed
if not os.path.exists(genpath):
    os.mkdir(genpath)
    os.mkdir(subjectfolder)
else:
    if not os.path.exists(subjectfolder):
        os.mkdir(subjectfolder)
```

```
#Create folder for session
folders = os.listdir(subjectfolder)
existing_folders = len([s for s in folders if any("session" in s for s in
folders)])
sessionfolder = subjectfolder + "/session" + str(existing_folders+1)
os.mkdir(sessionfolder)

# Set full filenames
gpiodatafile = sessionfolder + "/" + gpiodatafile
openbcidatafile = sessionfolder + "/" + openbcidatafile
sessionfile = sessionfolder + "/" + sessionfile
logfile = sessionfolder + "/" + logfile

# Return output
return sessionfolder,gpiodatafile,openbcidatafile,sessionfile,logfile

################################################
Input addresses
def i2c_addr_in():
in_addresses = collections.namedtuple('in_addresses', 'ads_addr mpu_addr
mpu_reg_addr clock_addr')

# Input module channels for A/D Converters
in_addresses.ads_addr = collections.namedtuple('ads_addr', 'desc vals
channels')
in_addresses.ads_addr.desc = ["ADS1: 4 Myoware sensors",
                              "ADS2: 4 Pressure sensors",
                              "ADS3: 2 ECG & 2 Flex sensors",
                              "ADS4: FES input channels"
                              ]
in_addresses.ads_addr.vals = [0x48, 0x49, 0x4b, 0x4a]
in_addresses.ads_addr.channels = [["Myoware 1", "Myoware 2", "Myoware 3",
"Myoware 4"],

["Force_Toe_R","Force_Heel_R","Force_Toe_L","Force_Heel_L"],

["Flex_Knee_R","Flex_Knee_L","Pulse","ECG"],
                                  ["FES_1_R","FES_2_R","FES_1_L","FES_2_L"]
                                  ]

# Input module channels for IMUs
in_addresses.mpu_addr = collections.namedtuple('mpu_addr','desc vals')
in_addresses.mpu_addr.desc = ["IMU1: Right Leg",
                             "IMU2: Left Leg"
                             ]
in_addresses.mpu_addr.channels = [["R_acc_x","R_acc_y","R_acc_z"],
                                  ["L_acc_x","L_acc_y","L_acc_z"]
                                 ]
in_addresses.mpu_addr.vals = [0x68, 0X69]

in_addresses.mpu_reg_addr = collections.namedtuple('mpu_reg_addr', 'desc
vals')
#Lines 1-2 : Self-Test
#Line  3   : Operation Mode Conditions
#Lines 4-5 : Configuration
#Lines 6-12: Data registers
in_addresses.mpu_reg_addr.desc =
["TEST_GYRO_X","TEST_GYRO_Y","TEST_GYRO_Z",
```

```
                                    "TEST_ACC_X","TEST_ACC_Y","TEST_ACC_Z",
                                    "SMPLRT_DIV","CONFIG","PWR_MGMT_1",
                                    "GYRO_CONFIG","ACC_CONFIG","ACC_CONFIG2",
                                    "WOM_THR","FIFO_EN","INT_ENABLE",
                                    "ACCEL_XOUT_H","ACCEL_XOUT_L",
                                    "ACCEL_YOUT_H","ACCEL_YOUT_L",
                                    "ACCEL_ZOUT_H","ACCEL_ZOUT_L",
                                    "TEMP_OUT_H","TEMP_OUT_L",
                                    "GYRO_XOUT_H","GYRO_XOUT_L",
                                    "GYRO_YOUT_H","GYRO_YOUT_L",
                                    "GYRO_ZOUT_H","GYRO_ZOUT_L",
                                    ]
in_addresses.mpu_reg_addr.vals = [0x00,0x01,0x02,
                                  0x0D,0x0E,0x0F,
                                  0x19,0x1A,0x6B,
                                  0x1B,0x1C,0x1D,
                                  0x1F,0x23,0x38,
                                  0x3B,0x3C,
                                  0x3D,0x3E,
                                  0x3F,0x40,
                                  0x41,0x42,
                                  0x43,0x44,
                                  0x45,0x46,
                                  0x47,0x48]

in_addresses.magn_addr = collections.namedtuple('magn_addr','desc vals')
in_addresses.magn_addr.desc = ["AK8963 (Magnetometer)"]
in_addresses.magn_addr.vals = [0x0C]

in_addresses.magn_reg_addr = collections.namedtuple('magn_reg_addr', 'desc
vals')
#Line 1 : Status/Control
#Line 2 : Self-Test
#Line 3 : Data
in_addresses.magn_reg_addr.desc = ["AK8963_ST1","AK8963_ST2","AK8963_CNTL",
                                   "ASTC",
                                   "HXL","HXH","HYL","HYH","HZL","HZH"]
in_addresses.magn_reg_addr.vals = [0x02,0x09,0x0A,
                                   0x0C,
                                   0x03,0x04,0x05,0x06,0x07,0x08]

# Input module channels for Clock (not used due to address conflict)
#in_addresses.clock_addr = collections.namedtuple('clock_addr', 'desc
vals')
#in_addresses.clock_addr.desc= ["Nothing"]
#in_addresses.clock_addr.vals= [0x00]

# Input module channels OpenBCI Board (EEG)
# EEG channel labels not defined, since they are included in
objects.openbci.info["eeg_names"]
#
in_addresses.eeg = '/dev/ttyUSB0'
in_addresses.eeg_board_id = 2
# in_addresses.eeg_channels = ["FP1","FP2","C3","C4","T5","T6","O1","O2",
                              # "F7","F8","FC1","FC2","T3","T4","CP1","CP2",
                              # "Head_Accx","Head_Accy","Head_Accz"
                             # ]
in_addresses.eeg_extra_channels = ["Accel_x","Accel_y","Accel_z"]+\
```

```
["Unknown","D11_dig","D12_dig","D13_dig","D12_anmode","D17","D18",
                                        "D11_ana","D12_ana","D13_ana"]+\
                                        ["timestamp","marker"]

# Return output
return in_addresses

################################################
Output addresses
def out_addr():
out_addresses = collections.namedtuple('out_addresses', 'fes wheelchair')
out_addresses.fes = "/dev/ttyUSB1"
out_addresses.wheelchair= ""

# Return output
return out_addresses

################################################
Parameters for configuration of external devices
def exp_params():
exp = collections.namedtuple('exp',['ads_rate','ads_gain',
    'fchoice','smplrt_div','dlpfcfg',
    'fes_baudrate','fes_bytesize','fes_parity','fes_stopbits',
    'fes_timeout','fes_write_timeout','fes_xonxoff',
    'fes_rtscts','fes_dsrdtr'])

# ADS
exp.ads_rate = 3300
exp.ads_gain = 2

# MPU
#  Sampling Rate
#   -1KHz : Fchoice=3     / DLPFCFG=1,2,3,4,5,6 / SMPLRT_DIV=0
#   -8KHz : Fchoice=3     / DLPFCFG=0,7         / SMPLRT_DIV=0
#   -32KHz: Fchoice=0,1,2 / DLPFCFG=-           / SMPLRT_DIV=0
exp.fchoice = 2
exp.smplrt_div = 0
#exp.dlpfcfg = 0

# OpenBCI
#  -markiv_eeg_scale: Multiply uVolts_per_count to convert the
channels_data to uVolts
#  -markiv_aux_scale: Multiply accel_G_per_count to convert the aux_data to
G
exp.markiv_timeout = 4
exp.markiv_eeg_scale = (4500000)/24/(2**23-1) #uV/count
exp.markiv_aux_scale = 0.002 / (2**4)
exp.markiv_num_samples = 45000
exp.read_board_samples = 1 # how many samples to read with
read_current_board_data
exp.markiv_file = ''
#exp.markiv_eeg_chans = 16
#exp.markiv_aux_chans = 3
#exp.markiv_srate = 125
#exp.markiv_datafile = "file://%filename%:w"

# FES
exp.fes_baudrate=115200
exp.fes_bytesize=serial.EIGHTBITS
```

```
exp.fes_parity="N"
exp.fes_stopbits=serial.STOPBITS_ONE
exp.fes_timeout=0
exp.fes_write_timeout=None
exp.fes_xonxoff=False
exp.fes_rtscts=False
exp.fes_dsrdtr=False

# Clock

# Return output
return exp

##################################################
Preprocessing parameters
def preprocess_params():
preprocess_pars = collections.namedtuple('preprocess_pars',
                                         ['force_scalein','force_scaleout',
                                         'flex_scalein','flex_scaleout'])

# EEG

# Head IMU

# EMG

# ECG

# Body IMU

# Pressure
#Rescale inputs to force min and max values
#-input range: force_scalein
#-output range: force_scaleout
preprocess_pars.force_scalein = [1.2,2.2]
preprocess_pars.force_scaleout = [0,3]

# Flex
#Rescale inputs to force min and max values
#-input range: flex_scalein
#-output range: flex_scaleout
preprocess_pars.flex_scalein = [0.7,1.1]
preprocess_pars.flex_scaleout = [0,2]

# Wheelchair

# FES

# Return output
return preprocess_pars

##################################################
Parameters for feature extraction
def feature_params():
featextract = collections.namedtuple('featextract', '')

# EEG

# EMG
```

```
# IMU (Body)

# Pressure

# Flex

# IMU (Wheelchair)

# Return output
return featextract

################################################
Parameters for feature processing and generation of outputs
def process_params():
process_pars = collections.namedtuple('process_pars',
['wheelchair_states','gait_states',

'thresholds_gait','thresholds_eeg',
                                                    'thresholds_emg',

'fes_current','fes_pulse_width',
                                                    'channel_mode'])

# Wheelchair out_states
process_pars.wheelchair_states = ['Forward', 'Backward', 'Right', 'Left',
'Stop']

# Gait states
process_pars.gait_states =
['stance','midstance_l','heelstrike_r','toeoff_l',
               'midstance_r','heelstrike_l','toeoff_r']

# Gait thresholds
force_heel_th = 1.8
force_toe_th = 1.8
flex_knee_th = 1
imu_x = 1
imu_y = 1
imu_z = 1 # Irrelevant based on sensor placement
process_pars.thresholds_gait = [force_heel_th, force_toe_th, flex_knee_th,
imu_x, imu_y]

# EEG thresholds
eeg_alpha = 1
eeg_beta = 1
eeg_gamma = 1
eeg_delta = 1
eeg_theta = 1
process_pars.thresholds_eeg =
[eeg_delta,eeg_theta,eeg_alpha,eeg_beta,eeg_gamma]

# EMG thresholds
emg_tibialis = 1
emg_gastrocnemius = 1
process_pars.thresholds_emg = [emg_tibialis,emg_gastrocnemius]

# FES command
# -channel_mode: 0 - channel list mode (initialisation)
```

```
#                1 - channel list mode (update)
#                2 - channel list mode (stop)
#                3 - single pulse mode
# -current     : pulse amplitude in mA
# -pulse_width : pulse duration in ms
process_pars.channel_mode = 3
process_pars.fes_current = 120
process_pars.fes_pulse_width = 200

# Wheelchair command

# Return output
return process_pars

################################################
Feedback parameters
def feedback_params():
feedback_pars = collections.namedtuple('feedback_pars',
    ['none'])

# Feedback displays

# FES command vs FES output

# FES output vs Movement

# Wheelchair command vs Movement

# Return output
return feedback_pars

################################################
Protocol stimuli
task_type = 1: Walking
task_type = 2: Wheelchair
def protocol_stimuli(task_type):
protocol_pars = collections.namedtuple('protocol_pars',
    ['stimuli'])

if task_type == 1:
    #protocol_pars.task = 'walking'
    protocol_pars.stimuli = [['Real right step', 'Real left step', 'Real 4
steps'],
                             ['Imaginary right step', 'Imaginary left
step', 'Imaginary 4 steps']]

else:
    #protocol_pars.task = 'wheelchair'
    protocol_pars.stimuli = [['Stop','Forward','Backward','Right','Left']]

# Return output
return protocol_pars
```

A.3 Αρχείο inspire_ dataprocess.py

```
Import modules
from datetime import datetime
import os, csv, time, numpy as np
import inspire_interface, inspire_objects, inspire_datarecord from
inspire_filters
```

```
import filter_live_chunk
from inspire_features import extract_band_power
from inspire_utils import print_performance
from gpiozero import LED
#Tell Python we plugged the LED into GPIO Pin 17
wheelchair_signal = LED(17)
################################################
Handle real-time processing
Note: Use 1st command for threading, 2nd for multiprocessing
def data_process(objects, pars, process, feedback, task_type,
guide,filt,shared_powers): #def data_process(objects, channels, pars,
process, task_type, guide, conn):
# Define global variable
global gpiodata

if task_type == 1:

    # Gait thresholds
    states = pars.process.gait_states
    sensor_thres = pars.process.thresholds_gait
    eeg_thres = pars.process.thresholds_eeg
    emg_thres = pars.process.thresholds_emg
    gait_thres = [emg_thres[0],emg_thres[1],emg_thres[0],emg_thres[1],

sensor_thres[1],sensor_thres[0],sensor_thres[1],sensor_thres[0],
                  sensor_thres[2],sensor_thres[2],
                  sensor_thres[3],sensor_thres[4],0,
                  sensor_thres[3],sensor_thres[4],0]

    # FES parameters
    channel_mode = pars.process.channel_mode
    pulse_width = pars.process.fes_pulse_width
    current = pars.process.fes_current
    serial = objects.fes

    # Wait for data recording to begin
    time.sleep(2)

    # Begin time monitoring (to avoid continuous output commands)
    start=time.time()

    while inspire_datarecord.recording:

        # Get latest measurements
        # Last 10 GPIO inputs
        # Last 130 OpenBCI measurements
        #print(inspire_datarecord.gpiodata[-11:-1])
        gpio = np.mean(np.delete(np.array(inspire_datarecord.gpiodata[-11:-
1]),[0,1],1),axis = 0)
        #eeg_data = objects.openbci.get_current_board_data(130)
        #objects.openbci.insert_marker(1) #insert integer as a marker

        # State Classification
        thres_comp =np.sign(np.subtract(gpio[4:8],gait_thres[4:8]))
#comparison with thresholds

if thres_comp == [1,1,1,1]: #stance
channels = []
elif thres_comp == [-1,-1,1,1]: #midstance_l
```

```
channels = [3,4]
elif thres_comp == [1,-1,-1,1]: #heelstrike_r
channels = [3]
elif thres_comp == [1,1,-1,1]: #toeoff_l
channels = [1,2,4]
elif thres_comp == [1,1,-1,-1]: #midstance_r
channels = [1,2]
elif thres_comp == [-1,1,1,-1]: #heelstrike_l
channels = [1]
elif thres_comp == [-1,1,1,1]: #toeoff_r
channels = [2,3,4]
else:
channels = []
      if thres_comp[0]==-1 & thres_comp[1]==-1: #midstance_l
            channels = [3,4]
        elif thres_comp[1]==-1 & thres_comp[2]==-1: #heelstrike_r
            channels = [3]
        elif thres_comp[2]==-1 & thres_comp[1]==1 & thres_comp[3]==1:
#toeoff_l
            channels = [1,2,4]
        elif thres_comp[2]==-1 & thres_comp[3]==-1: #midstance_r
            channels = [1,2]
        elif thres_comp[0]==-1 & thres_comp[3]==-1: #heelstrike_l
            channels = [1]
        elif thres_comp[0]==-1 & thres_comp[1]==1 & thres_comp[3]==1:
#toeoff_r
            channels = [2,3,4]
        else:
            channels = []

        # Send command to FES
        if time.time()-start > 3:
            for c in channels:
                fes_send(serial,channel_mode,pulse_width,current,c)
            start=time.time()

elif task_type == 2:
    # Wheelchair parameters

    # Create the memory variable right before the loop starts
    my_history_buffer = []
    while inspire_datarecord.recording:
    # 1. Grab raw data from the headset
        raw_data = objects.openbci.get_current_board_data(130)

        # --- THE GATEKEEPER ---
        # ONLY do math if we actually have a full 130 samples!
        if raw_data.shape[1] >= 130:
            start_time = time.perf_counter()

            # 2. Clean it perfectly using your live filter

            if filt == True:
                # Clean the data and update memory
                clean_data, my_history_buffer =
filter_live_chunk(raw_data[1], my_history_buffer)

            else:
                # Bypass the filter entirely!
```

```
                # Just grab the raw fuzzy data and pass it forward.
                clean_data = raw_data[1]


            # 3. Extract features one at a time
            theta_power = extract_band_power(clean_data, 5.0, 8.0)
            beta_power = extract_band_power(clean_data, 12.0, 25.0)

            ###set the perameters to tranfer them and then display them###
            shared_powers["theta"] = theta_power
            shared_powers["beta"] = beta_power

            ############################visualization



            try:
                # 1. Calculate the total combined energy of both bands
                total_power = theta_power + beta_power

                # 2. Prevent "Divide by Zero" if the headset disconnects
                if total_power > 0:
                    theta_percent = (theta_power / total_power) * 100
                    beta_percent = (beta_power / total_power) * 100
                else:
                    theta_percent = 0
                    beta_percent = 0

                # 3. Apply the Percentages to the Bar Heights
                inspire_interface.bars[0].set_height(theta_percent)
                inspire_interface.bars[1].set_height(beta_percent)

                # 4. Show the RAW data in the text labels so you still know
the absolute strength
                inspire_interface.theta_text.set_text(f"{theta_power:.1f}")
                inspire_interface.beta_text.set_text(f"{beta_power:.1f}")

            except Exception as e:
                print(f"⚠️ UI Update Error: {e}")
            # ----------------------------------------



            print(f"Theta: {theta_power:.2f} | Beta: {beta_power:.2f}")

            ##############################################
            # 4. Function for output (Wheelchair decision)

                # If Theta is double Beta, output = 1. Otherwise, 0.
            if theta_power > (beta_power * 2):
                command = 1
            else:
                command = 0

            # Send the command using your consistent function structure
            wheelchair_send(command, wheelchair_signal)
            ############################################################
```

```
            #checking the systems perfomance
            delay, cpu,temp = print_performance(start_time)
            time.sleep(0.5)
        else:
            # If we have less than 130 samples, just wait 10 milliseconds
and check again
            time.sleep(0.01)



##################################################Choice outputs
Send command to FES device
def fes_send(serial,channel_mode,pulse_width,current,channel_num):
# Calculate checksum
checksum = (channel_num-1+pulse_width+current) % 32

# Convert to binary
channel_mode_bin = format(channel_mode, '#04b')[2:4]
channel_num_bin = format(channel_num-1, '#05b')[2:5]
pulse_width_bin = format(pulse_width, '#011b')[2:11]
current_bin = format(current, '#09b')[2:9]
checksum_bin = format(checksum, '#07b')[2:7]

# Command bytes
command_bytes = []
command_bytes += ['1' + channel_mode_bin + checksum_bin]
command_bytes +=  ['0' + channel_num_bin + '00' + pulse_width_bin[0:2]]
command_bytes +=  ['0' + pulse_width_bin[2:11]]
command_bytes +=  ['0' + current_bin]

# Convert to integer
fes_command=[]
for b in command_bytes:
    fes_command += [int(b,2)]

# Send command to device
serial.write(fes_command)

#FOR DISPLAY PURPOSES after run: #import inspire_dataprocess
#inspire_dataprocess.fes_send(objects.fes,pars.process.channel_mode,pars.pr
ocess.fes_pulse_width,pars.process.fes_current,1)
def wheelchair_send(drive_command, pin_object): """ Sends a command to the
wheelchair motor controller (or LED). drive_command: 1 (Drive/ON) or 0
(Stop/OFF) """ # 1. Log the command (optional, but great for debugging) #
print(f"Sending Wheelchair Command: {drive_command}")
# 2. Execute the hardware output
if drive_command == 1:

    print('Power on')
    pin_object.on()   # Turn pin HIGH (3.3V)
else:
    print('Power off')
    pin_object.off()  # Turn pin LOW (0V)



################################################
```

A.4 Αρχείο inspire_datarecord.py

```
Import modules
from datetime import datetime
import os, csv, time
import inspire_interface, inspire_objects
################################################
Handle real-time recording (DEPRECATED)
def record2(objects,channels, pars, process): global data
data = [["inc","timestamp"] + channels.labels] # +
[[0]*len(channels["labels"])]
inc = 0

# Fetch preprocess parameters
force_scalein = pars.preprocess.force_scalein
force_scaleout = pars.preprocess.force_scaleout
flex_scalein = pars.preprocess.flex_scalein
flex_scaleout = pars.preprocess.flex_scaleout

# Start Recording
print("Recording started")
while recording:

    # Initialise vector
    #m = [0]*len(channels.ads['labels'])
    m= []

    # Get measurements
    for i in range(len(channels.ads['labels'])):

        #m[i] = channels.ads['obj'][i].voltage
        m = m + [channels.ads['obj'][i].voltage]

        # Rescale inputs
        if 'Force' in channels.ads['labels'][i]:
            m[i] = force_scaleout[0] + (m[i]-
force_scalein[0])*force_scaleout[1]/\
                (force_scalein[1]-force_scalein[0])
        elif 'Flex' in channels.ads['labels'][i]:
            m[i] = flex_scaleout[0] + (m[i]-
flex_scalein[0])*flex_scaleout[1]/\
                (flex_scalein[1]-flex_scalein[0])

    for i in range(len(objects.mpu)):
        #print(m)
        #print(inspire_objects.mpu6050_conv(objects.mpu[i],pars))
        m=m+inspire_objects.mpu6050_conv(objects.mpu[i],pars)

    #print(m)

    # Update feedback display
    for i in range(len(inspire_interface.bars)):
       inspire_interface.bars[i].set_height(m[i])

    #
    if any(channels.openbci['labels']):
        a = inspire_objects.eeg*pars.exp.markiv_eeg_scale
```

```
        b = inspire_objects.aux*pars.exp.markiv_aux_scale
        m = m + a.tolist() + b.tolist()

    # Append values to data table
    data=data+[[inc,datetime.now()]+m]
    inc = inc+1
return

################################################
Handle real-time recording
Note: Use 1st command for recording variable for threading, 2nd for
multiprocessing
def record(objects, pars, process): #def record(objects, pars, process,
conn):
# Define global variable
global gpiodata
global boarddata

# Fetch preprocess parameters
force_scalein = pars.preprocess.force_scalein
force_scaleout = pars.preprocess.force_scaleout
flex_scalein = pars.preprocess.flex_scalein
flex_scaleout = pars.preprocess.flex_scaleout

# Begin openbci recording
if objects.openbci != []:
    objects.openbci.start_stream(num_samples = pars.exp.markiv_num_samples,
\
                        streamer_params="file://" +
pars.config.openbcidatafile + ":w")
#time.sleep(0.5)

# GPIO recording initialisation
gpiodata = []
inc = -1
ads_num = len(objects.ads)
mpu_num = len(objects.mpu)

# Start GPIO Recording
print("Recording started")
while recording: #This for threading, use the below command for
multiprocessing
#while recording.value:
#while inc<=1000: #for testing

    # Initialise vector
    inc+=1
    m = [inc,datetime.now()]

    # Get ADS data
    for i in range(ads_num):
        for j in range(len(objects.ads[i].channels)):
            m += [objects.ads[i].channels[j].voltage]

    # Rescale inputs
        if 'Force' in objects.ads[i].chan_labels:
            m[i] = force_scaleout[0] + (m[i]-
force_scalein[0])*force_scaleout[1]/\
               (force_scalein[1]-force_scalein[0])
```

```
        elif 'Flex' in objects.ads[i].chan_labels:
            m[i] = flex_scaleout[0] + (m[i]-
flex_scalein[0])*flex_scaleout[1]/\
                   (flex_scalein[1]-flex_scalein[0])

    # Get IMU data
    for i in range(len(objects.mpu)):
        m += inspire_objects.mpu6050_conv(objects.mpu[i],pars)

    # Get OpenBCI data
    #print(objects.openbci.get_current_board_data(1)) #print last n
measurements
    #objects.openbci.insert_marker(1) #insert integer as a marker
    #print(m)

    # Send data to connection (only if multiprocessing is used)
    #conn.send(m)

    # Append data to accumulating variable for storage
    gpiodata += [m]
    '''
    # Update feedback display

    for i in range(len(inspire_interface.bars)):
        inspire_interface.bars[i].set_height(m[i+2])
    '''
#Send data through the connection (only for multiprocessing)
#conn.close
#conn.send(None)
#conn.send(gpiodata)
#conn.close (HERE OR 2 LINES ABOVE?)

# Test variable content
#print(gpiodata)
return

################################################
Create output file with session data
def save_sessiondata(pars,objects,gpiodata): #print(gpiodata) # Save GPIO
data with open(pars.config.gpiodatafile,'w') as f: write =
csv.writer(f,delimiter = ";") write.writerow(pars.session.gpio_headers)
write.writerows(gpiodata) f.close print("GPIO data saved in" +
pars.config.gpiodatafile)
# Add headers to OpenBCI data file
if objects.openbci!=[]:
    with open(pars.config.openbcidatafile,'r') as f:
        read = csv.reader(f,delimiter = "\t")
        lines = list(read)
    flat_headers = []
    for item in pars.session.openbci_headers:
        flat_headers.extend(str(item).split(','))
    lines.insert(0, flat_headers)
#lines.insert(0,pars.session.openbci_headers) change the , to ;

    with open(pars.config.openbcidatafile,'w') as f:
        write = csv.writer(f,delimiter = ";")
        write.writerows(lines)
        f.close
    print("OpenBCI data saved in" + pars.config.openbcidatafile)
```

```
else:
    print("OpenBCI data not saved (no stream available)")

'''
Create output file for the with session filtered data
def save_filteredeegdata(pars,objects,gpiodata):
# Add headers to OpenBCI data file
import inspire_filters
if objects.openbci!=[]:
    with open(pars.config.openbcidatafile,'r') as f:
        read = csv.reader(f,delimiter = ";")
        lines = list(read)
    inspire_filters

    with open(pars.config.fopenbcidatafile,'w') as f:
        write = csv.writer(f,delimiter = ";")
        write.writerows(lines)
        f.close
    print("OpenBCI filtered data saved in" + pars.config.fopenbcidatafile)
else:
    print("OpenBCI filtered data not saved (no stream available)")

'''
Create output file with session filtered data
def save_filteredeegdata(pars, objects, gpiodata):
import inspire_filters
import csv

if objects.openbci != []:
    # --- PHASE 1: Extract Pure Numbers ---
    with open(pars.config.openbcidatafile, 'r') as f:
        read = csv.reader(f, delimiter=";")
        lines = list(read)

    headers = lines[0]
    raw_numbers = lines[1:]

    # --- PHASE 2: Apply the Filter ---
    filtered_numbers = inspire_filters.apply_frequency_filters(raw_numbers)

    # --- PHASE 3: Rejoin the Headers and Data ---
    final_lines = [headers] + filtered_numbers

    # --- PHASE 4: Save to the new file ---
    with open(pars.config.fopenbcidatafile, 'w') as f:
        write = csv.writer(f, delimiter=";")
        write.writerows(final_lines)

    print("OpenBCI filtered data saved in " + pars.config.fopenbcidatafile)

else:
    print("OpenBCI filtered data not saved (no stream available)")
```

A.5 Αρχείο inspire_features.py

```
import numpy as np
```

```
from brainflow.data_filter import DataFilter, WindowOperations

def extract_band_power(clean_chunk, f_min, f_max, sampling_rate=125):
    """
    Takes a clean chunk of EEG data and extracts the power of
    a specific frequency band defined by f_min and f_max.
    """
    # 1. Convert to BrainFlow's required format
    data_array = np.array(clean_chunk, dtype=np.float64)

    # 2. Setup FFT parameters for a 130-sample chunk
    nfft = 128
    overlap = 64

    try:
        # 3. Calculate Power Spectral Density (Welch's Method)
        psd = DataFilter.get_psd_welch(
            data_array,
            nfft,
            overlap,
            int(sampling_rate), # <--- THE FIX: Force it to be a pure
integer
            WindowOperations.HANNING.value
        )

        # 4. Extract your requested frequency band
        power = DataFilter.get_band_power(psd, float(f_min), float(f_max))

        return power

    except Exception as e:
        print(f"Feature Extraction Error: {e}")
        return 0.0
```

A.6 Αρχείο inspire_filters.py

import numpy as np from scipy.signal

import butter, filtfilt, iirnotch

from brainflow.data_filter import DataFilter, FilterTypes

def apply_frequency_filters(number_data): """ Takes raw CSV numbers, applies Notch and Bandpass filters to the EEG columns, and returns the cleaned numbers ready to be saved. """ print('Applying filters ')

```
# 1. Convert the text data into a NumPy grid of numbers so we can do math
data = np.array(number_data, dtype=float)

# ========================================
# SETTINGS (Super easy to change)
# ========================================
fs = 125.0              # Sampling Rate (125 Hz for 16-channel Cyton+Daisy)

# Notch Filter Settings (Removes electrical hum)
```

```
notch_freq = 50.0       # 50Hz for Europe/Most of world
quality_factor = 30.0   # How "sharp" the notch cut is

# Bandpass Filter Settings (Keeps only actual brainwaves)
lowcut = 3.0            # Bottom frequency (3 Hz removes slow eye blinks)
highcut = 40.0          # Top frequency (40 Hz removes fast muscle tension)

# Columns to filter
eeg_start_col = 1       # Fp1 is at index 1
eeg_end_col = 16        # P4 is at index 16
# ========================================


# 2. Create the Math Filters using SciPy
nyq = 0.5 * fs # Nyquist Frequency (Required for the math)

# Build the Notch 50hz filter
b_notch, a_notch = iirnotch(notch_freq, quality_factor, fs)

#non standart 25hz filter
b_notch25, a_notch25 = iirnotch(w0=25.0, Q=52.0, fs=125)

# Build the Bandpass (4th order Butterworth)
low = lowcut / nyq
high = highcut / nyq
b_band, a_band = butter(4, [low, high], btype='band')


# 3. Apply the filters strictly to the EEG columns
# We use a loop to go through columns 1 to 16, one by one
for col in range(eeg_start_col, eeg_end_col + 1):

    # Grab the entire vertical column of data for just this one brainwave
channel
    channel_data = data[:, col]

    # Scrub it with the Notch filter
    channel_data = filtfilt(b_notch, a_notch, channel_data)
    # Scrub it with the 25 Hz Notch filter
    channel_data = filtfilt(b_notch25, a_notch25, channel_data)

    # Scrub it with the Bandpass filter
    channel_data = filtfilt(b_band, a_band, channel_data)

    # Put the sparkling clean data back into the grid
    data[:, col] = channel_data

crop_amount = 250 #2 sec

# Only crop if the file is actually long enough (e.g., longer than 4
seconds)
if len(data) > (crop_amount * 2):
    data = data[crop_amount:-crop_amount]

# 4. Convert the NumPy grid back into a standard Python list
#
```

```
print('Finished filtering')
return data.tolist()
```

''' def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125): """ Takes a raw chunk of EEG data, uses a rolling buffer to prevent edge effects, applies 50Hz Notch and 1-50Hz Bandpass filters, and returns the perfectly clean chunk. """ # 1. Add new data to history memory history_buffer.extend(new_chunk)

```
# 2. Keep memory strictly at 600 samples (roughly 5 seconds)
if len(history_buffer) > 600:
    history_buffer = history_buffer[-600:]

# 3. Convert to NumPy array (BrainFlow requires float64)
np_buffer = np.array(history_buffer, dtype=np.float64)

# 4. Apply filters to the ENTIRE memory
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 48.0, 52.0, 3, FilterTypes.BUTTERWORTH.value,
0
)
DataFilter.perform_bandpass(
    np_buffer, sampling_rate, 3.0, 50.0, 4, FilterTypes.BUTTERWORTH.value,
0
)

# 5. Snip off ONLY the newest chunk
chunk_length = len(new_chunk)
clean_chunk = np_buffer[-chunk_length:]

# Return both the clean data AND the updated memory
return clean_chunk.tolist(), history_buffer
```

''' def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125): history_buffer.extend(new_chunk)

```
if len(history_buffer) > 600:
    history_buffer = history_buffer[-600:]

np_buffer = np.array(history_buffer, dtype=np.float64)

# 1. 50Hz Notch (Start: 48.0 Hz, Stop: 52.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 48.0, 52.0, 2, FilterTypes.BUTTERWORTH.value,
0
)

# 2. 25Hz Hardware Spike Notch (Start: 24.5 Hz, Stop: 25.5 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 24.5, 25.5, 1, FilterTypes.BUTTERWORTH.value,
0
)

# 3. Main Bandpass (Start: 5.0 Hz, Stop: 45.0 Hz)
DataFilter.perform_bandpass(
    np_buffer, sampling_rate, 5.0, 45.0, 5, FilterTypes.BUTTERWORTH.value,
```

```
0
)

chunk_length = len(new_chunk)
clean_chunk = np_buffer[-chunk_length:]

return clean_chunk.tolist(), history_buffer
```

A.7 Αρχείο inspire_interface.py

```
Import modules

import pygame,os,pylab,time
import matplotlib matplotlib.use("Agg")
import matplotlib.backends.backend_agg as agg
import inspire_datarecord

################################################

Interface Initialisation

def interface(objects, pars, msg): global fig global fig_X global bars
global ax #global canvas

# Initialize pygame display
os.environ['SDL_VIDEO_CENTERED'] = '1' #bring pygame window to center of
screen
pygame.init() #begin pygame
pygame.display.set_caption(pars.gui_title) #set title for pygame window
screen = pygame.display.set_mode((pars.gui_width,pars.gui_height)) #create
pygame window

# Display messages and update display position for next object
pos = pars.gui_startpos
font = pygame.font.Font(pars.gui_font, pars.gui_fontsize)
for i in range(len(msg)):
    text = font.render(msg[i],True,pars.gui_fontcolor,pars.gui_fontbgr)
    screen.blit(text, pos)
    pygame.display.update()
    pos = [pos[0],pos[1]+20]
'''old display
# Create figure for live feedback
fig = pylab.figure(figsize=pars.fig_size,dpi=pars.fig_dpi)
ax = fig.subplots(1)
ax.set_ylim([pars.ylim_min,pars.ylim_max])

labels = []
for i in range(len(objects.ads)):
    labels = labels + objects.ads[i].chan_labels
for i in range(len(objects.mpu)):
    labels = labels + objects.mpu[i].chan_labels


for i in range(len(objects.openbci)):

labels = labels + objects.openbci[i].chan_labels

#fig_X = channels.ads["labels"]+channels.mpu["labels"]
fig_X = labels
```

```
# bars updated by inspire_datarecord.record
bars = ax.bar(fig_X,[0]*len(fig_X))

canvas = agg.FigureCanvasAgg(fig)
canvas.draw()
'''
#####make a figure with the eeg features
# Create figure for live feedback
fig = pylab.figure(figsize=pars.fig_size,dpi=pars.fig_dpi)
ax = fig.subplots(1)




# ---> START OF THE REPLACEMENT <---
ax.set_ylim([0, 100])
fig_X = ['Theta ', 'Beta ']

global bars, theta_text, beta_text

# Start  at a small number (like 10.0) so we don't divide by zero later
theta_peak = 10.0
beta_peak = 10.0
peak_ceiling = 100.0
bars = ax.bar(fig_X, [0, 0], color=['#f1c40f', '#2ecc71'])

# Create the text objects (Positioned at X=0 and X=1)
# ha='center' keeps them perfectly horizontally aligned inside the poles
theta_text = ax.text(0, 0, '0.0', ha='center', va='center', color='black',
fontsize=16, fontweight='bold', zorder=10)
beta_text = ax.text(1, 0, '0.0', ha='center', va='center', color='black',
fontsize=16, fontweight='bold', zorder=10)
# ---> END OF THE REPLACEMENT <---


canvas = agg.FigureCanvasAgg(fig)
canvas.draw()



return pygame,screen,pos,canvas


##################################################

''' def ask_filter_preference(pygame,screen,pars,pos): font =
pygame.font.Font(pars.gui_font, pars.gui_fontsize) msg = "Do you want to
apply filters? Press Y for Yes, N for No" text =
font.render(msg,True,pars.gui_fontcolor,pars.gui_fontbgr) screen.blit(text,
pos) pos = [pos[0],pos[1]+20] pygame.display.update()

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
           return False,pos

        if event.type == pygame.KEYDOWN:
```

```
            if event.key == pygame.K_y:
                return True,pos
            if event.key == pygame.K_n:
                return False,pos


'''

#########

User confirmation request (via pygame GUI) for proceeding with the main
recording

def confirm(pygame,screen,pars,pos,filt):

font = pygame.font.Font(pars.gui_font, pars.gui_fontsize)

msg = "Press  F if you want  to aplly filters or N if not"
text = font.render(msg,True,pars.gui_fontcolor,pars.gui_fontbgr)
screen.blit(text, pos)
pos = [pos[0],pos[1]+20]
pygame.display.update()

loop1=True
while loop1:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:
            filt=False
            loop1=False

        if event.type == pygame.KEYDOWN:

            if event.key == pygame.K_f:
                filt=True
                loop1=False
                msg = "Filters will be applied"
                text =
font.render(msg,True,pars.gui_fontcolor,pars.gui_fontbgr)
                screen.blit(text, pos)
                pos = [pos[0],pos[1]+20]
                pygame.display.update()
            if event.key == pygame.K_n:
                filt=False
                loop1=False

                msg = "Filters wont be applied"
                text =
font.render(msg,True,pars.gui_fontcolor,pars.gui_fontbgr)
                screen.blit(text, pos)
                pos = [pos[0],pos[1]+20]
                pygame.display.update()



msg = "Press Enter to begin, Esc to abort"
text = font.render(msg,True,pars.gui_fontcolor,pars.gui_fontbgr)
screen.blit(text, pos)
```

```
pos = [pos[0],pos[1]+20]
pygame.display.update()

while True:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:
           return False,pos,filt

        if event.type == pygame.KEYDOWN:

            if event.key == pygame.K_ESCAPE:
                return False,pos,filt

            if event.key == pygame.K_RETURN:
                return True,pos,filt


################################################

Monitor GUI state

Close GUI when a specific key press is detected

Note: Use 1st command for recording variable for threading, 2nd for
multiprocessing

#def monitor(): def monitor(pygame): print("Display started") display =
True while display: time.sleep(2) for event in pygame.event.get(): if
event.type == pygame.QUIT: display = False break if event.type ==
pygame.KEYDOWN: if event.key == pygame.K_ESCAPE: display = False break

inspire_datarecord.recording = False
#inspire_datarecord.recording.value = False

pygame.display.quit()
pygame.quit()
return


################################################

Refresh live feedback display

NOTE: Use 1st command for recording variable for threading, 2nd for
multiprocessing

def feedback_display(pygame,screen,pars,canvas): #def
feedback_display(pygame,screen,pars,canvas,conn):

# Refresh figures
while True:
    canvas.draw()
    canvas.flush_events()
    renderer = canvas.get_renderer()
    raw_data = renderer.tostring_argb()
    size = canvas.get_width_height()
    surf = pygame.image.fromstring(raw_data, size, "ARGB")
```

```
    # Get data for feedback
    # NOTE: Only if multiprocessing is used instead of threading
    #m = conn.recv()
    #for i in range(len(bars)):
    #    bars[i].set_height(m[i+2])

    # Refresh feedback figure
    # NOTE: Check if display is open first to prevent error when the gui is
closed
    #       Alternatively, use a "try-except" block
    if pygame.display.get_init():
        screen.blit(surf, (pars.config.gui_width-
pars.config.fig_size[1]*pars.config.fig_dpi,0))

        #pygame.display.update() # use this command for threading, the
below for multiprocessing#pygame.display.flip() # use this command for
threading, the below for multiprocessing


        #pygame.value.flip()
    else:
        return


################################################
```

A.8 Αρχείο inspire_objects.py

```
import collections, serial, time
import board, busio
import smbus
import adafruit_ads1x15.ads1015 as ADS from adafruit_ads1x15.analog_in
import AnalogIn from adafruit_ads1x15.ads1x15
import Mode from pyOpenBCI
import OpenBCICyton
#from pylsl import StreamInfo, StreamOutlet# DEPRECATED
from brainflow.board_shim import BoardShim, BrainFlowInputParams,
LogLevels, BoardIds
import numpy as np
################################################
Create bus
global bus, i2c bus = smbus.SMBus(1) i2c = busio.I2C(board.SCL, board.SDA)
################################################
Call functions for creating all objects
def objects(params):
# Create list of strings for storing warning messages for display
msg = []

# Initialise tuple with objects
inspire_objects = collections.namedtuple('inspire_objects', 'i2c ads mpu
openbci fes clock')

# Create objects
inspire_objects.ads,msg = obj_ads(params, msg)
inspire_objects.mpu,msg = obj_mpu(params, msg)
inspire_objects.openbci,msg = obj_openbci(params, msg)
inspire_objects.clock,msg = obj_clock(params, msg) #will be left empty, not
```

```
created due to address conflict
inspire_objects.fes,msg = obj_fes(params,msg)

# Return output
return inspire_objects,msg

################################################
#Create ads objects def obj_ads(params, msg):
# Create ADS objects and check availability
ads = [0]*len(params.inaddr.ads_addr.vals)
not_found = []
for i in range (len(ads)):

    try:
        ads[i] = ADS.ADS1015(i2c,address=params.inaddr.ads_addr.vals[i])
        ads[i].chan_labels = params.inaddr.ads_addr.channels[i]
    except:
        not_found = not_found + [i]
        msg=msg+["Warning: Object '"+params.inaddr.ads_addr.desc[i]+"' in
address "+str(hex(params.inaddr.ads_addr.vals[i]))+" cannot be found"]

# Remove undetected objects
for i in range(len(not_found)-1,-1,-1):
    del ads[not_found[i]]

# Set ADS parameters
for i in range(len(ads)):
    ads[i].gain = params.exp.ads_gain
    ads[i].data_rate = params.exp.ads_rate
    ads[i].mode = Mode.CONTINUOUS

# Set input channels
# AnalogIn(ads, ADS.P0, ADS.P1) for differential reading
pins = [0,1,2,3]
for i in range(len(ads)):
    ads[i].channels = [0]*4
    for j in range(4):
        ads[i].channels[j] = AnalogIn(ads[i], pins[j])

# Return output
return ads,msg

################################################
Class for IMU object
Used by obj_mpu to create IMU object
class MPU_OBJ: addr = desc = [] gyro_sens = accel_sens = []
################################################
Create mpu objects
def obj_mpu(params, msg):
# Initialise IMU objects
mpu = [0]*len(params.inaddr.mpu_addr.vals)


# Check sensor availability
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('PWR_MGMT_1')]

not_found = []
found = []
```

```
for i in range (len(params.inaddr.mpu_addr.vals)):
    mpu[i] = MPU_OBJ()
    try:
        bus.read_byte_data(params.inaddr.mpu_addr.vals[i],reg_addr)
        found = found + [i]
    except:
        not_found = not_found + [i]
        msg=msg+["Warning: Object '"+params.inaddr.mpu_addr.desc[i]+"' in
address "\
                 +str(hex(params.inaddr.mpu_addr.vals[i]))+" cannot be
found"]

# Perform self-test and create objects for modules with a successful result
for i in found:

    test_fail = False
    #obj_mpu_selftest(params,i2c_address)

    if test_fail:
        not_found = not_found + [i]
        #found.remove(i)
        msg=msg+["Warning: Failed self-test for object
'"+params.inaddr.mpu_addr.desc[i]\
                 +"' in address "+str(hex(params.inaddr.mpu_addr.vals[i]))]
    else:
        # Create objects
        mpu[i].chan_labels = params.inaddr.mpu_addr.channels[i]
        mpu[i].addr = params.inaddr.mpu_addr.vals[i]
        mpu[i].gyro_sens,mpu[i].accel_sens = MPU6050_start(params,\

params.inaddr.mpu_addr.vals[i])

# Remove objects that were not detected or failed to pass self-test
for i in range(len(not_found)-1,-1,-1):
    del mpu[not_found[i]]

# Return output
return mpu,msg

##################################################
Initialisation of a single MPU sensor (accelerometer & gyroscope)
def MPU6050_start(params,addr):
# Set sampling rate
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('SMPLRT_DIV')]
bus.write_byte_data(addr, reg_addr, params.exp.smplrt_div)
time.sleep(0.1)

# Reset all sensors
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('PWR_MGMT_1')]
bus.write_byte_data(addr,reg_addr,0x00)
time.sleep(0.1)

# Power management and crystal settings
bus.write_byte_data(addr, reg_addr, 0x01)
time.sleep(0.1)
```

```
# Write to Configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('CONFIG')]
bus.write_byte_data(addr, reg_addr, 0)
time.sleep(0.1)

# Write to Gyro configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('GYRO_CONFIG')]
gyro_config_sel = [0b00000,0b010000,0b10000,0b11000] # byte registers
gyro_config_vals = [250.0,500.0,1000.0,2000.0] # degrees/sec
gyro_indx = 0
bus.write_byte_data(addr, reg_addr, int(gyro_config_sel[gyro_indx]))
time.sleep(0.1)

# Write to Accel configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACC_CONFIG')]
accel_config_sel = [0b00000,0b01000,0b10000,0b11000] # byte registers
accel_config_vals = [2.0,4.0,8.0,16.0] # g (g = 9.81 m/s^2)
accel_indx = 0
bus.write_byte_data(addr, reg_addr, int(accel_config_sel[accel_indx]))
time.sleep(0.1)

# Interrupt register (related to overflow of data [FIFO])
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('INT_ENABLE')]
bus.write_byte_data(addr, reg_addr, 1)
time.sleep(0.1)

# Return output
return gyro_config_vals[gyro_indx],accel_config_vals[accel_indx]

################################################
Read data from MPU sensor (accelerometer & gyroscope) - Called by
mpu6050_conv
def read_raw_bits(addr, reg):
# Read accel and gyro values
high = bus.read_byte_data(addr, reg)
low = bus.read_byte_data(addr, reg+1)

# Combine higha and low for unsigned bit value
value = ((high << 8) | low)

# Convert to +- value
if(value > 32768):
    value -= 65536

# Return output
return value

################################################
Read data from MPU sensor (accelerometer & gyroscope) - Called by
mpu6050_conv
def mpu6050_conv(obj, params):
# Register addresses (gyro not used)
reg_addr_x1 = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_XOUT_H')]
reg_addr_y1 = params.inaddr.mpu_reg_addr.vals[\
```

```
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_YOUT_H')]
reg_addr_z1 = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_ZOUT_H')]
#reg_addr_x2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_XOUT_H')]
#reg_addr_y2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_YOUT_H')]
#reg_addr_z2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_ZOUT_H')]

# Raw acceleration bits
acc_x = read_raw_bits(obj.addr,reg_addr_x1)
acc_y = read_raw_bits(obj.addr,reg_addr_y1)
acc_z = read_raw_bits(obj.addr,reg_addr_z1)

# Raw gyroscope bits (gyro not used)
#gyro_x = read_raw_bits(obj.addr,reg_addr_x2)
#gyro_y = read_raw_bits(obj.addr,reg_addr_y2)
#gyro_z = read_raw_bits(obj.addr,reg_addr_z2)

# Raw temperature bits (temperature measurement not used)
#t_val = read_raw_bits(TEMP_OUT_H)

# Convert to acceleration in g
a_x = (acc_x/(2.0**15.0))*obj.accel_sens
a_y = (acc_y/(2.0**15.0))*obj.accel_sens
a_z = (acc_z/(2.0**15.0))*obj.accel_sens

# Convert to gyro dps (gyro not used)
#w_x = (gyro_x/(2.0**15.0))*obj.gyro_sens
#w_y = (gyro_y/(2.0**15.0))*obj.gyro_sens
#w_z = (gyro_z/(2.0**15.0))*obj.gyro_sens

# Convert to temperature measurement (not used)
#temp = ((t_val)/333.87)+21.0

# Store values in single list
chan_vals = [a_x, a_y, a_z]

# Return output
return chan_vals

################################################
Perform a self-test on MPU9050
def obj_mpu_selftest(params,i2c_address):
# Set digital low pass filter (DLPF) to 2 for registers
# -26 (CONFIG)
# -29 (ACCEL_CONFIG 2)
dlfp = 2

# Set full scale range for registers
# -27 (GYRO_CONFIG): 250dps
#-28 (ACCEL_CONFIG): 2g
acc_fs_sel = 00

# Set sampling rate to 1kHz

# Average 200 readings for gyro & acc
# The averaged values will be the LSB of:
```

```
# GX_OS, GY_OS, GZ_OS, AX_OS, AY_OS, AZ_OS

# Set USR_Reg: (1Bh) Gyro_Config, gdrive_axisCTST [0-2] to b111 to enable
Self-Test

# Set USR_Reg: (1Ch) Accel_Config, AX/Y/Z_ST_EN [0-2] to b111 to enable
Self-Test

time.sleep(0.02)

# Read & average values again

# Calculate response
#mpu = [0]*3
#return mpu,msg

################################################
Class for OpenBCI object (DEPRECATED)
class OPENBCI_OBJ: obj = chan_labels = []
################################################
#Functions for streaming OpenBCI data (DEPRECATED) def
lsl_streamers(sample): global eeg, aux eeg = np.array(sample.channels_data)
aux = np.array(sample.aux_data)
def board_stream(board_bci): board_bci.start_stream(lsl_streamers)
################################################
Create OpenBCI EEG object
def obj_openbci(params, msg):
# Set logging
#BoardShim.set_log_file(params.config.logfile)
BoardShim.disable_board_logger()

# Set BrainFlow arguments
board_pars = BrainFlowInputParams()
board_pars.timeout = params.exp.markiv_timeout
board_pars.file = params.exp.markiv_file
board_pars.serial_port = params.inaddr.eeg
board_id = params.inaddr.eeg_board_id

# Initialise object
board = BoardShim(board_id, board_pars)

# Get board info
#  - accel_channels      / BoardShim.get_accel_channels(board_id)
#  - analog_channels     / BoardShim.get_analog_channels(board_id)
#  - ecg_channels
#  - eeg_channels        / BoardShim.get_eeg_channels(board_id)
#  - eeg_names           / BoardShim.get_eeg_names(board_id)
#  - emg_channels
#  - eog_channels
#  - marker_channel      / BoardShim.get_marker_channel(board_id)
#  - name                / BoardShim.get_device_name(board_id)
#  - num_rows            / BoardShim.get_num_rows(board_id)
#  - other_channels      / BoardShim.get_other_channels(board_id)
#  - package_num_channel / BoardShim.get_package_num_channel(board_id)
#  - sampling_rate       / BoardShim.get_sampling_rate(board_id)
#  - timestamp_channel   / BoardShim.get_timestamp_channel(board_id)
board.info = board.get_board_descr(board_id)
#board.info.keys()
```

```
# Delete reduntant fields
redundant_fields = ["ecg_channels","emg_channels","eog_channels"]
for field in redundant_fields:
    del board.info[field]

# Prepare session
try:
    board.prepare_session()
except Exception as e:
    msg=msg+["Warning - Unable to establish OpenBCI connection ('" +
params.inaddr.eeg + "')"]
    msg=msg+[" -" + str(e)]
    board = []

# Return output
return board,msg

DEPRECATED
def obj_openbci2(params, msg):
openbci = [OPENBCI_OBJ()]
try:
    openbci[0].obj = OpenBCICyton(port=params.inaddr.eeg, daisy=True)
    openbci[0].chan_labels = params.inaddr.eeg_channels
    found = True
except:
    msg=msg+["Warning: OpenBCI Dongle not found in '" + params.inaddr.eeg +
"'"]
    openbci = []
    found = False

#Check connectivity
if found:
    try:
        openbci_port = openbci[0].obj.find_port()
    except:
        msg=msg+["Warning: OpenBCI Recording device not found"]
        openbci = []
        found = False

if found:
    # Declare Mark IV data stream
    info_eeg = StreamInfo('OpenBCIEEG', 'EEG', \
                          params.exp.markiv_eeg_chans,
params.exp.markiv_frate, \
                          'float32', 'OpenBCIEEG')
    outlet_eeg = StreamOutlet(info_eeg)

    # Declare Accelerometer data stream
    info_aux = StreamInfo('OpenBCIAUX', 'AUX', \
                          params.exp.markiv_aux_chans,
params.exp.markiv_frate, \
                          'float32', 'OpenBCIAUX')
    outlet_aux = StreamOutlet(info_aux)

# Return output
return openbci,msg

################################################
```

```
Create FES object
def obj_fes(params, msg):
error = False
try:
    fes_out = serial.Serial(port=params.outaddr.fes,
baudrate=params.exp.fes_baudrate,
                            bytesize=params.exp.fes_bytesize,
parity=params.exp.fes_parity,
                            stopbits=params.exp.fes_stopbits,
                            timeout=params.exp.fes_timeout,
                            write_timeout=params.exp.fes_write_timeout,
                            xonxoff=params.exp.fes_xonxoff,
                            rtscts=params.exp.fes_rtscts,
dsrdtr=params.exp.fes_dsrdtr)
except:
    fes_out =[]
    msg=msg+["Warning: Object 'FES' not connected in port
'"+params.outaddr.fes+"'"]

if fes_out != []:
    fes_out.flush()
    fes_out.write([0xc0])
    time.sleep(0.05) # Delay is necessary, or else read will return nothing
    response = fes_out.read()
    #print(response)
    if response != b'\x81':
        fes_out =[]
        msg=msg+["Warning: Object 'FES' in port '"+params.outaddr.fes+"'
has not acknowledged connection"]

# Return output
return fes_out,msg

################################################
Create clock object (not used due to address conflict with MPU)
def obj_clock(params, msg): clock = []
# Return output
return clock, msg

################################################
Define data headers
def data_headers(objects,pars):
# GPIO data headers
ads_count = len(objects.ads)
mpu_count = len(objects.mpu)
if mpu_count + ads_count != 0:
    gpiodata = ["inc","timestamp"]
    for i in range(ads_count):
        gpiodata += objects.ads[i].chan_labels
    for i in range(mpu_count):
        gpiodata += objects.mpu[i].chan_labels

else:
    gpiodata = []
#gpiodata = [gpiodata]

# OpenBCI board data headers
if objects.openbci != []:
    boarddata =
```

```
["inc"]+[objects.openbci.info["eeg_names"]]+pars.inaddr.eeg_extra_channels
    # \
                # ["Accel_x","Accel_y","Accel_z"]+\
                #
["Unknown","D11_dig","D12_dig","D13_dig","D12_anmode","D17","D18",
                 # "D11_ana","D12_ana","D13_ana"]+\
                # ["timestamp","marker"]
else:
    boarddata = []
#boarddata = [boarddata]

# Return output
return gpiodata,boarddata
```

A.9 Αρχείο inspire_threads.py

```
import collections, serial, time
import board, busio
import smbus
import adafruit_ads1x15.ads1015 as ADS from adafruit_ads1x15.analog_in
import AnalogIn from adafruit_ads1x15.ads1x15
import Mode from pyOpenBCI
import OpenBCICyton #from pylsl import StreamInfo, StreamOutlet# DEPRECATED
from brainflow.board_shim
import BoardShim, BrainFlowInputParams, LogLevels, BoardIds
import numpy as np
################################################
Create bus
global bus, i2c bus = smbus.SMBus(1) i2c = busio.I2C(board.SCL, board.SDA)
################################################
Call functions for creating all objects
def objects(params):
# Create list of strings for storing warning messages for display
msg = []

# Initialise tuple with objects
inspire_objects = collections.namedtuple('inspire_objects', 'i2c ads mpu
openbci fes clock')

# Create objects
inspire_objects.ads,msg = obj_ads(params, msg)
inspire_objects.mpu,msg = obj_mpu(params, msg)
inspire_objects.openbci,msg = obj_openbci(params, msg)
inspire_objects.clock,msg = obj_clock(params, msg) #will be left empty, not
created due to address conflict
inspire_objects.fes,msg = obj_fes(params,msg)

# Return output
return inspire_objects,msg

################################################
#Create ads objects def obj_ads(params, msg):
# Create ADS objects and check availability
ads = [0]*len(params.inaddr.ads_addr.vals)
not_found = []
for i in range (len(ads)):

    try:
```

```
        ads[i] = ADS.ADS1015(i2c,address=params.inaddr.ads_addr.vals[i])
        ads[i].chan_labels = params.inaddr.ads_addr.channels[i]
    except:
        not_found = not_found + [i]
        msg=msg+["Warning: Object '"+params.inaddr.ads_addr.desc[i]+"' in
address "+str(hex(params.inaddr.ads_addr.vals[i]))+" cannot be found"]

# Remove undetected objects
for i in range(len(not_found)-1,-1,-1):
    del ads[not_found[i]]

# Set ADS parameters
for i in range(len(ads)):
    ads[i].gain = params.exp.ads_gain
    ads[i].data_rate = params.exp.ads_rate
    ads[i].mode = Mode.CONTINUOUS

# Set input channels
# AnalogIn(ads, ADS.P0, ADS.P1) for differential reading
pins = [0,1,2,3]
for i in range(len(ads)):
    ads[i].channels = [0]*4
    for j in range(4):
        ads[i].channels[j] = AnalogIn(ads[i], pins[j])

# Return output
return ads,msg

##################################################
Class for IMU object
Used by obj_mpu to create IMU object
class MPU_OBJ: addr = desc = [] gyro_sens = accel_sens = []
##################################################
Create mpu objects
def obj_mpu(params, msg):
# Initialise IMU objects
mpu = [0]*len(params.inaddr.mpu_addr.vals)


# Check sensor availability
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('PWR_MGMT_1')]

not_found = []
found = []

for i in range (len(params.inaddr.mpu_addr.vals)):
    mpu[i] = MPU_OBJ()
    try:
        bus.read_byte_data(params.inaddr.mpu_addr.vals[i],reg_addr)
        found = found + [i]
    except:
        not_found = not_found + [i]
        msg=msg+["Warning: Object '"+params.inaddr.mpu_addr.desc[i]+"' in
address "\
                 +str(hex(params.inaddr.mpu_addr.vals[i]))+" cannot be
found"]

# Perform self-test and create objects for modules with a successful result
```

```
for i in found:

    test_fail = False
    #obj_mpu_selftest(params,i2c_address)

    if test_fail:
        not_found = not_found + [i]
        #found.remove(i)
        msg=msg+["Warning: Failed self-test for object
'"+params.inaddr.mpu_addr.desc[i]\
                 +"' in address "+str(hex(params.inaddr.mpu_addr.vals[i]))]
    else:
        # Create objects
        mpu[i].chan_labels = params.inaddr.mpu_addr.channels[i]
        mpu[i].addr = params.inaddr.mpu_addr.vals[i]
        mpu[i].gyro_sens,mpu[i].accel_sens = MPU6050_start(params,\

params.inaddr.mpu_addr.vals[i])

# Remove objects that were not detected or failed to pass self-test
for i in range(len(not_found)-1,-1,-1):
    del mpu[not_found[i]]

# Return output
return mpu,msg

##################################################
Initialisation of a single MPU sensor (accelerometer & gyroscope)
def MPU6050_start(params,addr):
# Set sampling rate
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('SMPLRT_DIV')]
bus.write_byte_data(addr, reg_addr, params.exp.smplrt_div)
time.sleep(0.1)

# Reset all sensors
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('PWR_MGMT_1')]
bus.write_byte_data(addr,reg_addr,0x00)
time.sleep(0.1)

# Power management and crystal settings
bus.write_byte_data(addr, reg_addr, 0x01)
time.sleep(0.1)

# Write to Configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('CONFIG')]
bus.write_byte_data(addr, reg_addr, 0)
time.sleep(0.1)

# Write to Gyro configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('GYRO_CONFIG')]
gyro_config_sel = [0b00000,0b010000,0b10000,0b11000] # byte registers
gyro_config_vals = [250.0,500.0,1000.0,2000.0] # degrees/sec
gyro_indx = 0
bus.write_byte_data(addr, reg_addr, int(gyro_config_sel[gyro_indx]))
time.sleep(0.1)
```

```
# Write to Accel configuration register
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACC_CONFIG')]
accel_config_sel = [0b00000,0b01000,0b10000,0b11000] # byte registers
accel_config_vals = [2.0,4.0,8.0,16.0] # g (g = 9.81 m/s^2)
accel_indx = 0
bus.write_byte_data(addr, reg_addr, int(accel_config_sel[accel_indx]))
time.sleep(0.1)

# Interrupt register (related to overflow of data [FIFO])
reg_addr = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('INT_ENABLE')]
bus.write_byte_data(addr, reg_addr, 1)
time.sleep(0.1)

# Return output
return gyro_config_vals[gyro_indx],accel_config_vals[accel_indx]

#################################################
Read data from MPU sensor (accelerometer & gyroscope) - Called by
mpu6050_conv
def read_raw_bits(addr, reg):
# Read accel and gyro values
high = bus.read_byte_data(addr, reg)
low = bus.read_byte_data(addr, reg+1)

# Combine higha and low for unsigned bit value
value = ((high << 8) | low)

# Convert to +- value
if(value > 32768):
    value -= 65536

# Return output
return value

#################################################
Read data from MPU sensor (accelerometer & gyroscope) - Called by
mpu6050_conv
def mpu6050_conv(obj, params):
# Register addresses (gyro not used)
reg_addr_x1 = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_XOUT_H')]
reg_addr_y1 = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_YOUT_H')]
reg_addr_z1 = params.inaddr.mpu_reg_addr.vals[\
    params.inaddr.mpu_reg_addr.desc.index('ACCEL_ZOUT_H')]
#reg_addr_x2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_XOUT_H')]
#reg_addr_y2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_YOUT_H')]
#reg_addr_z2 = params.inaddr.mpu_reg_addr.vals[\
#    params.inaddr.mpu_reg_addr.desc.index('GYRO_ZOUT_H')]

# Raw acceleration bits
acc_x = read_raw_bits(obj.addr,reg_addr_x1)
acc_y = read_raw_bits(obj.addr,reg_addr_y1)
acc_z = read_raw_bits(obj.addr,reg_addr_z1)
```

```
# Raw gyroscope bits (gyro not used)
#gyro_x = read_raw_bits(obj.addr,reg_addr_x2)
#gyro_y = read_raw_bits(obj.addr,reg_addr_y2)
#gyro_z = read_raw_bits(obj.addr,reg_addr_z2)

# Raw temperature bits (temperature measurement not used)
#t_val = read_raw_bits(TEMP_OUT_H)

# Convert to acceleration in g
a_x = (acc_x/(2.0**15.0))*obj.accel_sens
a_y = (acc_y/(2.0**15.0))*obj.accel_sens
a_z = (acc_z/(2.0**15.0))*obj.accel_sens

# Convert to gyro dps (gyro not used)
#w_x = (gyro_x/(2.0**15.0))*obj.gyro_sens
#w_y = (gyro_y/(2.0**15.0))*obj.gyro_sens
#w_z = (gyro_z/(2.0**15.0))*obj.gyro_sens

# Convert to temperature measurement (not used)
#temp = ((t_val)/333.87)+21.0

# Store values in single list
chan_vals = [a_x, a_y, a_z]

# Return output
return chan_vals

#####################################################
Perform a self-test on MPU9050
def obj_mpu_selftest(params,i2c_address):
# Set digital low pass filter (DLPF) to 2 for registers
# -26 (CONFIG)
# -29 (ACCEL_CONFIG 2)
dlfp = 2

# Set full scale range for registers
# -27 (GYRO_CONFIG): 250dps
#-28 (ACCEL_CONFIG): 2g
acc_fs_sel = 00

# Set sampling rate to 1kHz

# Average 200 readings for gyro & acc
# The averaged values will be the LSB of:
# GX_OS, GY_OS, GZ_OS, AX_OS, AY_OS, AZ_OS

# Set USR_Reg: (1Bh) Gyro_Config, gdrive_axisCTST [0-2] to b111 to enable
Self-Test

# Set USR_Reg: (1Ch) Accel_Config, AX/Y/Z_ST_EN [0-2] to b111 to enable
Self-Test

time.sleep(0.02)

# Read & average values again

# Calculate response
#mpu = [0]*3
```

```
#return mpu,msg

################################################
Class for OpenBCI object (DEPRECATED)
class OPENBCI_OBJ: obj = chan_labels = []
################################################
#Functions for streaming OpenBCI data (DEPRECATED) def
lsl_streamers(sample): global eeg, aux eeg = np.array(sample.channels_data)
aux = np.array(sample.aux_data)
def board_stream(board_bci): board_bci.start_stream(lsl_streamers)
################################################
Create OpenBCI EEG object
def obj_openbci(params, msg):
# Set logging
#BoardShim.set_log_file(params.config.logfile)
BoardShim.disable_board_logger()

# Set BrainFlow arguments
board_pars = BrainFlowInputParams()
board_pars.timeout = params.exp.markiv_timeout
board_pars.file = params.exp.markiv_file
board_pars.serial_port = params.inaddr.eeg
board_id = params.inaddr.eeg_board_id

# Initialise object
board = BoardShim(board_id, board_pars)

# Get board info
#  - accel_channels      / BoardShim.get_accel_channels(board_id)
#  - analog_channels     / BoardShim.get_analog_channels(board_id)
#  - ecg_channels
#  - eeg_channels        / BoardShim.get_eeg_channels(board_id)
#  - eeg_names           / BoardShim.get_eeg_names(board_id)
#  - emg_channels
#  - eog_channels
#  - marker_channel      / BoardShim.get_marker_channel(board_id)
#  - name                / BoardShim.get_device_name(board_id)
#  - num_rows            / BoardShim.get_num_rows(board_id)
#  - other_channels      / BoardShim.get_other_channels(board_id)
#  - package_num_channel / BoardShim.get_package_num_channel(board_id)
#  - sampling_rate       / BoardShim.get_sampling_rate(board_id)
#  - timestamp_channel   / BoardShim.get_timestamp_channel(board_id)
board.info = board.get_board_descr(board_id)
#board.info.keys()


# Delete reduntant fields
redundant_fields = ["ecg_channels","emg_channels","eog_channels"]
for field in redundant_fields:
    del board.info[field]

# Prepare session
try:
    board.prepare_session()
except Exception as e:
    msg=msg+["Warning - Unable to establish OpenBCI connection ('" +
params.inaddr.eeg + "')"]
    msg=msg+[" -" + str(e)]
    board = []
```

```
# Return output
return board,msg

DEPRECATED
def obj_openbci2(params, msg):
openbci = [OPENBCI_OBJ()]
try:
    openbci[0].obj = OpenBCICyton(port=params.inaddr.eeg, daisy=True)
    openbci[0].chan_labels = params.inaddr.eeg_channels
    found = True
except:
    msg=msg+["Warning: OpenBCI Dongle not found in '" + params.inaddr.eeg +
"'"]
    openbci = []
    found = False

#Check connectivity
if found:
    try:
        openbci_port = openbci[0].obj.find_port()
    except:
        msg=msg+["Warning: OpenBCI Recording device not found"]
        openbci = []
        found = False

if found:
    # Declare Mark IV data stream
    info_eeg = StreamInfo('OpenBCIEEG', 'EEG', \
                          params.exp.markiv_eeg_chans,
params.exp.markiv_frate, \
                          'float32', 'OpenBCIEEG')
    outlet_eeg = StreamOutlet(info_eeg)

    # Declare Accelerometer data stream
    info_aux = StreamInfo('OpenBCIAUX', 'AUX', \
                          params.exp.markiv_aux_chans,
params.exp.markiv_frate, \
                          'float32', 'OpenBCIAUX')
    outlet_aux = StreamOutlet(info_aux)

# Return output
return openbci,msg

##################################################
Create FES object
def obj_fes(params, msg):
error = False
try:
    fes_out = serial.Serial(port=params.outaddr.fes,
baudrate=params.exp.fes_baudrate,
                            bytesize=params.exp.fes_bytesize,
parity=params.exp.fes_parity,
                            stopbits=params.exp.fes_stopbits,
                            timeout=params.exp.fes_timeout,
                            write_timeout=params.exp.fes_write_timeout,
                            xonxoff=params.exp.fes_xonxoff,
                            rtscts=params.exp.fes_rtscts,
dsrdtr=params.exp.fes_dsrdtr)
```

```
except:
    fes_out =[]
    msg=msg+["Warning: Object 'FES' not connected in port
'"+params.outaddr.fes+"'"]

if fes_out != []:
    fes_out.flush()
    fes_out.write([0xc0])
    time.sleep(0.05) # Delay is necessary, or else read will return nothing
    response = fes_out.read()
    #print(response)
    if response != b'\x81':
        fes_out =[]
        msg=msg+["Warning: Object 'FES' in port '"+params.outaddr.fes+"'
has not acknowledged connection"]

# Return output
return fes_out,msg

##################################################
Create clock object (not used due to address conflict with MPU)
def obj_clock(params, msg): clock = []
# Return output
return clock, msg

##################################################
Define data headers
def data_headers(objects,pars):
# GPIO data headers
ads_count = len(objects.ads)
mpu_count = len(objects.mpu)
if mpu_count + ads_count != 0:
    gpiodata = ["inc","timestamp"]
    for i in range(ads_count):
        gpiodata += objects.ads[i].chan_labels
    for i in range(mpu_count):
        gpiodata += objects.mpu[i].chan_labels

else:
    gpiodata = []
#gpiodata = [gpiodata]

# OpenBCI board data headers
if objects.openbci != []:
    boarddata =
["inc"]+[objects.openbci.info["eeg_names"]]+pars.inaddr.eeg_extra_channels
    # \
                # ["Accel_x","Accel_y","Accel_z"]+\
                #
["Unknown","D11_dig","D12_dig","D13_dig","D12_anmode","D17","D18",
                 # "D11_ana","D12_ana","D13_ana"]+\
                # ["timestamp","marker"]
else:
    boarddata = []
#boarddata = [boarddata]

# Return output
return gpiodata,boarddata
```

A.10 Αρχείο inspire_utils.py

```
'''
import time
import psutil

def print_performance(start_time): """ Stops the stopwatch, calculates
execution delay and CPU usage, and prints a clean dashboard to the
terminal. """ # 1. Stop the stopwatch end_time = time.perf_counter()

# 2. Calculate the metrics
time_delay_ms = (end_time - start_time) * 1000
cpu_usage = psutil.cpu_percent()

# 3. Print the dashboard
print(f"⏱️ Delay: {time_delay_ms:.2f} ms | 💻 CPU: {cpu_usage}% ")

# Return the numbers just in case you want to use them later
return time_delay_ms, cpu_usage


'''
import time
import psutil

def print_performance(start_time): """ Stops the stopwatch, calculates
execution delay, CPU usage, and Pi temperature with absolutely minimal
processing overhead. """ # 1. Stop the stopwatch instantly end_time =
time.perf_counter() time_delay_ms = (end_time - start_time) * 1000

# 2. CPU Usage (psutil is written in C, so it is already very fast)
cpu_usage = psutil.cpu_percent()

# 3. Ultra-Fast Temperature Read (Zero subprocess overhead)
try:
    # Read directly from the Pi's thermal zone file
    with open("/sys/class/thermal/thermal_zone0/temp", "r") as f:
        # The Pi stores temp in millidegrees (e.g., 48000 = 48.0 C)
        temperature = float(f.readline()) / 1000.0
        temp_display = f"{temperature:.1f}°C"
except Exception:
    # Fallback if running on Windows/Mac import csv

import matplotlib.pyplot as plt

import numpy as np

from scipy.signal import welch



import csv

import matplotlib.pyplot as plt

import numpy as np
```

```
from scipy.signal import welch


# --- CONFIGURATION ---

sampling_rate = 125.0  # OpenBCI Daisy rate

num_channels = 16      # Total number of EEG channels to average


print("--- INSPiRE Global Frequency Domain Visualizer ---")

session_num = input("Enter session number : ")


base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'

raw_filepath = base_path + 'openbcirecordings.csv'


#filt_filepath = base_path + 'filteredopenbcirecordings.csv'


#for testing

filt_filepath = base_path + 'upgraded_filtered_recordings.csv'


def load_all_channels(filename):

    """Loads columns 1 through 16 into a massive math grid."""

    data = []

    try:

        with open(filename, 'r') as f:

            reader = csv.reader(f, delimiter=';')

            next(reader) # Skip headers

            for row in reader:

                try:

                    # Grab columns 1 through 16

                    row_vals = [float(row[i]) for i in range(1, 17)]

                    data.append(row_vals)
```

```
                except (ValueError, IndexError):
                    pass # Ignore text markers or incomplete rows
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Calculating Average Power Spectral Density...")

    # Transpose data so it is shaped like (Channels, Samples)
    raw_data = raw_data.T
    filt_data = filt_data.T


    # --- 2. The Math Trick ---
    # Scipy can calculate all 16 channels at the exact same time (axis=1)
    f_raw, psd_raw_all = welch(raw_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')
    f_filt, psd_filt_all = welch(filt_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')

    # Now we average the 16 spectra together vertically (axis=0)
    psd_raw_avg = np.mean(psd_raw_all, axis=0)
    psd_filt_avg = np.mean(psd_filt_all, axis=0)
```

```
    # --- 3. Draw the Graph ---

    print("Drawing graph...")

    plt.figure(figsize=(12, 7))

    xlim_max = 60.0



    # Plot the averaged spectra

    plt.semilogy(f_raw, psd_raw_avg, label='Raw Global Average',
color='red', alpha=0.5)

    plt.semilogy(f_filt, psd_filt_avg, label='Filtered Global Average',
color='blue', linewidth=2)



    # Highlight Brainwave Bands

    plt.axvspan(1, 4, color='gray', alpha=0.1) # Delta

    plt.axvspan(4, 8, color='gray', alpha=0.15) # Theta

    plt.axvspan(8, 13, color='gold', alpha=0.2, label='Alpha Band
(Relaxation)')

    plt.axvspan(13, 30, color='blue', alpha=0.05, label='Beta Band
(Focus)')



    # Chart Styling

    plt.title("Global EEG Spectrum (Average of all 16 Channels)")

    plt.xlabel("Frequency (Hz)")

    plt.ylabel("Average Power (dB/Hz)")

    plt.xlim(0, xlim_max)

    plt.grid(True, which="both", ls="-", alpha=0.5)

    plt.legend()



    plt.show()

import csv

import matplotlib.pyplot as plt

import numpy as np
```

```
from scipy.signal import welch


import csv

import matplotlib.pyplot as plt

import numpy as np

from scipy.signal import welch


# --- CONFIGURATION ---

sampling_rate = 125.0  # OpenBCI Daisy rate

num_channels = 16      # Total number of EEG channels to average


print("--- INSPiRE Global Frequency Domain Visualizer ---")

session_num = input("Enter session number : ")


base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'

raw_filepath = base_path + 'openbcirecordings.csv'


#filt_filepath = base_path + 'filteredopenbcirecordings.csv'


#for testing

filt_filepath = base_path + 'upgraded_filtered_recordings.csv'


def load_all_channels(filename):

    """Loads columns 1 through 16 into a massive math grid."""

    data = []

    try:

        with open(filename, 'r') as f:

            reader = csv.reader(f, delimiter=';')
```

```
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass # Ignore text markers or incomplete rows
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Calculating Average Power Spectral Density...")

    # Transpose data so it is shaped like (Channels, Samples)
    raw_data = raw_data.T
    filt_data = filt_data.T



    # --- 2. The Math Trick ---
    # Scipy can calculate all 16 channels at the exact same time (axis=1)
    f_raw, psd_raw_all = welch(raw_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')
```

```
    f_filt, psd_filt_all = welch(filt_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')



    # Now we average the 16 spectra together vertically (axis=0)

    psd_raw_avg = np.mean(psd_raw_all, axis=0)

    psd_filt_avg = np.mean(psd_filt_all, axis=0)



    # --- 3. Draw the Graph ---

    print("Drawing graph...")

    plt.figure(figsize=(12, 7))

    xlim_max = 60.0



    # Plot the averaged spectra

    plt.semilogy(f_raw, psd_raw_avg, label='Raw Global Average',
color='red', alpha=0.5)

    plt.semilogy(f_filt, psd_filt_avg, label='Filtered Global Average',
color='blue', linewidth=2)



    # Highlight Brainwave Bands

    plt.axvspan(1, 4, color='gray', alpha=0.1) # Delta

    plt.axvspan(4, 8, color='gray', alpha=0.15) # Theta

    plt.axvspan(8, 13, color='gold', alpha=0.2, label='Alpha Band
(Relaxation)')

    plt.axvspan(13, 30, color='blue', alpha=0.05, label='Beta Band
(Focus)')



    # Chart Styling

    plt.title("Global EEG Spectrum (Average of all 16 Channels)")

    plt.xlabel("Frequency (Hz)")

    plt.ylabel("Average Power (dB/Hz)")

    plt.xlim(0, xlim_max)

    plt.grid(True, which="both", ls="-", alpha=0.5)
```

```
    plt.legend()


    plt.show()
import csv
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import welch


import csv
import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import welch


# --- CONFIGURATION ---
sampling_rate = 125.0  # OpenBCI Daisy rate
num_channels = 16      # Total number of EEG channels to average


print("--- INSPiRE Global Frequency Domain Visualizer ---")
session_num = input("Enter session number : ")


base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'


#filt_filepath = base_path + 'filteredopenbcirecordings.csv'


#for testing
filt_filepath = base_path + 'upgraded_filtered_recordings.csv'


def load_all_channels(filename):
```

```
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass # Ignore text markers or incomplete rows
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Calculating Average Power Spectral Density...")

    # Transpose data so it is shaped like (Channels, Samples)
    raw_data = raw_data.T
    filt_data = filt_data.T
```

```
    # --- 2. The Math Trick ---

    # Scipy can calculate all 16 channels at the exact same time (axis=1)

    f_raw, psd_raw_all = welch(raw_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')

    f_filt, psd_filt_all = welch(filt_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')



    # Now we average the 16 spectra together vertically (axis=0)

    psd_raw_avg = np.mean(psd_raw_all, axis=0)

    psd_filt_avg = np.mean(psd_filt_all, axis=0)



    # --- 3. Draw the Graph ---

    print("Drawing graph...")

    plt.figure(figsize=(12, 7))

    xlim_max = 60.0



    # Plot the averaged spectra

    plt.semilogy(f_raw, psd_raw_avg, label='Raw Global Average',
color='red', alpha=0.5)

    plt.semilogy(f_filt, psd_filt_avg, label='Filtered Global Average',
color='blue', linewidth=2)



    # Highlight Brainwave Bands

    plt.axvspan(1, 4, color='gray', alpha=0.1) # Delta

    plt.axvspan(4, 8, color='gray', alpha=0.15) # Theta

    plt.axvspan(8, 13, color='gold', alpha=0.2, label='Alpha Band
(Relaxation)')

    plt.axvspan(13, 30, color='blue', alpha=0.05, label='Beta Band
(Focus)')



    # Chart Styling
```

```
    plt.title("Global EEG Spectrum (Average of all 16 Channels)")

    plt.xlabel("Frequency (Hz)")

    plt.ylabel("Average Power (dB/Hz)")

    plt.xlim(0, xlim_max)

    plt.grid(True, which="both", ls="-", alpha=0.5)

    plt.legend()



    plt.show()


    temperature = 0.0
    temp_display = "N/A"

# 4. Print the dashboard
print(f"⏱️ Delay: {time_delay_ms:.2f} ms | 💻 CPU: {cpu_usage}% | 🌡️ Temp:
{temp_display}")

return time_delay_ms, cpu_usage, temperature
```

A.11 Αρχείο inspiret_freguency.py

```
import csv

import matplotlib.pyplot as plt

import numpy as np

from scipy.signal import welch


import csv

import matplotlib.pyplot as plt

import numpy as np

from scipy.signal import welch


# --- CONFIGURATION ---

sampling_rate = 125.0  # OpenBCI Daisy rate

num_channels = 16      # Total number of EEG channels to average


print("--- INSPiRE Global Frequency Domain Visualizer ---")
```

```
session_num = input("Enter session number : ")


base_path = f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'


#filt_filepath = base_path + 'filteredopenbcirecordings.csv'


#for testing
filt_filepath = base_path + 'upgraded_filtered_recordings.csv'


def load_all_channels(filename):
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass # Ignore text markers or incomplete rows
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)
```

```
# --- 1. Load the Data ---

print("Loading all 16 channels of data...")

raw_data = load_all_channels(raw_filepath)

filt_data = load_all_channels(filt_filepath)


if raw_data is not None and filt_data is not None:

    print("Calculating Average Power Spectral Density...")


    # Transpose data so it is shaped like (Channels, Samples)

    raw_data = raw_data.T

    filt_data = filt_data.T




    # --- 2. The Math Trick ---

    # Scipy can calculate all 16 channels at the exact same time (axis=1)

    f_raw, psd_raw_all = welch(raw_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')

    f_filt, psd_filt_all = welch(filt_data, fs=sampling_rate, nperseg=250,
axis=1, detrend='linear')


    # Now we average the 16 spectra together vertically (axis=0)

    psd_raw_avg = np.mean(psd_raw_all, axis=0)

    psd_filt_avg = np.mean(psd_filt_all, axis=0)


    # --- 3. Draw the Graph ---

    print("Drawing graph...")

    plt.figure(figsize=(12, 7))

    xlim_max = 60.0


    # Plot the averaged spectra
```

```
    plt.semilogy(f_raw, psd_raw_avg, label='Raw Global Average',
color='red', alpha=0.5)

    plt.semilogy(f_filt, psd_filt_avg, label='Filtered Global Average',
color='blue', linewidth=2)



    # Highlight Brainwave Bands

    plt.axvspan(1, 4, color='gray', alpha=0.1) # Delta

    plt.axvspan(4, 8, color='gray', alpha=0.15) # Theta

    plt.axvspan(8, 13, color='gold', alpha=0.2, label='Alpha Band
(Relaxation)')

    plt.axvspan(13, 30, color='blue', alpha=0.05, label='Beta Band
(Focus)')



    # Chart Styling

    plt.title("Global EEG Spectrum (Average of all 16 Channels)")

    plt.xlabel("Frequency (Hz)")

    plt.ylabel("Average Power (dB/Hz)")

    plt.xlim(0, xlim_max)

    plt.grid(True, which="both", ls="-", alpha=0.5)

    plt.legend()



    plt.show()
```

A.12 Αρχείο inspiret_multyvisual.py

```
import csv
import matplotlib.pyplot as plt
import numpy as np

# --- CONFIGURATION ---
num_samples_to_plot = 200  # 4 seconds of data (125Hz). Change this to see
more/less time!
num_channels = 16

print("--- INSPiRE 16-Channel Time Domain Visualizer ---")
session_num = input("Enter session number : ")
```

```
base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'
filt_filepath = base_path + 'filteredopenbcirecordings.csv'
#for testing
filt_filepath = base_path + 'upgraded_filtered_recordings.csv'



def load_all_channels(filename):
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Drawing massive 16-row graph... (this might take a second)")

    # --- 2. Create the 16-Row Figure ---
    # figsize=(15, 12) makes a tall window so the rows aren't squished
    fig, axes = plt.subplots(num_channels, 1, figsize=(15, 12),
sharex=True)
    fig.suptitle(f"Time Domain: Raw vs Filtered - All 16 Channels (First
{num_samples_to_plot} samples)", fontsize=16)

    # --- 3. Loop through and draw each row ---
    for i in range(num_channels):
        # Extract just the chunk of time we want to look at
        raw_ch = raw_data[:num_samples_to_plot, i]
        filt_ch = filt_data[:num_samples_to_plot, i]

        # Plot Red (Raw) behind Blue (Filtered)
        axes[i].plot(raw_ch, color='red', alpha=0.5, linewidth=1)
        axes[i].plot(filt_ch, color='blue', linewidth=1.5)

        # Format each row to look clean
        axes[i].set_ylabel(f"Ch {i+1}", rotation=0, labelpad=25,
verticalalignment='center', fontweight='bold')
        axes[i].grid(True, alpha=0.3)
        axes[i].set_yticks([]) # Hide the messy voltage numbers on the side
```

```
    # Put the time label only on the absolute bottom graph
    axes[-1].set_xlabel("Samples (Time)")

    # --- 4. Add a Master Legend at the top ---
    from matplotlib.lines import Line2D
    custom_lines = [Line2D([0], [0], color='red', alpha=0.5, lw=2),
                    Line2D([0], [0], color='blue', lw=2)]
    fig.legend(custom_lines, ['Raw Data (Fuzzy)', 'Filtered Data (Clean)'],
loc='upper right')

    # This squeezes the rows together perfectly so they fit the window
    plt.tight_layout()
    plt.subplots_adjust(top=0.93, hspace=0.1)
    plt.show()
```

A.13 Αρχείο inspiret_offline_filter.py

```
import csv
import matplotlib.pyplot as plt
import numpy as np

# --- CONFIGURATION ---
num_samples_to_plot = 200  # 4 seconds of data (125Hz). Change this to see
more/less time!
num_channels = 16

print("--- INSPiRE 16-Channel Time Domain Visualizer ---")
session_num = input("Enter session number : ")

base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'
filt_filepath = base_path + 'filteredopenbcirecordings.csv'
#for testing
filt_filepath = base_path + 'upgraded_filtered_recordings.csv'



def load_all_channels(filename):
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)
```

```
# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Drawing massive 16-row graph... (this might take a second)")

    # --- 2. Create the 16-Row Figure ---
    # figsize=(15, 12) makes a tall window so the rows aren't squished
    fig, axes = plt.subplots(num_channels, 1, figsize=(15, 12),
sharex=True)
    fig.suptitle(f"Time Domain: Raw vs Filtered - All 16 Channels (First
{num_samples_to_plot} samples)", fontsize=16)

    # --- 3. Loop through and draw each row ---
    for i in range(num_channels):
        # Extract just the chunk of time we want to look at
        raw_ch = raw_data[:num_samples_to_plot, i]
        filt_ch = filt_data[:num_samples_to_plot, i]

        # Plot Red (Raw) behind Blue (Filtered)
        axes[i].plot(raw_ch, color='red', alpha=0.5, linewidth=1)
        axes[i].plot(filt_ch, color='blue', linewidth=1.5)

        # Format each row to look clean
        axes[i].set_ylabel(f"Ch {i+1}", rotation=0, labelpad=25,
verticalalignment='center', fontweight='bold')
        axes[i].grid(True, alpha=0.3)
        axes[i].set_yticks([]) # Hide the messy voltage numbers on the side

    # Put the time label only on the absolute bottom graph
    axes[-1].set_xlabel("Samples (Time)")

    # --- 4. Add a Master Legend at the top ---
    from matplotlib.lines import Line2D
    custom_lines = [Line2D([0], [0], color='red', alpha=0.5, lw=2),
                    Line2D([0], [0], color='blue', lw=2)]
    fig.legend(custom_lines, ['Raw Data (Fuzzy)', 'Filtered Data (Clean)'],
loc='upper right')

    # This squeezes the rows together perfectly so they fit the window
    plt.tight_layout()
    plt.subplots_adjust(top=0.93, hspace=0.1)
    plt.show()
```

A.14 Αρχείο inspiret_present.py

```
import csv
import matplotlib.pyplot as plt
import numpy as np

# --- CONFIGURATION ---
num_samples_to_plot = 200  # 4 seconds of data (125Hz). Change this to see
more/less time!
num_channels = 16

print("--- INSPiRE 16-Channel Time Domain Visualizer ---")
session_num = input("Enter session number : ")
```

```
base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'
filt_filepath = base_path + 'filteredopenbcirecordings.csv'
#for testing
filt_filepath = base_path + 'upgraded_filtered_recordings.csv'



def load_all_channels(filename):
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    # Grab columns 1 through 16
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

# --- 1. Load the Data ---
print("Loading all 16 channels of data...")
raw_data = load_all_channels(raw_filepath)
filt_data = load_all_channels(filt_filepath)

if raw_data is not None and filt_data is not None:
    print("Drawing massive 16-row graph... (this might take a second)")

    # --- 2. Create the 16-Row Figure ---
    # figsize=(15, 12) makes a tall window so the rows aren't squished
    fig, axes = plt.subplots(num_channels, 1, figsize=(15, 12),
sharex=True)
    fig.suptitle(f"Time Domain: Raw vs Filtered - All 16 Channels (First
{num_samples_to_plot} samples)", fontsize=16)

    # --- 3. Loop through and draw each row ---
    for i in range(num_channels):
        # Extract just the chunk of time we want to look at
        raw_ch = raw_data[:num_samples_to_plot, i]
        filt_ch = filt_data[:num_samples_to_plot, i]

        # Plot Red (Raw) behind Blue (Filtered)
        axes[i].plot(raw_ch, color='red', alpha=0.5, linewidth=1)
        axes[i].plot(filt_ch, color='blue', linewidth=1.5)

        # Format each row to look clean
        axes[i].set_ylabel(f"Ch {i+1}", rotation=0, labelpad=25,
verticalalignment='center', fontweight='bold')
        axes[i].grid(True, alpha=0.3)
```

```
        axes[i].set_yticks([]) # Hide the messy voltage numbers on the side

    # Put the time label only on the absolute bottom graph
    axes[-1].set_xlabel("Samples (Time)")

    # --- 4. Add a Master Legend at the top ---
    from matplotlib.lines import Line2D
    custom_lines = [Line2D([0], [0], color='red', alpha=0.5, lw=2),
                    Line2D([0], [0], color='blue', lw=2)]
    fig.legend(custom_lines, ['Raw Data (Fuzzy)', 'Filtered Data (Clean)'],
loc='upper right')

    # This squeezes the rows together perfectly so they fit the window
    plt.tight_layout()
    plt.subplots_adjust(top=0.93, hspace=0.1)
    plt.show()
```

A.15 Αρχείο inspiret_presentrealtime.py

```
import csv
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.widgets import Slider
from brainflow.data_filter import DataFilter, FilterTypes
from matplotlib.ticker import LinearLocator, FormatStrFormatter

# --- CONFIGURATION ---
window_size = 500  # How many samples to show on screen at once (4 seconds
at 125Hz)
num_channels = 16
chunk_size = 130   # Simulate reading ~1 second of data at a time
sampling_rate = 125

print("--- INSPiRE 16-Channel Real-Time Simulator Visualizer ---")
session_num = input("Enter session number : ")

base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_filepath = base_path + 'openbcirecordings.csv'

# ========================================
# 1. YOUR LIVE FILTER FUNCTION (Cleaned)
# ========================================
def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125):
    history_buffer.extend(new_chunk)

    if len(history_buffer) > 600:
        history_buffer = history_buffer[-600:]

    np_buffer = np.array(history_buffer, dtype=np.float64)

    # 1. 50Hz Notch (Start: 48.0 Hz, Stop: 52.0 Hz)
    DataFilter.perform_bandstop(
        np_buffer, sampling_rate, 48.0, 52.0, 2,
FilterTypes.BUTTERWORTH.value, 0
    )

    # 2. 25Hz Hardware Spike Notch (Start: 24.5 Hz, Stop: 25.5 Hz)
```

```
    DataFilter.perform_bandstop(
        np_buffer, sampling_rate, 24.5, 25.5, 1,
FilterTypes.BUTTERWORTH.value, 0
    )

    # 3. Main Bandpass (Start: 5.0 Hz, Stop: 45.0 Hz)
    DataFilter.perform_bandpass(
        np_buffer, sampling_rate, 5.0, 45.0, 5,
FilterTypes.BUTTERWORTH.value, 0
    )

    chunk_length = len(new_chunk)
    clean_chunk = np_buffer[-chunk_length:]

    return clean_chunk.tolist(), history_buffer

# ========================================
# 2. LOAD RAW DATA
# ========================================
def load_all_channels(filename):
    """Loads columns 1 through 16 into a massive math grid."""
    data = []
    try:
        with open(filename, 'r') as f:
            reader = csv.reader(f, delimiter=';')
            next(reader) # Skip headers
            for row in reader:
                try:
                    row_vals = [float(row[i]) for i in range(1, 17)]
                    data.append(row_vals)
                except (ValueError, IndexError):
                    pass
    except FileNotFoundError:
        print(f"❌ Error: Could not find file -> {filename}")
        return None
    return np.array(data)

print("Loading raw CSV data...")
raw_data = load_all_channels(raw_filepath)

if raw_data is not None:
    total_samples = len(raw_data)
    print(f"Simulating real-time filtering for {total_samples} samples...")

    # ========================================
    # 3. THE REAL-TIME SIMULATION LOOP
    # ========================================
    history_buffers = [[] for _ in range(num_channels)]
    filtered_output = [[] for _ in range(num_channels)]

    for start_idx in range(0, total_samples, chunk_size):
        end_idx = start_idx + chunk_size
        live_chunk_all_channels = raw_data[start_idx:end_idx]

        if len(live_chunk_all_channels) == 0:
            break

        for ch in range(num_channels):
            ch_chunk = live_chunk_all_channels[:, ch].tolist()
```

```
            clean_ch_chunk, history_buffers[ch] =
filter_live_chunk(ch_chunk, history_buffers[ch])
            filtered_output[ch].extend(clean_ch_chunk)

    # Convert output back into a Numpy grid
    filt_data = np.array(filtered_output).T

    # --- THE DIPLOMA FIX: Crop the Filter Startup Ringing ---
    # We slice off the first 3 seconds (375 samples) of the violent math
swings.
    cutoff_samples = 3 * sampling_rate
    if len(filt_data) > cutoff_samples:
        filt_data = filt_data[cutoff_samples:, :]

    # Update total samples to match the newly cropped data length for the
slider
    total_samples_cropped = len(filt_data)
    # ------------------------------------------------------

    # ========================================
    # 4. SCROLLABLE VISUALIZATION
    # ========================================
    print("Drawing massive 16-row interactive graph...")

    fig, axes = plt.subplots(num_channels, 1, figsize=(15, 12),
sharex=True)
    fig.suptitle("Simulated Live Time Domain: Filtered Data ", fontsize=16)

    # Draw EVERY sample for each row
    for i in range(num_channels):
        filt_ch = filt_data[:, i]

        axes[i].plot(filt_ch, color='blue', linewidth=1.5)

        axes[i].set_ylabel(f"Ch {i+1}", rotation=0, labelpad=25,
verticalalignment='center', fontweight='bold')
        axes[i].grid(True, alpha=0.3)

        # --- THE CLEAN OVERLAP FIX: Dynamic Padding ---
        # 1. Find the exact minimum and maximum of this specific channel
        y_min, y_max = np.min(filt_ch), np.max(filt_ch)

        # (Safety check in case a channel is completely flat/dead)
        if y_max - y_min < 0.0001:
            y_min, y_max = -1.0, 1.0

        y_center = (y_max + y_min) / 2.0

        # 2. Add a 20% invisible margin above and below the wave
        margin = (y_max - y_min) * 0.20
        axes[i].set_ylim(y_min - margin, y_max + margin)

        # 3. Force exactly 3 ticks at the data's exact min, center, and max
        axes[i].set_yticks([y_min, y_center, y_max])

        # 4. Format them cleanly to 1 decimal place
        from matplotlib.ticker import FormatStrFormatter
        axes[i].tick_params(axis='y', labelsize=6)
        axes[i].yaxis.set_major_formatter(FormatStrFormatter('%.1f'))
```

```
        # ------------------------------------------


    axes[-1].set_xlabel("Samples (Time)")
    axes[0].set_xlim(0, window_size)

    # Make room at the bottom for the slider
    plt.subplots_adjust(top=0.93, bottom=0.15, hspace=0.25)

    # Create slider
    ax_slider = plt.axes([0.15, 0.05, 0.7, 0.03])
    time_slider = Slider(
        ax=ax_slider,
        label='Scroll Time ',
        valmin=0,
        valmax=max(0, total_samples_cropped - window_size),
        valinit=0,
        valstep=10
    )

    def update_window(val):
        start_point = int(time_slider.val)
        axes[0].set_xlim(start_point, start_point + window_size)
        fig.canvas.draw_idle()

    time_slider.on_changed(update_window)

    from matplotlib.lines import Line2D
    custom_lines = [Line2D([0], [0], color='blue', lw=2)]
    fig.legend(custom_lines, ['Live Filtered Data (Clean)'], loc='upper
right')

    plt.show()
```

A.16 Αρχείο inspiret_realtimefft_visualizer.py

```
import csv i

mport numpy as np

import matplotlib.pyplot as plt

 from brainflow.data_filter import DataFilter, FilterTypes

from scipy.signal import welch,butter, filtfilt

 import matplotlib.pyplot as plt

==========================================
1. YOUR EXACT FILTER FUNCTION
==========================================

def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125):
history_buffer.extend(new_chunk)

if len(history_buffer) > 600:
    history_buffer = history_buffer[-600:]

np_buffer = np.array(history_buffer, dtype=np.float64)
```

```
# 1. 50Hz Notch (Start: 48.0 Hz, Stop: 52.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 48.0, 52.0, 2, FilterTypes.BUTTERWORTH.value,
0
)

# 2. 25Hz Hardware Spike Notch (Start: 24.0 Hz, Stop: 26.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 24.5, 25.5, 1, FilterTypes.BUTTERWORTH.value,
0
)

# 3. Main Bandpass (Start: 2.0 Hz, Stop: 45.0 Hz)
DataFilter.perform_bandpass(
    np_buffer, sampling_rate, 5.0, 45.0, 5, FilterTypes.BUTTERWORTH.value,
0
)


global_filtered_segment = np.mean(simulated_realtime_output, axis=0)


chunk_length = len(new_chunk)

clean_chunk = np_buffer[-chunk_length:]

return clean_chunk.tolist(), history_buffer

========================================
2. RUNNING THE LOOP
========================================
```

if **name** == "**main**": print("Loading raw CSV data...")

```
session_num = input("Enter the session number to filter (e.g., 61): ")
raw_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/openbcirecordings.csv'

channel_14_data = []

try:
    with open(raw_path, 'r') as f:
        reader = csv.reader(f, delimiter=';')
        next(reader)
        for row in reader:
            try:
                channel_14_data.append(float(row[1]))
            except:
                pass
except FileNotFoundError:
    print("❌ Error: Could not find the CSV file. Check the path!")
    exit()

chunk_size = 130
total_samples = len(channel_14_data)
```

```
simulated_realtime_output = []
my_history_buffer = []

print("Running modular real-time loop...")
for start_idx in range(0, total_samples, chunk_size):
    end_idx = start_idx + chunk_size
    live_chunk = channel_14_data[start_idx:end_idx]

    if len(live_chunk) < chunk_size:
        break

    clean_live_chunk, my_history_buffer = filter_live_chunk(live_chunk,
my_history_buffer)
    simulated_realtime_output.extend(clean_live_chunk)

# ========================================
# 3. THE FREQUENCY DOMAIN (FFT) MATH
# ========================================
print("Calculating FFT (Frequency Space)...")
sampling_rate = 125

# Ensure both arrays are exactly the same length for a fair comparison
'''

'''
# 1. Grab the length of your 1D list
data_length = len(simulated_realtime_output)

# 2. Convert your 1D lists directly into numpy arrays
raw_segment = np.array(channel_14_data[:data_length])
filtered_segment = np.array(simulated_realtime_output)

# --- Croping the Filter Startup Ringing ---
# We slice off the first 3 seconds (375 samples) to hide the startup math.
cutoff_samples = 3 * sampling_rate

if len(filtered_segment) > (cutoff_samples + 250):
    # Chop off the first 3 seconds of data for both lines
    raw_segment = raw_segment[cutoff_samples:]
    filtered_segment = filtered_segment[cutoff_samples:]
# ---------------------------------------------------------


# 3. Apply Welch to the single channel
# Using nperseg=250 (If this throws an error, change it to 125!)
freqs, raw_fft = welch(raw_segment, fs=sampling_rate, nperseg=250,
detrend='linear')
_, filtered_fft = welch(filtered_segment, fs=sampling_rate, nperseg=250,
detrend='linear')

# ========================================
# 4. DRAWING THE GRAPH (Updated to match Script 1)
# ========================================
print("Drawing Frequency Graph...")

# --- 1. Set Up the Masterpiece Canvas ---
plt.figure(figsize=(12, 8))
plt.title("Real-Time EEG Spectrum ", fontsize=14)
```

```
# --- 2. Plot the Lines (Forcing the Logarithmic Y-Axis) ---
plt.semilogy(freqs, raw_fft, label="Raw Unfiltered Signal", color='red',
alpha=0.5)
plt.semilogy(freqs, filtered_fft, label="Clean Filtered Signal",
color='blue', linewidth=2)

# --- 3. Add the Colored Background Bands ---
# Grey for Delta/Theta (0 to 8 Hz)
plt.axvspan(0, 8, color='grey', alpha=0.15)

# Yellow for Alpha Band (8 to 13 Hz)
plt.axvspan(8, 13, color='gold', alpha=0.2, label="Alpha Band
(Relaxation)")

# Light Blue for Beta Band (13 to 30 Hz)
plt.axvspan(13, 30, color='mediumslateblue', alpha=0.1, label="Beta Band
(Focus)")

# --- 4. Format the Axes and Grid ---
plt.xlim(0, 60)  # Lock the X-axis so we only see up to 60 Hz
plt.xlabel("Frequency (Hz)", fontsize=12, fontweight='bold')
plt.ylabel("Average Power (dB/Hz)", fontsize=12, fontweight='bold')

# Add the grid background to match the offline graph
plt.grid(True, which="major", linestyle='-', alpha=0.6)
plt.grid(True, which="minor", linestyle=':', alpha=0.3)

# Place the legend
plt.legend(loc='lower center', fontsize=10)

# --- 5. Render ---
plt.tight_layout()

# If this is inside a live updating loop, use plt.pause(0.01) instead of
plt.show()
plt.show()
```

A.17 Αρχείο inspiret_ realtimefft_visualizermulty.py

```
import csv
import numpy as np
import matplotlib.pyplot as plt
from brainflow.data_filter import DataFilter, FilterTypes
from scipy.signal import welch
import csv
import numpy as np
import matplotlib.pyplot as plt
from brainflow.data_filter
import DataFilter, FilterTypes
from scipy.signal import welch,butter, filtfilt
import matplotlib.pyplot as plt
========================================
1. YOUR EXACT FILTER FUNCTION
========================================
def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125):
history_buffer.extend(new_chunk)
if len(history_buffer) > 600:
    history_buffer = history_buffer[-600:]
```

```
np_buffer = np.array(history_buffer, dtype=np.float64)

# 1. 50Hz Notch (Start: 48.0 Hz, Stop: 52.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 48.0, 52.0, 2, FilterTypes.BUTTERWORTH.value,
0
)

# 2. 25Hz Hardware Spike Notch (Start: 24.5 Hz, Stop: 25.5 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 24.5, 25.5, 1, FilterTypes.BUTTERWORTH.value,
0
)

# 3. Main Bandpass (Start: 5.0 Hz, Stop: 45.0 Hz)
DataFilter.perform_bandpass(
    np_buffer, sampling_rate, 5.0, 45.0, 5, FilterTypes.BUTTERWORTH.value,
0
)

chunk_length = len(new_chunk)
clean_chunk = np_buffer[-chunk_length:]

return clean_chunk.tolist(), history_buffer

========================================
2. RUNNING THE LOOP
========================================
if name == "main": print("Loading raw CSV data...")
session_num = input("Enter the session number to filter (e.g., 61): ")
raw_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/openbcirecordings.csv'

global_average_data = []

# Change this to 8 if you don't have the 16-channel Daisy module attached
NUM_CHANNELS = 16

try:
    with open(raw_path, 'r') as f:
        reader = csv.reader(f, delimiter=';')
        next(reader) # Skip the header row
        for row in reader:
            try:
                # Grab all EEG columns for this row (Columns 1 through
NUM_CHANNELS)
                channel_values = [float(row[i]) for i in range(1,
NUM_CHANNELS + 1)]

                # Calculate the average power across all electrodes for
this exact sample
                row_average = sum(channel_values) / len(channel_values)

                global_average_data.append(row_average)
            except:
                pass
except FileNotFoundError:
```

```
    print("❌ Error: Could not find the CSV file. Check the path!")
    exit()

chunk_size = 130
total_samples = len(global_average_data)

simulated_realtime_output = []
my_history_buffer = []

print("Running modular real-time loop...")
for start_idx in range(0, total_samples, chunk_size):
    end_idx = start_idx + chunk_size

    # Pull the live chunk from our new averaged array
    live_chunk = global_average_data[start_idx:end_idx]

    if len(live_chunk) < chunk_size:
        break

    clean_live_chunk, my_history_buffer = filter_live_chunk(live_chunk,
my_history_buffer)
    simulated_realtime_output.extend(clean_live_chunk)

# ==========================================
# 3. THE FREQUENCY DOMAIN (FFT) MATH
# ==========================================
print("Calculating FFT (Frequency Space)...")
sampling_rate = 125

# 1. Grab the length of your 1D list
data_length = len(simulated_realtime_output)

# 2. Convert your 1D lists directly into numpy arrays
raw_segment = np.array(global_average_data[:data_length])
filtered_segment = np.array(simulated_realtime_output)

# --- THE DIPLOMA FIX: Crop the Filter Startup Ringing ---
# We slice off the first 3 seconds (375 samples) to hide the startup math.
cutoff_samples = 3 * sampling_rate

if len(filtered_segment) > (cutoff_samples + 250):
    # Chop off the first 3 seconds of data for both lines
    raw_segment = raw_segment[cutoff_samples:]
    filtered_segment = filtered_segment[cutoff_samples:]
# ---------------------------------------------------------

# 3. Apply Welch to the single averaged channel
freqs, raw_fft = welch(raw_segment, fs=sampling_rate, nperseg=250,
detrend='linear')
_, filtered_fft = welch(filtered_segment, fs=sampling_rate, nperseg=250,
detrend='linear')

# ==========================================
# 4. DRAWING THE GRAPH
# ==========================================
print("Drawing Frequency Graph...")

# --- 1. Set Up the Masterpiece Canvas ---
plt.figure(figsize=(12, 8))
```

```
plt.title("Real-Time EEG Spectrum (Global Average)", fontsize=14)

# --- 2. Plot the Lines (Forcing the Logarithmic Y-Axis) ---
plt.semilogy(freqs, raw_fft, label="Raw Unfiltered Signal", color='red',
alpha=0.5)
plt.semilogy(freqs, filtered_fft, label="Clean Filtered Signal",
color='blue', linewidth=2)

# --- 3. Add the Colored Background Bands ---
# Grey for Delta/Theta (0 to 8 Hz)
plt.axvspan(0, 8, color='grey', alpha=0.15)

# Yellow for Alpha Band (8 to 13 Hz)
plt.axvspan(8, 13, color='gold', alpha=0.2, label="Alpha Band
(Relaxation)")

# Light Blue for Beta Band (13 to 30 Hz)
plt.axvspan(13, 30, color='mediumslateblue', alpha=0.1, label="Beta Band
(Focus)")

# --- 4. Format the Axes and Grid ---
plt.xlim(0, 60)  # Lock the X-axis so we only see up to 60 Hz
plt.xlabel("Frequency (Hz)", fontsize=12, fontweight='bold')
plt.ylabel("Average Power (dB/Hz)", fontsize=12, fontweight='bold')

# Add the grid background to match the offline graph
plt.grid(True, which="major", linestyle='-', alpha=0.6)
plt.grid(True, which="minor", linestyle=':', alpha=0.3)

# Place the legend
plt.legend(loc='lower center', fontsize=10)

# --- 5. Render ---
plt.tight_layout()
plt.show()
```

A.18 Αρχείο inspiret_ singlefiltersvisual.py

```
import csv
import matplotlib.pyplot as plt

# --- 1. THE FUNCTION DEFINITION ---
def get_channel_data(filename, channel_index):
    data = []
    with open(filename, 'r') as f:
        reader = csv.reader(f, delimiter=';')
        next(reader)
        for row in reader:
            try:
                data.append(float(row[channel_index]))
            except ValueError:
                pass
    return data

# --- 2. THE SCRIPT ---
```

```
print("Loading data...")
'''
raw_path =
'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session28/openbc
irecordings.csv'
filt_path =
'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session28/filter
edopenbcirecordings.csv'
'''

print("--- INSPiRE 16-Channel Time Domain Visualizer ---")
session_num = input("Enter session number : ")

base_path =
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/'
raw_path = base_path + 'openbcirecordings.csv'
filt_path = base_path + 'filteredopenbcirecordings.csv'



raw_fp1 = get_channel_data(raw_path, 1)
filt_fp1 = get_channel_data(filt_path, 1)

print("Drawing graph...")
plt.figure(figsize=(12, 6))

plt.plot(raw_fp1[:500], label='Raw Data (Fuzzy)', color='red', alpha=0.5,
linewidth=1)
plt.plot(filt_fp1[:500], label='Filtered Data (Smooth)', color='blue',
linewidth=2)

plt.title("Filter Verification: Fp1 Channel")
plt.xlabel("Samples")
plt.ylabel("Voltage")
plt.legend()
plt.grid(True)
plt.show()
```

A.19 Αρχείο inspiret_testrealtime.py

```
import csv
import numpy as np
import matplotlib.pyplot as plt from brainflow.data_filter
import DataFilter, FilterTypes
========================================
THE INDEPENDENT REAL-TIME FILTER FUNCTION
========================================
def filter_live_chunk(new_chunk, history_buffer, sampling_rate=125):
history_buffer.extend(new_chunk)
if len(history_buffer) > 600:
    history_buffer = history_buffer[-600:]

np_buffer = np.array(history_buffer, dtype=np.float64)


# 1. 50Hz Notch (Start: 48.0 Hz, Stop: 52.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 48.0, 52.0, 2, FilterTypes.BUTTERWORTH.value,
```

```
0
)

# 2. 25Hz Hardware Spike Notch (Start: 24.0 Hz, Stop: 26.0 Hz)
DataFilter.perform_bandstop(
    np_buffer, sampling_rate, 24.5, 25.5, 1, FilterTypes.BUTTERWORTH.value,
0
)

# 3. Main Bandpass (Start: 2.0 Hz, Stop: 45.0 Hz)
DataFilter.perform_bandpass(
    np_buffer, sampling_rate, 5.0, 45.0, 5, FilterTypes.BUTTERWORTH.value,
0
)


global_filtered_segment = np.mean(simulated_realtime_output, axis=0)


chunk_length = len(new_chunk)

clean_chunk = np_buffer[-chunk_length:]

return clean_chunk.tolist(), history_buffer

========================================
TESTING SCRIPT
========================================
if name == "main": print("Loading raw CSV data...") raw_path =
'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session28/openbc
irecordings.csv'
session_num = input("Enter the session number to filter (e.g., 61): ")

raw_path=
f'/home/raspberry/Desktop/Documents/INSPiRE/Data/panagiotis/session{session
_num}/openbcirecordings.csv'

channel_1_data = []

try:
    with open(raw_path, 'r') as f:
        reader = csv.reader(f, delimiter=';')
        next(reader)
        for row in reader:
            try:
                channel_1_data.append(float(row[1]))
            except:
                pass
except FileNotFoundError:
    print("❌ Error: Could not find the CSV file. Check the path!")
    exit()

chunk_size = 130
total_samples = len(channel_1_data)

# Initialize variables for the loop
simulated_realtime_output = []
my_history_buffer = []  # Our state memory
```

```
print("Running modular real-time loop...")
for start_idx in range(0, total_samples, chunk_size):
    end_idx = start_idx + chunk_size
    live_chunk = channel_1_data[start_idx:end_idx]

    if len(live_chunk) < chunk_size:
        break

    # --- THE MAGIC HAPPENS HERE ---
    # We pass the raw data and our memory to the function.
    # It hands back the clean data and the updated memory!
    clean_live_chunk, my_history_buffer = filter_live_chunk(live_chunk,
my_history_buffer)
    # ------------------------------

    simulated_realtime_output.extend(clean_live_chunk)

print("Drawing Verification Graph...")
plt.figure(figsize=(12, 6))
samples_to_view = chunk_size * 4

for i in range(1, 5):
    plt.axvline(x=chunk_size * i, color='gray', linestyle='--', alpha=0.5)

plt.plot(channel_1_data[:samples_to_view], label="Raw Data (Fuzzy)",
color='red', alpha=0.3)
plt.plot(simulated_realtime_output[:samples_to_view], label="Function
Output (Smooth)", color='blue', linewidth=2)

plt.title("Real time filter Function Verification")
plt.xlabel("Samples")
plt.ylabel("Voltage")
plt.legend()
plt.show()
```